



Universidad Técnica Estatal de Quevedo

Integrantes:

- CASANOVA MORANTE HECTOR MIGUEL
- CEDEÑO SALVATIERRA ORLANDO JESUS
- JAIME PESANTEZ ANDERSON DAVID
- LETURNE PLUAS JHON BYRON
- PIÑA PAREDES MIGUEL ANGEL
- TENECELA INTRIAGO ANGELICA MICHELLE

Asignatura:

GESTIÓN DE LA CONFIGURACIÓN DE SOFTWARE



Contenido

1. Sistema de facturación – Línea base (V1.0)	7
1.1. Metodología de Desarrollo en Cascada Adaptada al Proyecto de Gestión de Clientes, Productos y Facturas	7
1.2. Análisis.....	8
1.2.1. Definición del problema	8
1.2.2. Análisis del problema	8
1.2.3. Escritura de los requisitos	9
1.2.3.1. Requisitos funcionales	9
1.2.3.2. Requisitos No funcionales	9
1.2.4. Asignación de roles dentro del grupo de trabajo	10
1.3. Diseño.....	10
1.3.1. Historias de usuario.....	11
1.3.2. Diagrama de caso de uso.....	23
1.3.3. Diagrama de CU (Cliente)	25
1.3.4. Diagrama de CU (Producto)	25
1.3.5. Diagrama de CU (Factura).....	26
1.4. Plantillas de casos de uso	27
1.4.1. Caso de uso: Registrar cliente.....	27
1.4.2. Caso de uso: Modificar cliente	28
1.4.3. Caso de uso: Eliminar cliente	30
1.4.4. Caso de uso: Listar clientes.....	32
1.4.5. Caso de uso: Visualizar cliente	33
1.4.6. Caso de uso: Buscar cliente	35
1.4.7. Caso de uso: Registrar factura	36
1.4.8. Caso de uso: Listar facturas	39
1.4.9. Caso de uso: Visualizar factura.....	40
1.4.10. Caso de uso: Imprimir factura.....	41
1.4.11. Caso de uso: Registrar producto.....	42
1.4.12. Caso de uso: Listar productos	44
1.4.13. Caso de uso: Buscar productos.....	45
1.4.14. Caso de uso: Visualizar producto.....	47
1.4.15. Caso de uso: Modificar producto	48

1.4.16.	Caso de uso: Eliminar producto	50
1.5.	Diagrama de flujo de dato	52
1.6.	Diagrama de clases.....	52
1.7.	Diagrama de actividades	55
1.7.1.	Diagrama de actividad (Cliente)	55
1.7.2.	Diagrama de actividad (Productos).....	55
1.7.3.	Diagrama de actividad (Factura).....	57
1.8.	Diagramas de bases de datos.....	58
1.8.1.	Conceptual	58
1.8.2.	Logico	58
1.8.3.	Físico.....	59
1.9.	Construcción y pruebas	60
1.9.1.	Codificación.....	60
1.9.2.	Pruebas de integración	60
1.9.3.	Pruebas de rendimiento.....	60
1.9.4.	Creación del ejecutable	60
1.10.	Instalación.....	60
1.10.1.	Configuración de computadores.....	61
1.10.2.	Despliegue del proyecto	61
1.10.3.	Dependencias y hardware utilizado.....	61
1.11.	Mantenimiento.....	64
1.11.1.	Preventivo.....	64
1.11.2.	Correctivo.....	65
1.11.3.	Adaptativo	65
1.12.	GitHub Proyecto V1.0	66
2.	Proyecto de facturación (V1.1)	67
2.1.	Requisitos	70
2.2.	Diagramas de casos de uso.....	71
2.2.1.	Diagrama general	71
2.2.2.	Diagrama de CU (Cliente)	72
2.2.3.	Diagrama de CU (Empleado).....	73
2.2.4.	Diagrama de CU (Producto)	74
2.2.5.	Diagrama de CU (Factura).....	75

2.3.	Casos de uso	76
2.3.1.	Caso de uso Registrar Factura (Modificación por nueva versión).....	76
2.3.2.	Caso de uso Registrar empleado	78
2.3.3.	Caso de uso: Actualizar empleado	81
2.3.4.	Caso de uso: Log In	82
2.3.5.	Caso de uso: Cambiar contraseña	84
2.3.6.	Listar empleados	86
2.3.7.	Caso de uso: Visualizar empleado	87
2.3.8.	Caso de uso: Registrar producto (Modificación)	90
2.3.9.	Caso de uso: Modificar producto (Modificación).....	91
2.3.10.	Caso de uso: Eliminar producto (Modificación)	93
2.4.	Diagramas de actividades.....	95
2.4.1.	Diagrama de actividad (Empleados).....	95
2.4.2.	Diagrama de actividad (Cliente)	96
2.4.3.	Diagrama de actividad (Productos).....	97
2.4.4.	Diagrama de actividad (Factura).....	98
2.5.	Diagrama de flujo de datos.....	99
2.6.	Diagrama de clases.....	100
2.6.1.	Paquete DTO (Objeto de transferencia de datos)	100
2.6.2.	Paquete de Conexión con la base de datos.....	101
2.6.3.	Paquete de controladores	102
2.6.4.	Paquete DAO (Objeto de acceso a datos)	103
2.6.5.	Paquete de utensilios.....	105
2.7.	Diagramas de bases de datos.....	106
2.7.1.	Conceptual	106
2.7.2.	Lógico	107
2.7.3.	Físico.....	108
2.7.4.	Diagrama en SQL Server	109
3.	Sistema de facturación V1.2	110
3.1.	Solicitud de cambios en el software	111
3.2.	Orden de cambio de ingeniería (OCI)	113
3.3.	Línea de referencia	115
3.4.	Requisitos	116

3.5.	Planificación.....	116
3.6.	Diagramas de casos de uso.....	117
3.6.1.	Diagrama general	117
3.6.2.	Diagrama de CU (Producto)	119
3.7.	Casos de uso	120
3.7.1.	Caso de uso: Registrar proveedor	120
3.7.2.	Caso de uso: Modificar proveedor	122
3.7.3.	Caso de uso: Eliminar proveedor	124
3.7.4.	Caso de uso: Listar proveedores	126
3.7.5.	Caso de uso: Visualizar proveedor	128
3.7.6.	Caso de uso: Buscar proveedor.....	129
3.7.7.	Caso de uso: Crear productos (Modificación)	130
3.8.	Diagramas de actividades.....	133
3.8.1.	Diagrama de actividad (Proveedor)	133
3.8.2.	Diagrama de actividad (Productos).....	134
3.9.	Diagrama de flujo de datos.....	135
3.10.	Diagrama de clases	136
3.10.1.	Paquete DTO (Objeto de transferencia de datos).....	136
3.10.2.	Paquete de controladores.....	137
3.10.3.	Paquete DAO (Objeto de acceso a datos.....	139
3.10.4.	Paquete Utensilios	140
3.10.5.	Paquete conexión.....	141
3.11.	Diagramas de bases de datos	142
3.11.1.	Conceptual.....	142
3.11.2.	Lógico.....	142
3.11.3.	Físico	143
3.11.4.	Diagrama en SQL Server	144
3.12.	Pruebas unitarias.....	145
3.12.1.	Pruebas unitarias para validación de datos.....	145
3.12.2.	Pruebas unitarias de conexión	147
3.12.3.	Pruebas unitarias para entidad clientes.....	147
3.12.4.	Pruebas unitarias para entidad empleado	151
3.12.5.	Pruebas unitarias para entidad proveedor.....	156

3.12.6.	Pruebas unitarias para entidad productos	160
3.12.7.	Pruebas unitarias para entidad factura.....	163
3.13.	Repositorio.....	169

Figuras

Fig. 1	Planificación	8
Fig. 2	Diagrama general de caso de uso	24
Fig. 3	Diagrama de CU (Cliente).....	25
Fig. 4	Diagrama de CU (Producto)	25
Fig. 5	Diagrama de CU (Factura)	26
Fig. 6	Diagrama de clases	54
Fig. 7	Diagrama físico de base de dato SQLServer	59
Fig. 8	Diagrama físico de base de dato PowerDesigner	59

1. Sistema de facturación – Línea base (V1.0)

1.1. Metodología de Desarrollo en Cascada Adaptada al Proyecto de Gestión de Clientes, Productos y Facturas

En nuestro proyecto de desarrollo de software para la gestión de clientes, productos y facturas, hemos optado por la metodología de desarrollo en cascada debido a la claridad y estabilidad de los requisitos desde el inicio. Comenzaremos con la fase de análisis, donde se recopilarán y analizarán en detalle los requisitos del sistema. Esto implicará comprender las necesidades existentes, como el registro de clientes, la gestión de productos y la generación de facturas.

En la fase de diseño, crearemos una arquitectura detallada del sistema, identificando cómo se implementarán las funcionalidades requeridas. Desarrollaremos diagramas como el de clases, de flujo de datos y de actividades, centrándonos en la estructura de la base de datos, la interfaz de usuario y los procesos de negocio específicos del sistema.

Una vez completado el diseño, pasaremos a la fase de construcción y pruebas. Aquí, se llevará a cabo la implementación del sistema según lo diseñado, escribiendo y probando el código para garantizar que las funcionalidades cumplan con los requisitos establecidos.

Posteriormente, en la fase de instalación y mantenimiento, implementaremos el sistema, lo desplegaremos y presentarlo. Adicionalmente, nos enfocaremos en el mantenimiento continuo del software, realizando actualizaciones, correcciones de errores y mejoras según sea necesario en el transcurso del semestre.

La planificación desarrollada en Excel nos permitirá llevar un seguimiento detallado de cada fase del proyecto, asignando recursos, estableciendo fechas límite y monitoreando el progreso. Esto asegurará que cumplamos con los plazos y objetivos establecidos, manteniendo un enfoque organizado y eficiente en cada etapa del desarrollo del sistema.

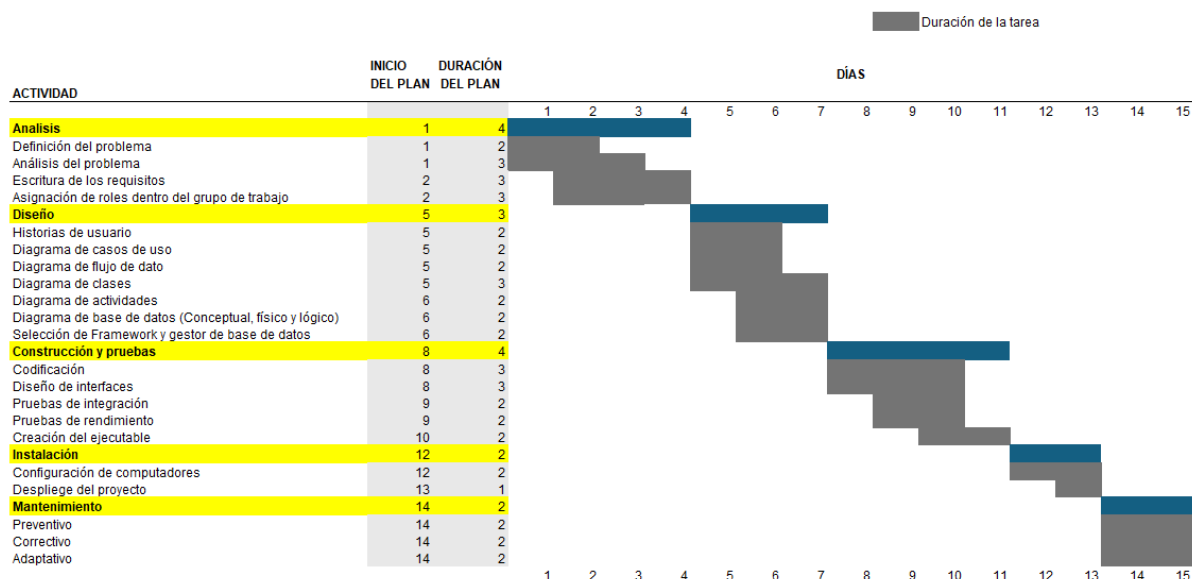


Fig. 1 Planificación

1.2. Análisis

Para la sección de análisis se aborda el problema, planteando las propiedades del mismo y las soluciones correspondientes para cada una de sus características a resolver. Esencialmente buscamos aclarar la forma en la que se va a dar solución al problema y como va a estar constituido el sistema.

1.2.1. Definición del problema

Desarrollar un software de facturación en el que permita la creación, lectura, edición y eliminación de: usuarios, productos y facturas. En cuanto a los usuarios solo se solicitará nombre y cedula, por parte de los productos, nombre y precio unitario y por cuanto a la factura se deberá calcular el total de compra sumando el precio de los productos y agregándole el IVA. Finalmente, el software permitirá generar la factura ya sea descargando en formato PDF o en vista previa.

1.2.2. Análisis del problema

El problema a tratar es crear un sistema de facturación, que cuente con entidades de clientes, productos y factura, su objetivo principal es que tenga las funcionalidades completas de CRUD, para cada una de sus entidades con su base de datos correspondiente para almacenar toda la información. Para darle solución a este problema se desarrollará una aplicación de escritorio con C# y se utilizará como sistema gestor de base de datos SQL Server.

1.2.3. Escritura de los requisitos

Los requisitos importantes que se han considerado son que se puedan desarrollar las funciones básicas de un CRUD tanto para los usuarios, productos y facturas. A continuación, se detalla la lista de los requisitos funcionales y no funcionales:

1.2.3.1. Requisitos funcionales

R1: Gestión de clientes:

- El sistema debe permitir la creación los clientes con los campos de: nombre y cédula.
- El sistema debe permitir la visualización de la lista de clientes.
- El sistema debe permitir la búsqueda de clientes por nombre y cédula.
- El sistema debe permitir la actualización de la información de un cliente existente.
- El sistema debe permitir la eliminación de un cliente del sistema.

R2: Gestión de productos:

- El sistema debe permitir la creación de productos con los campos de: nombre, precio unitario y el IVA correspondiente.
- El sistema debe permitir la visualización de la lista de productos.
- El sistema debe permitir la búsqueda de productos por nombre.
- El sistema debe permitir la actualización de la información de un producto existente.
- El sistema debe permitir la eliminación de un producto del sistema.

R3: Gestión de facturas:

- El sistema debe permitir la creación de una factura, en el que se selecciona a un cliente registrado y que incluya la lista de productos comprados, el total de la compra y el IVA.
- El sistema debe permitir la visualización de la lista de facturas.
- El sistema debe permitir la búsqueda de facturas por cliente.
- El sistema debe permitir la impresión de una factura en formato PDF.

1.2.3.2. Requisitos No funcionales

R4: Usabilidad:

- La interfaz de usuario debe ser intuitiva y fácil de usar, permitiendo a los usuarios realizar las operaciones CRUD con un mínimo de clics.

R5: Escalabilidad:

- El sistema debe ser escalable para soportar un aumento en el número de usuarios y la cantidad de datos sin afectar el rendimiento.

R6: Mantenimiento:

- El código fuente del sistema debe seguir las mejores prácticas de codificación y estar bien documentado para facilitar el mantenimiento.

1.2.4. Asignación de roles dentro del grupo de trabajo

Para la distribución de los roles dentro del equipo se analizaron las fortalezas y habilidades de cada integrante, los roles fueron asignados fueron: director, desarrollador, tester, analista de requerimientos.

INTEGRANTE	ROL
CASANOVA MORANTE HECTOR MIGUEL	Gerente de proyecto, desarrollador, tester
CEDEÑO SALVATIERRA ORLANDO JESUS	Gerente de proyecto, tester, desarrollador
JAIME PESANTEZ ANDERSON DAVID	Desarrollador, analista de requerimientos
LETURNE PLUAS JHON BYRON	Gerente de proyecto, desarrollador, tester
TENECELA INTRIAGO ANGELICA MICHELLE	Desarrollador, analista de requerimientos

1.3. Diseño

En esta fase la metodología de desarrollo se busca transformar los requisitos del software en una arquitectura y diseño detallado que guiarán la implementación del sistema. Esta fase es esencial porque define la estructura del sistema, especifica cómo los distintos componentes interactuarán entre sí y determina cómo se verá y funcionará la interfaz de usuario. Un diseño bien elaborado no solo facilita la construcción y el mantenimiento del software, sino que también asegura que el sistema cumpla con los requisitos y expectativas del cliente de manera eficiente y efectiva.

1.3.1. Historias de usuario

Las historias de usuarios en el desarrollo del software sirven para capturar de manera clara y concisa las necesidades y expectativas de los usuarios finales, respecto a las funcionalidades del sistema.

HISTORIA DE USUARIO No. 1			
ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Registrar mis clientes dentro del sistema.	Asegurar que toda la información necesaria esté disponible para las ventas	Alta
Criterio de selección			
• El sistema solo debe permitir el registro de los siguientes datos: apellidos, nombres, cédula). • En el campo cédula solo debe permitir números. • Validar el no ingreso de datos duplicados			
Diseño de interfaz			
NUEVO CLIENTE Cédula Ingrese cédula Apellidos Ingrese Apellidos Nombres Ingrese Nombres Registrar			

HISTORIA DE USUARIO No. 2			
ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Actualizar datos de clientes	La información este siempre actualizada y sean correctos.	Media
Criterio de selección			
• Debe ser capaz de buscar un cliente por nombre, apellidos y cédula. • En el campo cédula solo debe permitir números. • Debe permitir la actualización de los campos: cedula, nombre, apellidos. • Validar el no ingreso de datos duplicados. • Guardar cambios al accionar botón actualizar. • Debe salir un formulario de pregunta, para confirmar si desea actualizar la información del cliente.			
Diseño de interfaz			
			

CLIENTE REGISTRADO

Cédula

1207021633

Apellidos

Cedeño Salvatierra

Nombres

Orlando Jesús

Desea Actualizar los Datos
del Cliente?

SI

NO

ACTUALIZAR

HISTORIA DE USUARIO No. 3

ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Eliminar cliente del sistema.	La base de datos esté libre de registros innecesarios o incorrectos	Alta
Criterio de selección			
• Debe ser capaz de buscar un cliente por nombre, apellidos y cédula. • Debe haber una confirmación de eliminación para evitar errores.			

Diseño de interfaz

CLIENTES REGISTRADOS

10

Cédula	Apellidos	Nombres	
1207021633	CEDEÑO SALVATIERRA	ORLANDO JESÚS	 

CLIENTE REGISTRADO

Cédula

1207021633

Apellidos

Cedeño Salvatierra

Nombres

Orlando Jesús

ATENCIÓN
¿Desea eliminar el cliente?

SI

NO

ELIMINAR

HISTORIA DE USUARIO No. 4

ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Crear Producto	Tener registrados los productos a vender en mi empresa	Alta
Criterio de selección			
- El formulario debe incluir los campos: nombre del producto y precio unitario. - Todos los campos obligatorios deben ser validados. - El precio debe estar expresado en decimal y siempre será positivo.			
Diseño de interfaz			

NUEVO PRODUCTO

Nombre

Mouse Ramitech

Precio Unitario

5.00

AGREGAR

HISTORIA DE USUARIO No. 5

ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Actualizar la información de mis productos.	Mantener datos actualizados, permitiendo así una correcta facturación.	Alta
Criterio de selección			
• Debe permitir la búsqueda de productos por nombre. • Se debe mostrar el listado de productos. • Tener un botón de edición que envíe a otro formulario para actualizar los datos. • Existir botón Actualizar. • Aparecer un formulario de confirmación, si desea o no cambiar los datos.			
Diseño de interfaz			

PRODUCTOS REGISTRADOS



Nombre	Precio Unitario		
Mouse Ramitech	\$5.00		
Parlante Genius	\$17.00		
MousePad	\$4.00		
Audifono Xiaomi	\$25.00		
Teclado Meetion	\$11.00		
Forro Laptop	\$20.00		
Soporte Laptop	\$17.00		
Disco Solido 240 GB	\$17.00		

ACTUALIZAR PRODUCTO

Nombre

Audifono Xiaomi

Precio Unitario

30.00

ATENCIÓN

¿Desea actualizar los datos del producto?

SI

NO

AGREGAR

HISTORIA DE USUARIO No. 6			
ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Eliminar un producto del sistema.	Dismuir datos innecesarios y evitar la venta de productos descontinuados.	Alta
Criterio de selección			
• Debe permitir la búsqueda de productos por nombre. • Se debe mostrar el listado de productos. • Tener un botón de eliminación que envíe a otro formulario para confirmar la eliminación del producto. • Aparecer un formulario de confirmación, si desea o no cambiar los datos.			
Diseño de interfaz			

PRODUCTOS REGISTRADOS

🔍

Nombre	Precio Unitario		
Mouse Ramitech	\$5.00		
Parlante Genius	\$17.00		
MousePad			
Audifono Xiaomi			
Teclado Meetion	\$11.00		
Forro Laptop	\$20.00		
Soporte Laptop	\$17.00		
Disco Solido 240 GB	\$17.00		

ATENCIÓN

¿Desea eliminar el producto seleccionado?

SI

NO

HISTORIA DE USUARIO No. 7

ROL	DESEO	PARA QUE	PRIORIDAD
Yo como dueño de empresa.	Crear Factura	Registrar las ventas realizadas	Alta
Criterio de selección			
• Debe permitir buscar y seleccionar el cliente a facturar. • Seleccionar el producto o los productos a facturar. • Visualizar el detalle de la factura. • Imprimir y visualizar en vista previa de impresión.			

- El sistema debe validar que todos los campos requeridos estén completos antes de permitir la creación de la factura.
- Generar un número de factura único para cada nueva factura.
- Los productos se deben permitir seleccionar del listado registrado en el sistema.
- Los clientes se deben buscar y se desplegara una lista, con todos los clientes registrados, se selecciona el cliente deseado para la factura.
- Establecer un botón guardar, para crear la factura.

Diseño de interfaz

FACTURA 001

Productos

Cliente

Fecha: 26/5/2024

		1207021633 - CEDEÑO SALVATIERRA ORLANDO JESÚS			
Nombre	Precio Unitario	DETALLE DE FACTURA			
		CANTIDAD	NOMBRE	PRECIO U.	TOTAL
Mouse Ramitech	\$5.00	2	MousePad	\$3.48	\$6.96
Parlante Genius	\$17.00	1	Disco Solido 240 GB	\$14.78	\$14,78
MousePad	\$4.00				
Audifono Xiaomi	\$25.00				
Teclado Meetion	\$11.00				
Forro Laptop	\$20.00				
Soporte Laptop	\$17.00				
Disco Solido 240 GB	\$17.00				
				SUBTOTAL	\$21,74
				IVA 15%	\$3,26
				TOTAL A PAGAR	\$25.00

GUARDAR

HISTORIA DE USUARIO No. 8											
ROL	DESEO	PARA QUE	PRIORIDAD								
Yo como dueño de empresa.	Imprimir la Factura	Proporcionar un comprobante físico de la transacción al cliente	Alta								
Criterio de selección											
- El sistema debe permitir seleccionar la factura que se desea imprimir desde una lista de facturas generadas. - Mostrar una vista previa de la factura antes de imprimir. - Permitir seleccionar la impresora desde la cual se realizará la impresión.											
Diseño de interfaz											
FACTURAS REGISTRADAS  <table> <thead> <tr> <th>Nº</th><th>Nombre</th><th>Fecha</th><th></th></tr> </thead> <tbody> <tr> <td>1</td><td>Cedeño Salvatierra Orlando</td><td>26/5/2024 20:00:00</td><td></td></tr> </tbody> </table>				Nº	Nombre	Fecha		1	Cedeño Salvatierra Orlando	26/5/2024 20:00:00	
Nº	Nombre	Fecha									
1	Cedeño Salvatierra Orlando	26/5/2024 20:00:00									

FACTURA 001

Productos

Ciente

Fecha: 26/5/2024

Nombre	Precio Unitario
Mouse Ramitech	\$5.00
Parlante Genius	\$17.00
MousePad	\$4.00
Audifono Xiaomi	\$25.00
Teclado Meetion	\$11.00
Forro Laptop	\$20.00
Soporte Laptop	\$17.00
Disco Solido 240 GB	\$17.00

1207021633 - CEDEÑO SALVATIERRA ORLANDO JESÚS

DETALLE DE FACTURA

CANTIDAD	NOMBRE	PRECIO U.	TOTAL
2	MousePad	\$3.48	\$6.96
1	Disco Solido 240 GB	\$14.78	\$14.78
SUBTOTAL			\$21.74
IVA 15%			\$3.26
TOTAL A PAGAR			\$25.00

GUARDAR

Imprimir

7 hojas de papel

Destino

Microsoft Print to PDF

Orientación

Vertical

Horizontal

Páginas

Todo

Modo de color

Color

Más ajustes

Imprimir usando el diálogo del sistema...

Imprimir

Cancelar

1.3.2. Diagrama de caso de uso

Es fundamental para definir las interacciones entre los usuarios y el sistema. Este tipo de documento describe las acciones que los usuarios pueden realizar y cómo el sistema responde a estas acciones, proporcionando una visión clara y estructurada de los requisitos funcionales.

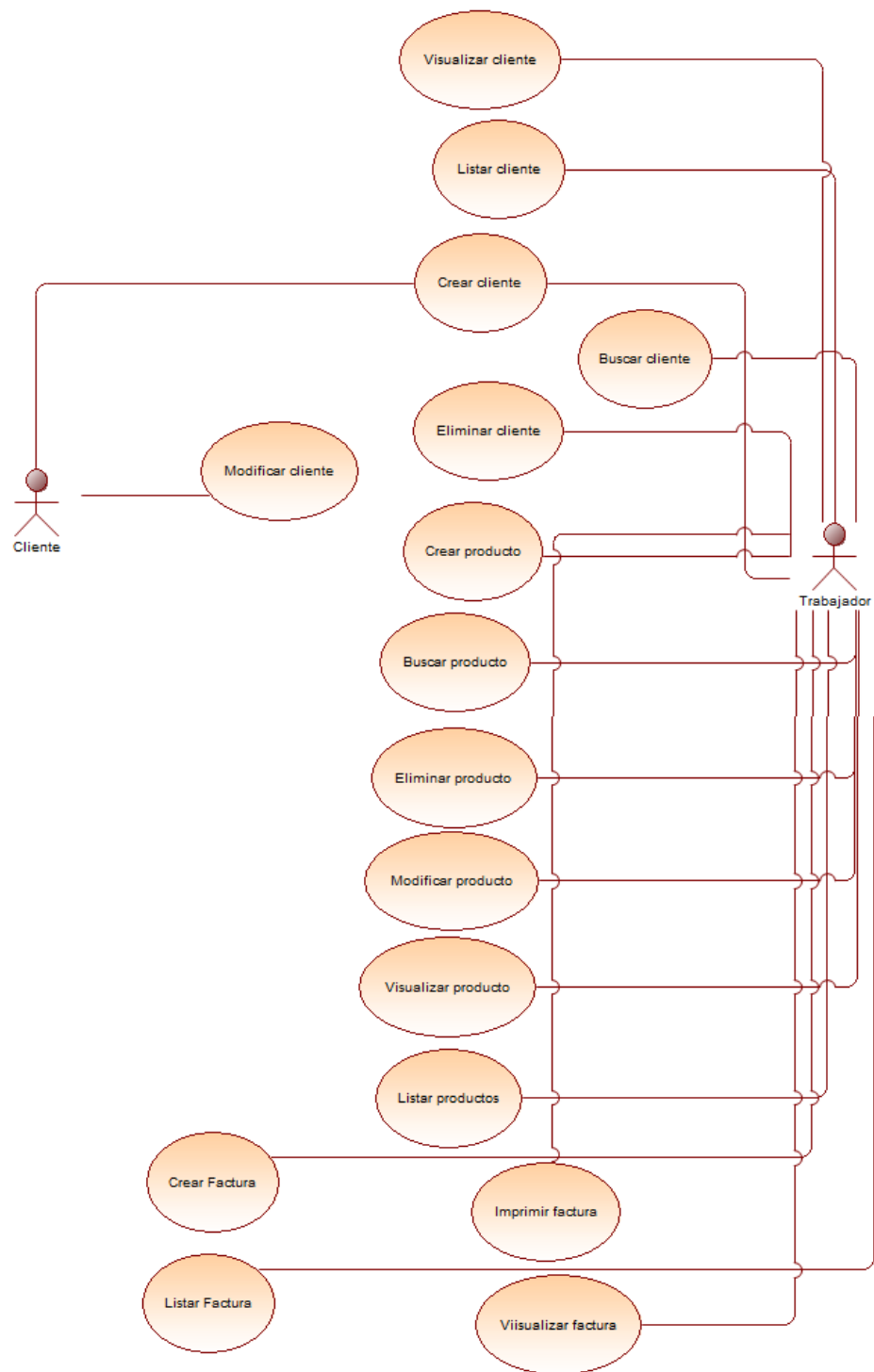


Fig. 2 Diagrama general de caso de uso

1.3.3. Diagrama de CU (Cliente)

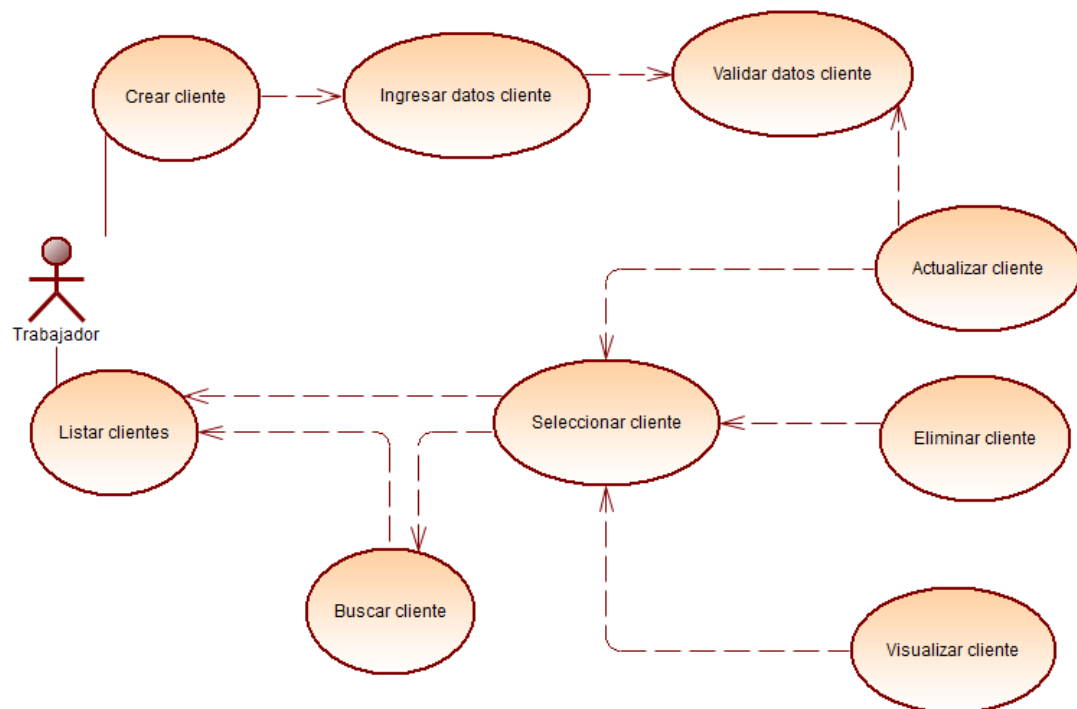


Fig. 3 Diagrama de CU (Cliente)

1.3.4. Diagrama de CU (Producto)

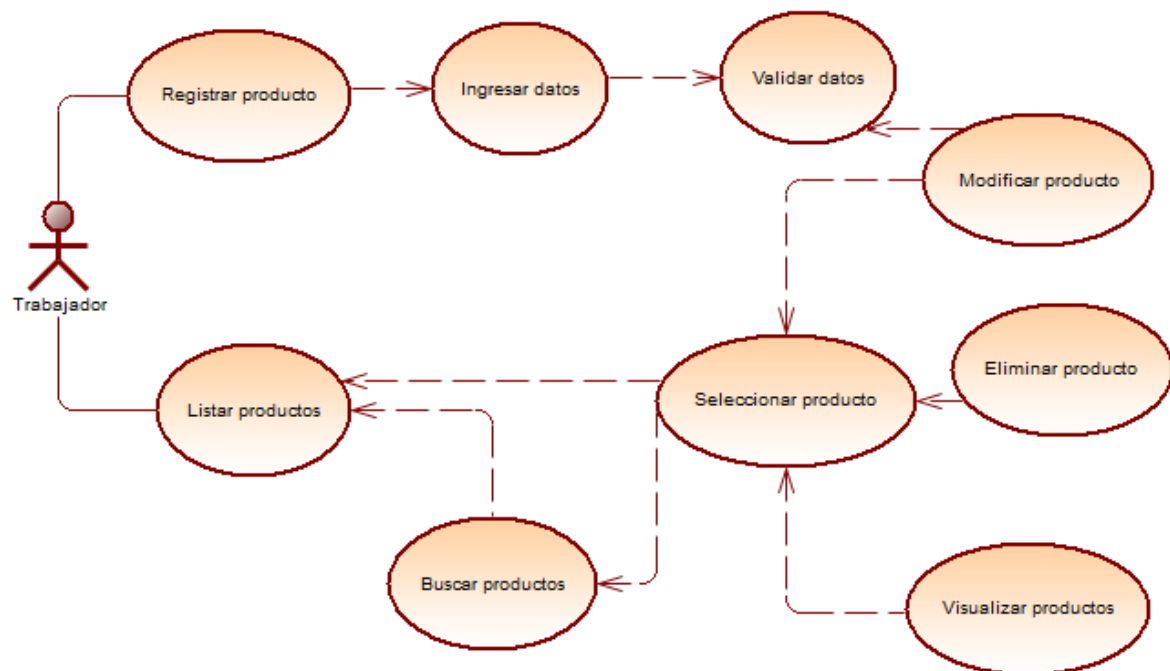


Fig. 4 Diagrama de CU (Producto)

1.3.5. Diagrama de CU (Factura)

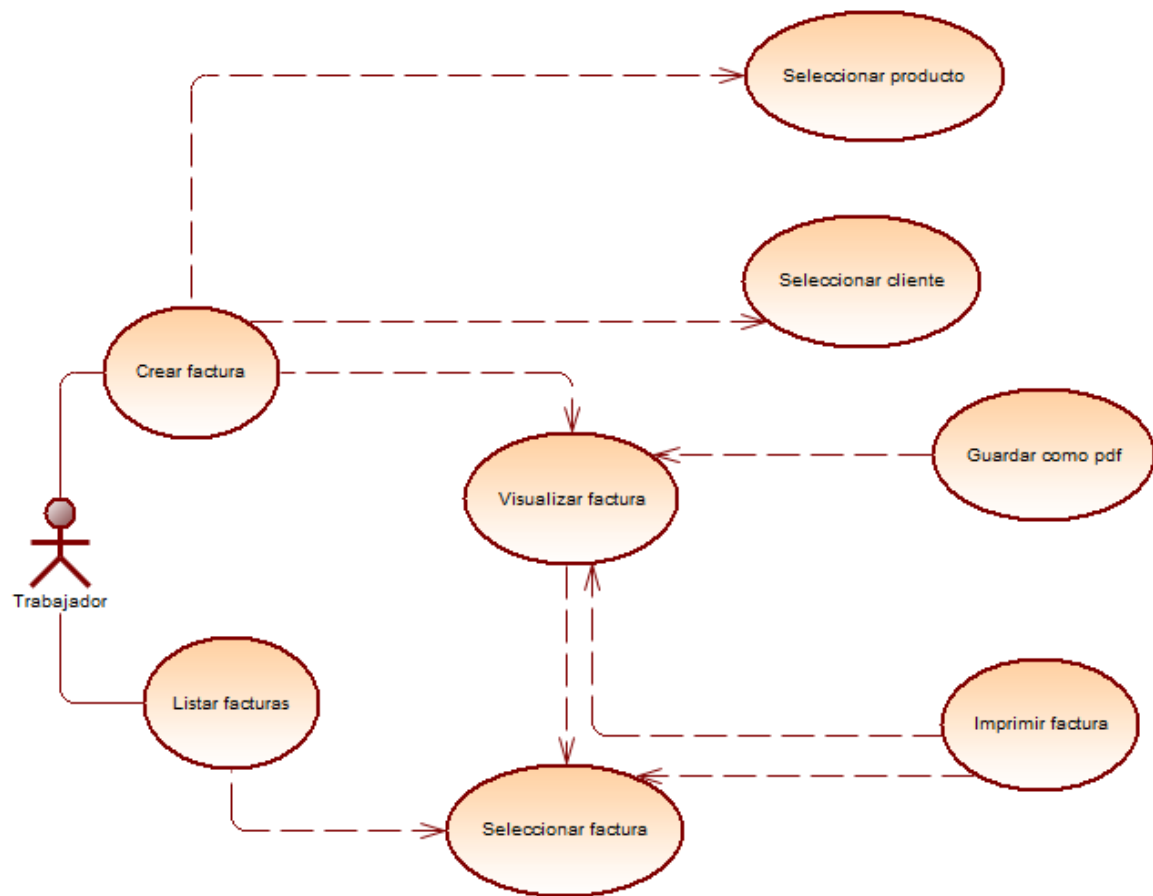


Fig. 5 Diagrama de CU (Factura)

1.4. Plantillas de casos de uso

1.4.1. Caso de uso: Registrar cliente

Caso de uso	Registrar cliente	Excepciones	
Iteración	1, última modificación: 26/05/2024	5.1. El sistema retroalimenta al usuario mostrando una notificación con los campos erróneos en el registro de clientes.	
		Prioridad	Alta
Actor principal	Trabajador.	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Registrar un cliente en el sistema con los datos brindados por el mismo.	Frecuencia de uso	Alta, debe implementarse de inmediato.
Precondiciones	El sistema debe estar funcionando correctamente, deben obtenerse los datos del cliente.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un cliente no existente en el sistema desea realizar compras de un producto y no está registrado.	Actores secundarios	Cliente
Escenario		Canales a los actores secundarios	
Actores	Sistema	Cliente: Interacción directa con el trabajador.	
1. El trabajador seleccione el botón “Crear clientes”			
2. El trabajador procede a pedirle los datos al cliente para registrarlo en el sistema.			

3. El cliente brinda los datos solicitados por el trabajador.		Asuntos pendientes 1. ¿Qué mecanismos son los correctos para proteger que los datos ingresados en la base de datos sean íntegros?
4. El trabajador presiona el botón “Guardar”.		
	5. El sistema verifica que los datos cumplen los parámetros necesarios (longitud, campos vacíos)	
	6. El sistema guarda los datos.	
	7. Muestra una notificación de “Datos guardados correctamente”.	
8. El trabajador notifica al cliente que sus datos fueron registrados.		

1.4.2. Caso de uso: **Modificar cliente**

Caso de uso	Modificar cliente	Excepciones
Iteración	1, última modificación: 26/05/2024	2.1. El cliente no está registrado en el sistema, por lo que procede a crearlo (ver el caso de uso Registrar cliente). 7.1. El sistema retroalimenta al usuario mostrando una notificación con los campos erróneos en la modificación de los datos del cliente.

		Prioridad	Media
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Actualizar los datos del cliente con los nuevos proporcionados por el mismo.	Frecuencia de uso	Media
Precondiciones	El sistema debe estar funcionando correctamente. Además, el cliente debe estar registrado en el sistema.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El cliente decide modificar sus datos en el sistema.	Actores secundarios	Cliente
Escenario		Canales a los actores secundarios	
Actores	Sistema	Cliente: Interacción directa con el trabajador.	
1. El cliente se acerca al trabajador para actualizar sus datos en el sistema.			
2. El trabajador busca al cliente por sus nombres o cédula.			
3. El trabajador presiona “Lista de clientes” (ver el caso de uso Listar clientes) y selecciona el cliente a modificar y presiona el botón “Editar”.			

4. El trabajador procede a pedirle los datos al cliente para modificarlos en el sistema.		
5. El trabajador presiona el botón “Guardar”.		Asuntos pendientes
6. El trabajador presiona el botón “Actualizar”.		1. ¿Qué mecanismos son los correctos para proteger que los datos modificados en la base de datos mantengan su integridad?
	7. El sistema verifica que los datos cumplen los parámetros necesarios (longitud, campos vacíos)	
	8. El sistema guarda los datos.	
	9. Muestra una notificación de “Datos modificados correctamente”.	
10. El trabajador notifica al cliente que sus datos fueron modificados.		

1.4.3. Caso de uso: Eliminar cliente

Caso de uso	Eliminar cliente	Excepciones	
Iteración	1, última modificación: 26/05/2024	3.1. El cliente no está registrado en el sistema, por lo que procede a crearlo (ver el caso de uso Registrar cliente).	
		5.1. El cliente no puede ser eliminado ya el registro correspondiente está relacionada con otras tablas.	
		Prioridad	Media
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Eliminar (desactivar) el cliente del sistema.	Frecuencia de uso	Baja
Precondiciones	El sistema debe estar funcionando correctamente. Además, el cliente debe estar registrado en el sistema.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un cliente desea darse de baja del sistema.	Actores secundarios	Cliente
Escenario		Canales a los actores secundarios	
Actores	Sistema	Cliente: Interacción directa con el trabajador.	
1. El cliente se acerca al trabajador para eliminar sus datos del sistema.			
2. El trabajador presiona el botón “Listar clientes” (ver el caso de uso Listar clientes).			
3. El trabajador busca al cliente			

por sus nombres o cédula		
4. El trabajador selecciona el cliente y presiona el botón “Eliminar”.		Asuntos pendientes
	5. El sistema procede a eliminar cliente del sistema.	
	6. Muestra una notificación de “Cliente eliminado correctamente”.	
	7. El sistema redirige al trabajador a la lista de clientes.	
8. El trabajador notifica al cliente que ha sido dado de baja del sistema.		

1.4.4. Caso de uso: Listar clientes

Caso de uso	Listar clientes	Excepciones	
Iteración	1, última modificación: 26/05//2024	3.1. El sistema no retorna ningún dato.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Listar los clientes registrados en el sistema.	Frecuencia de uso	Alta

Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador accede a la lista de clientes registrados.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador presiona el botón “Listar clientes”.			
	2. El sistema consulta a la base de datos los registros de la tabla cliente.		
	3. El sistema retorna los datos de la base de datos.		
	4. El sistema muestra los datos de los clientes en forma de lista.	Asuntos pendientes	
5. El usuario visualiza todos los datos cargados.		1. ¿Cómo debería diferenciarse los clientes dados de baja del sistema de los activos?	

1.4.5. Caso de uso: Visualizar cliente

Caso de uso	Visualizar cliente	Excepciones	
Iteración	1, última modificación: 26/05//2024	3.1. El sistema no retorna ningún dato.	
		Prioridad	Alta

Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Visualizar un cliente específico que haya sido seleccionado por el trabajador.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador desea ver los datos detallados de un cliente.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador presiona el botón “Listar clientes” (ver el caso de uso Listar clientes).			
2. El trabajador selecciona un cliente específico y presiona el botón “Ver”.	.		
	3. El sistema consulta a la base de datos los registros solicitados del cliente seleccionado.		
	4. El sistema muestra los datos detallados del cliente.	Asuntos pendientes	
5. El usuario visualiza los datos		1. ¿Cómo debería diferenciarse los clientes dados de baja del sistema de los activos?	

del cliente seleccionado.		
------------------------------	--	--

1.4.6. Caso de uso: Buscar cliente

Caso de uso	Buscar cliente	Excepciones	
Iteración	1, última modificación: 26/05//2024	3.1. El sistema no retorna ningún dato.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Buscar un cliente o clientes de interés por el trabajador.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador desea buscar dentro de la lista de clientes alguno que cumpla con características definidas por él.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador presiona el botón “Listar clientes” (ver el caso de uso Listar clientes).			
2. El trabajador ingresa en un cuadro de texto	.		

que desea buscar en la lista de clientes.		
	3. El sistema consulta a la base de datos los clientes que cumplan con las especificaciones de búsqueda.	
	4. El sistema muestra los clientes que cumplen las características de búsqueda.	Asuntos pendientes
5. El usuario visualiza la lista de clientes.		1. ¿Cómo debería diferenciarse los clientes dados de baja del sistema de los activos?

1.4.7. Caso de uso: Registrar factura

Caso de uso	Registrar factura	Excepciones
Interacción	1, última modificación: 26/05//2024	4.1. No existe el producto solicitado. 6.1. Posibles cálculos incorrectos si los precios de los productos no son coherentes. 7.1. No existen productos seleccionados para la canasta del cliente. 10.1. El reporte no funciona correctamente debido a una mal configuración.

		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Registrar la factura de los productos que fueron seleccionados por un cliente específico.	Frecuencia de uso	Alta
Precondiciones	- El sistema debe estar funcionando correctamente. - El cliente al que va dirigida la factura debe estar registrado en el sistema. - Deben existir productos registrados en el sistema.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un cliente desea realizar la compra de una serie de productos.	Actores secundarios	Cliente
Escenario		Canales a los actores secundarios	
Actores	Sistema	Cliente: Interacción directa con el trabajador.	
1. El cliente debe mencionar al trabajador los productos que desea.			
2. El trabajador accede a la sección de facturas presionando el botón con el mismo nombre.			

3. El trabajador presiona el botón “Seleccionar cliente” (ver el caso de uso Listar clientes) y selecciona al cliente correspondiente.		
4. El trabajador presiona el botón “Seleccionar productos” (ver el caso de uso Listar productos)		
5. El trabajador busca el/los producto(s) que el cliente desea.		Asuntos pendientes
6. El trabajador agrega el producto a la canasta del cliente.		1. ¿Debería el sistema preguntar al trabajador si desea visualizar la factura o simplemente mostrarla?
	7. El sistema calcula el IVA, subtotal y el total a pagar.	
8. El trabajador presiona el botón “Factura productos”		
	9. El sistema envía los productos a la base de datos y calcula tanto el IVA, total entre otros valores.	

	10. El sistema guarda el encabezado de la factura (datos del cliente), el detalle (productos) y los totales.	
	11. El sistema muestra una pre visualización al trabajador de la factura generada.	
12. El trabajador decide si desea imprimir la factura generada por el sistema.		

1.4.8. Caso de uso: Listar facturas

Caso de uso	Listar facturas	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. No existen facturas.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 4to incremento
Objetivo en contexto	Listar las facturas registradas en el sistema.	Frecuencia de uso	Media
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador desea visualizar las facturas que han sido registradas en el sistema.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	

Actores	Sistema	
1. El trabajador entra al apartado de Facturas.		
	2. El sistema lista todas las facturas registradas a la fecha.	N/A
		Asuntos pendientes

1.4.9. Caso de uso: Visualizar factura

Caso de uso	Visualizar factura	Excepciones	
Iteración	1, última modificación: 26/05/2024	1.1. No existen facturas registradas en el sistema. Por lo tanto, no es posible realizar una visualización.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Pre visualizar las facturas generadas por el sistema.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente	Canal al actor	A través de un computador con conexión a internet.
Disparador	El trabajador debe haber seleccionado una factura específica.	Actores secundarios	N/A

Escenario		Canales a los actores secundarios
Actores	Sistema	N/A
1. El trabajador accede a la lista de Facturas (ver caso de uso Listar facturas) y presione el botón con el ícono representativo a “visualizar”		
	2. El sistema muestra una la factura seleccionada por el trabajador.	
		Asuntos pendientes
		N/A

1.4.10. Caso de uso: Imprimir factura

Caso de uso	Imprimir factura	Excepciones	
Iteración	1, última modificación: 26/05/2024	1.1. No existen facturas registradas en el sistema. Por lo tanto, no es posible realizar una visualización.	
		4.1. No existe ninguna impresora disponible para imprimir el reporte.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Imprimir el reporte de una factura específica	Frecuencia de uso	Alta

	con su encabezado y detalle.		
Precondiciones	El sistema debe estar funcionando correctamente	Canal al actor	A través de un computador con conexión a internet.
Disparador	El trabajador debe haber seleccionado una factura.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la lista de Facturas (ver caso de uso Listar facturas), selecciona una y presione el botón con el ícono representativo a “visualizar”			
	2. El sistema muestra la factura seleccionada por el trabajador.		
3. El trabajador presiona el botón ícono para imprimir el reporte.			
	4. El sistema envía a imprimir el reporte de la factura seleccionada.	Asuntos pendientes	
		N/A	

1.4.11. Caso de uso: Registrar producto

Caso de uso	Registrar producto	Excepciones
--------------------	--------------------	--------------------

Interacción	1, última modificación: 26/05//2024	4.1. Los datos ingresados no son válidos. Por lo tanto, se retroalimenta al usuario de los errores.	
		5.1. Al registrar los datos en la base de datos existe algún error que evita la inserción.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al trabajador registrar productos al sistema.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente	Canal al actor	A través de un computador con conexión a internet.
Disparador	El trabajador desea registrar un producto en el sistema.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la opción “Registrar producto” en la interfaz.			
	2. El sistema muestra un formulario para el ingreso de los datos del producto.		
3. El trabajador ingresa los datos solicitados en el formulario.			
		Asuntos pendientes	

	4. El sistema valida los datos ingresados (longitud y tipos).	1. ¿Debería permitirse utilizar diferentes tipos de moneda?
	5. El sistema registra el producto en la base de datos.	
	6. El sistema notifica al trabajador que el producto ha sido registrado correctamente.	

1.4.12. Caso de uso: Listar productos

Caso de uso	Listar productos	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. El sistema no muestra ningún elemento puesto que no existen productos registrados.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Mostrar en forma de lista los productos que han sido registrados en el sistema.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El trabajador desea visualizar una lista de todos los productos registrados.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la opción			

“Listar productos” en la interfaz.		
	2. El sistema accede a la base de datos para recuperar la lista de productos registrados.	Asuntos pendientes 1. ¿Las opciones por cada producto deberían mostrarse en forma de íconos o en una lista desplegable?
	3. El sistema muestra la lista de los productos registrados a la fecha.	

1.4.13. Caso de uso: Buscar productos

Caso de uso	Buscar productos	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. El sistema no muestra ningún elemento puesto que no existen productos registrados.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al usuario buscar productos que cumplan con algunas características.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador con conexión a internet.

Disparador	El trabajador desea visualizar una lista de todos los productos registrados.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la opción “Listar productos” en la interfaz.			
	2. El sistema accede a la base de datos para recuperar la lista de productos que cumplen con las características de búsqueda.		
		Asuntos pendientes	
	3. El sistema muestra la lista de los productos que cumplen las características.	N/A	

1.4.14. Caso de uso: Visualizar producto

Caso de uso	Visualizar producto	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. La lista de productos no ha sido actualizada y el producto seleccionado ha sido eliminado. Por lo tanto, al visualizarlo se produce un error.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al trabajador visualizar los datos de un producto específico.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El trabajador desea visualizar los datos de un producto.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la opción “Ver” en la lista de productos desplegada (ver el caso de uso Listar productos).			
	2. El sistema realiza una consulta a la base de datos para obtener el producto seleccionado.		

	3. El sistema muestra los datos detallados del producto seleccionado por el trabajador.	Asuntos pendientes
		1. ¿Qué medidas deberían considerarse para evitar errores en el caso especificado en la excepción 2.1?

1.4.15. Caso de uso: Modificar producto

Caso de uso	Modificar producto	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. La lista de productos no ha sido actualizada y el producto seleccionado ha sido eliminado. Por lo tanto, al editarlo se produce un error. 5.1. Los datos ingresados no son válidos. Por lo tanto, se retroalimenta al usuario de los errores. 6.1. Al registrar los datos en la base de datos existe algún error que evita la modificación del producto.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al trabajador modificar los datos de un producto específico.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador con conexión a internet.

Disparador	El trabajador desea modificar los datos de un producto.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la opción “Modificar” en la lista de productos desplegada (ver el caso de uso Listar productos).			
	2. El sistema realiza una consulta a la base de datos para obtener el producto seleccionado.		
	3. El sistema muestra los datos detallados del producto seleccionado por el trabajador.		
4. El trabajador modifica los datos de interés permitidos por el sistema.			
	5. El sistema valida los datos ingresados (longitud y tipos).		
	6. El sistema modifica los datos del producto	Asuntos pendientes 1. ¿Qué medidas deberían considerarse para evitar errores en el caso especificado en la excepción 2.1?	

	seleccionado en la base de datos.	
	7. El sistema notifica al trabajador que el producto ha sido modificado correctamente.	

1.4.16. Caso de uso: Eliminar producto

Caso de uso	Eliminar producto	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. El producto seleccionado no puede ser eliminado por que está asociado con los registros de otra tabla.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al trabajador eliminar un producto específico.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El trabajador desea eliminar un producto.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la opción			

“Eliminar” en la lista de productos desplegada (ver el caso de uso Listar productos).		
	2. El sistema solicita confirmación para la eliminar el producto seleccionado.	
	3. El sistema elimina el registro del producto de la base de datos.	
	4. El sistema actualiza la lista de productos.	
		Asuntos pendientes
		1. ¿Qué mecanismos podrían aplicarse para que todos los usuarios mantengan la lista de productos actualizada?

1.5. Diagrama de flujo de dato

El desarrollo del diagrama de flujo de datos es crucial para visualizar cómo la información se mueve a través del sistema, representando gráficamente las entradas y salidas de datos, las fuentes y destinos, y los procesos que transforman los datos. La exposición de este diagrama facilita la identificación de puntos de integración entre diferentes módulos, ayuda a detectar inconsistencias y omisiones en el manejo de datos, y mejora la comunicación entre los miembros del equipo de desarrollo y otros interesados.

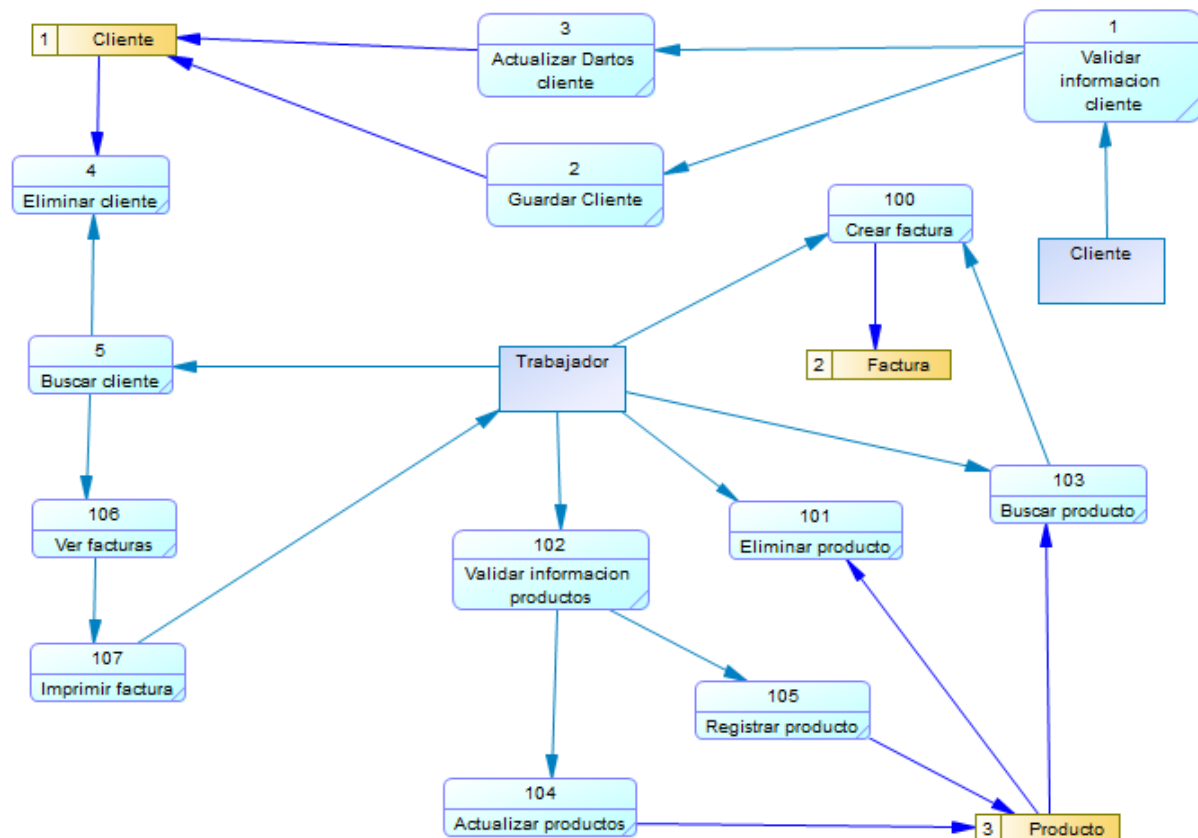


Fig. 5 Diagrama de flujo de datos V1.0

1.6. Diagrama de clases

Este artefacto muestra la estructura estática del sistema, mostrando las clases, sus atributos, métodos y las relaciones entre ellas, proporciona una visión clara de cómo los diferentes elementos del sistema están organizados y cómo interactúan entre sí, facilitando el diseño orientado a objetos y ayuda a identificar y definir las responsabilidades de cada clase.

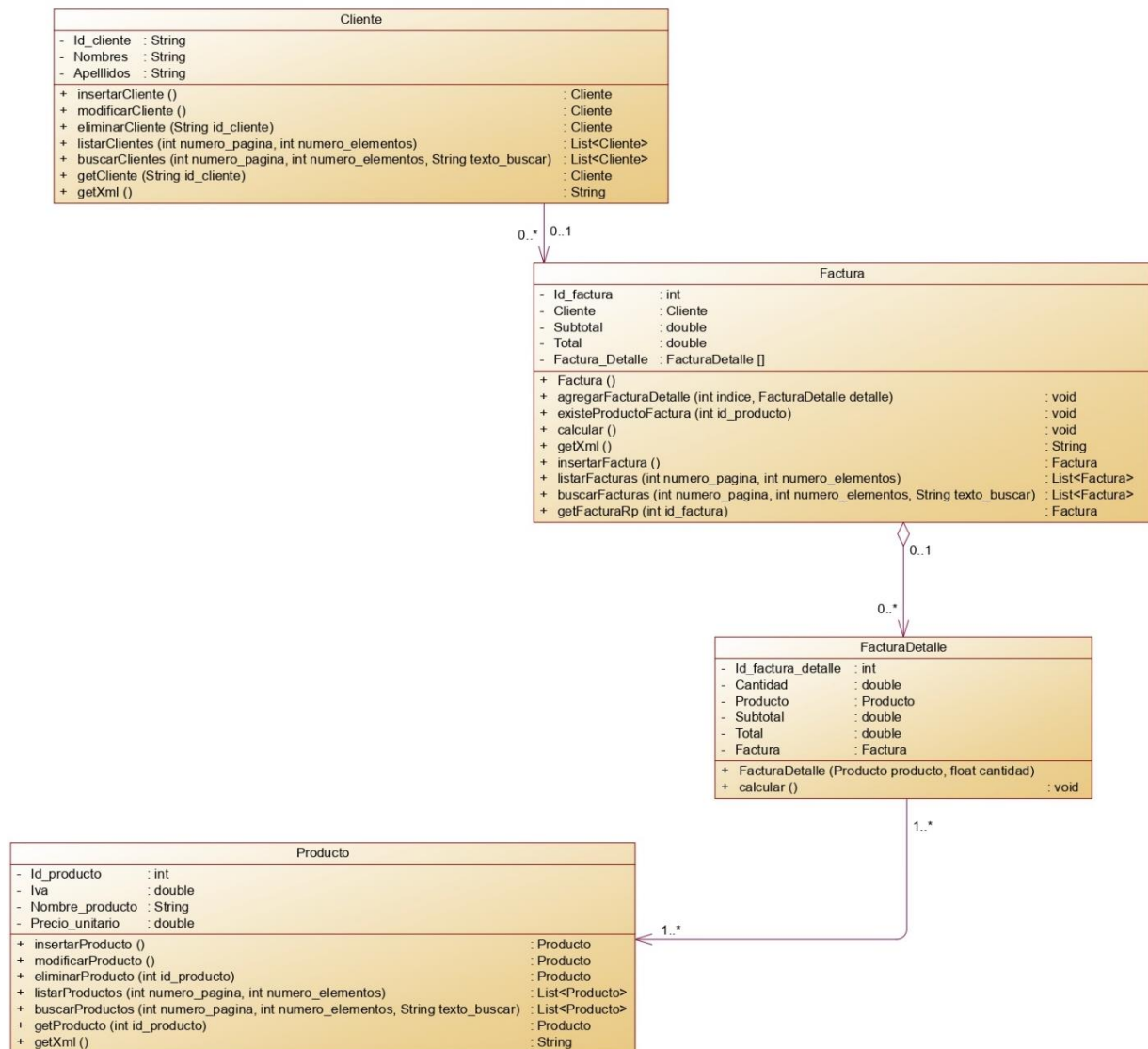


Fig. 6. Diagrama de clases general V1.0

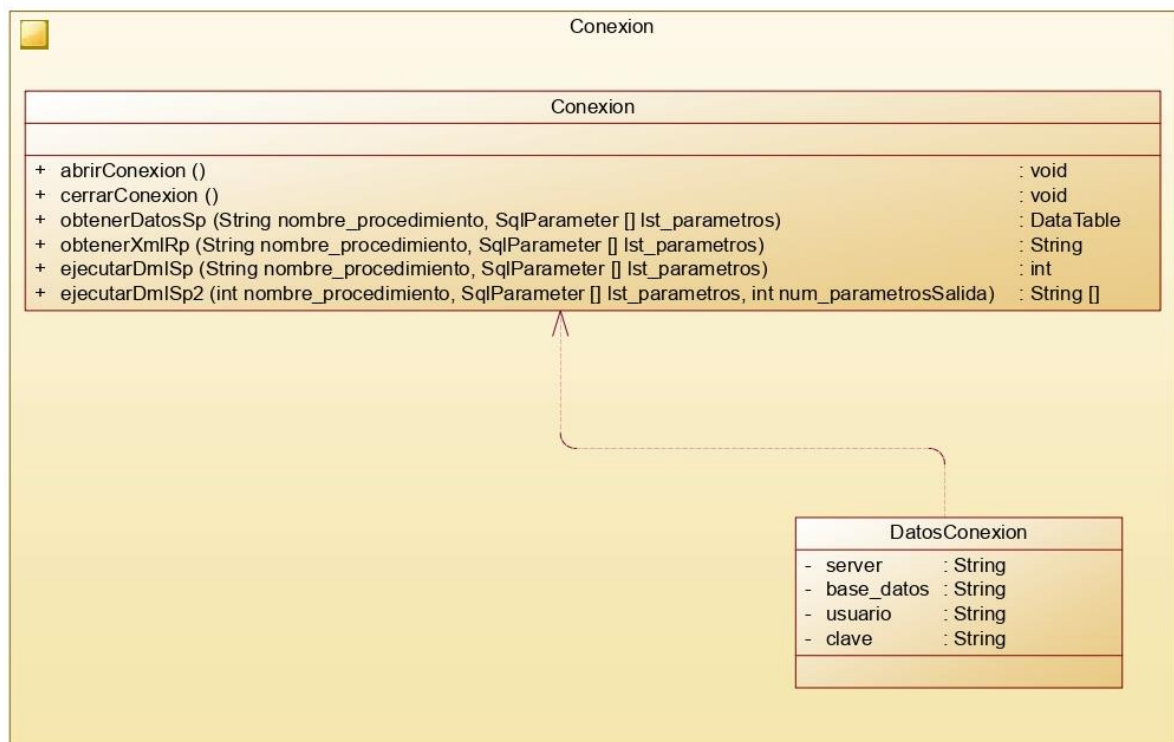


Fig. 7 Diagrama de clases: paquete de conexión con la base de datos V1.0

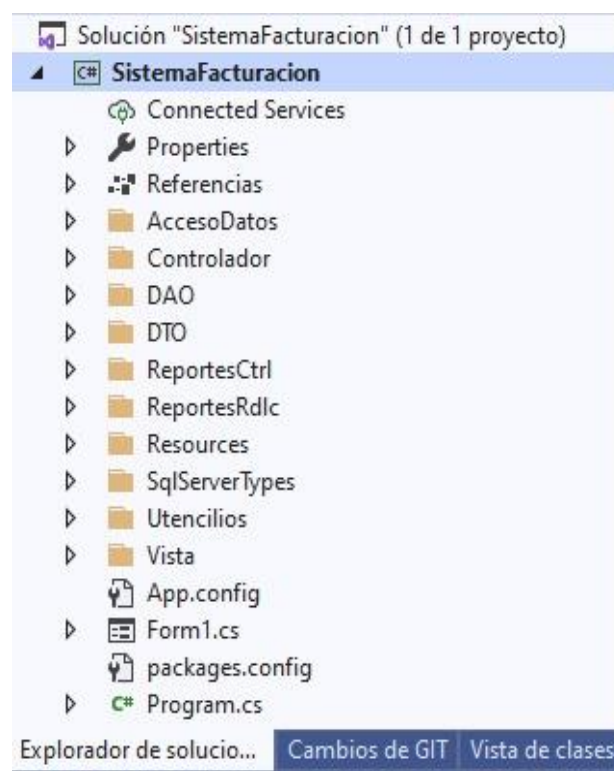


Fig. 7 Arquitectura del proyecto V1.0

1.7. Diagrama de actividades

En este artefacto se modela el flujo de trabajo y los procesos dinámicos dentro del sistema, en él se ilustra las secuencias de actividades y decisiones que los usuarios o el sistema realizan para completar tareas específicas.

1.7.1. Diagrama de actividad (Cliente)

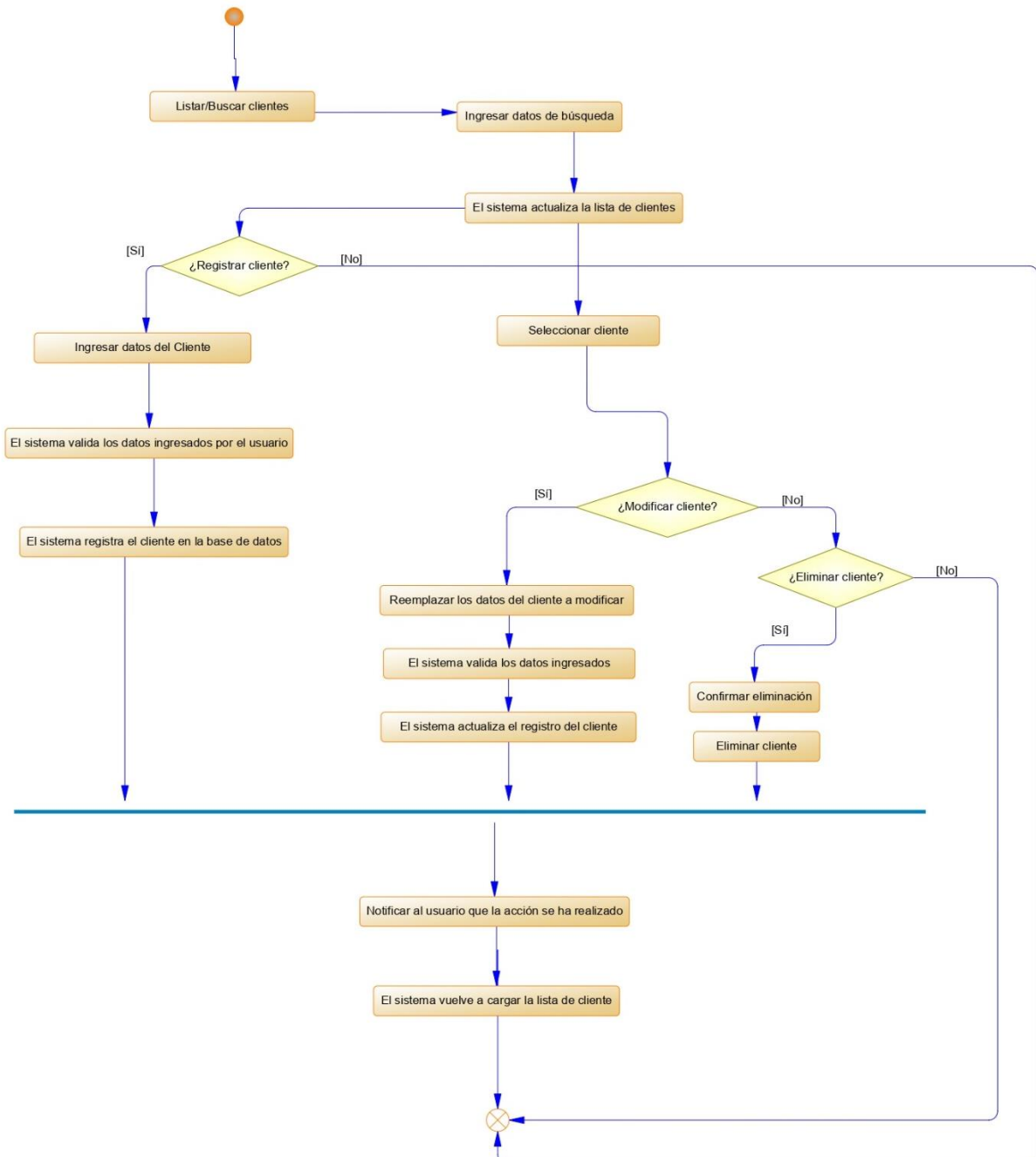


Fig. 8 Diagrama de actividad cliente V1.0

1.7.2. Diagrama de actividad (Productos)

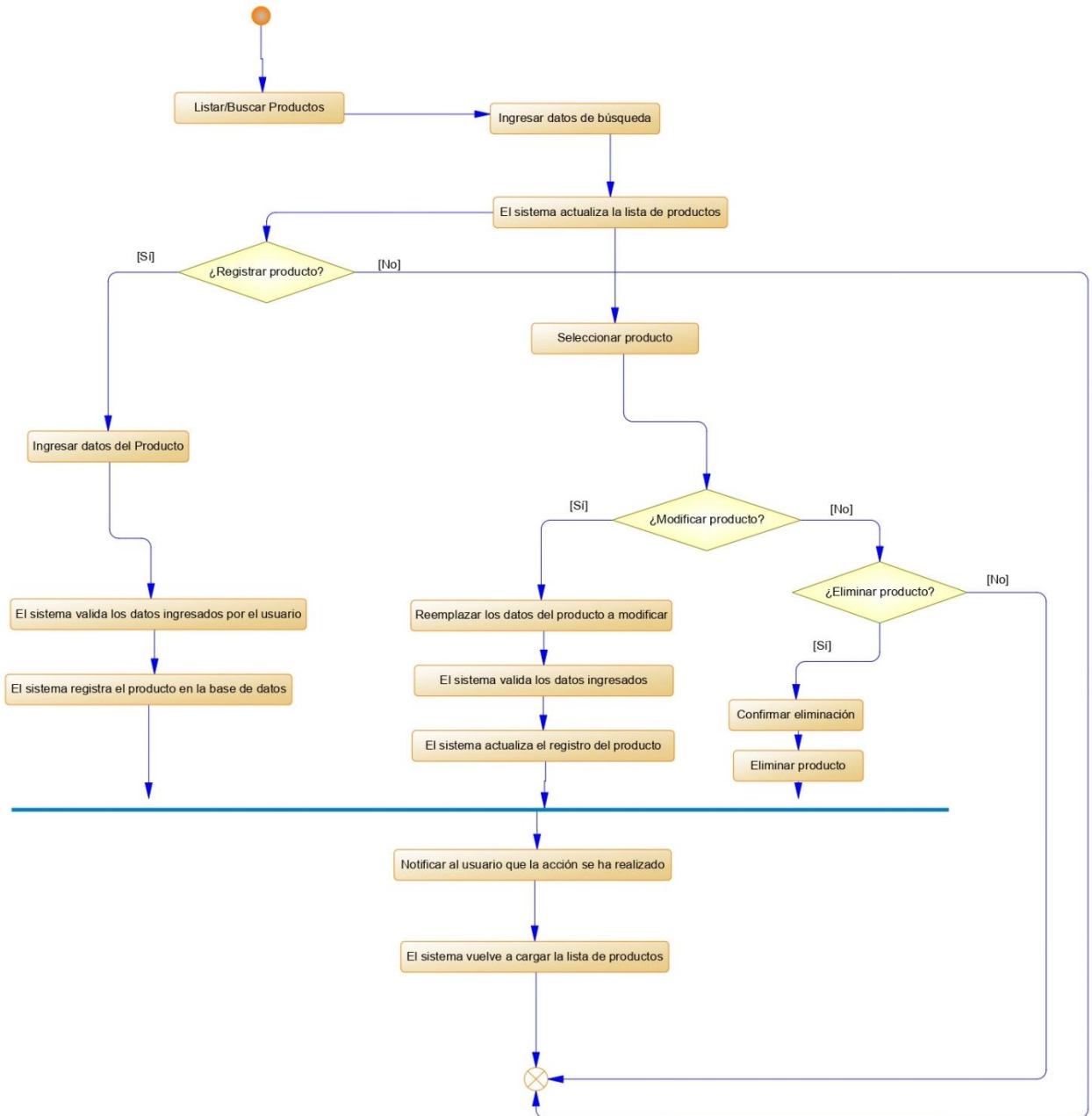


Fig. 9 Diagrama de actividad productos V1.0

1.7.3. Diagrama de actividad (Factura)

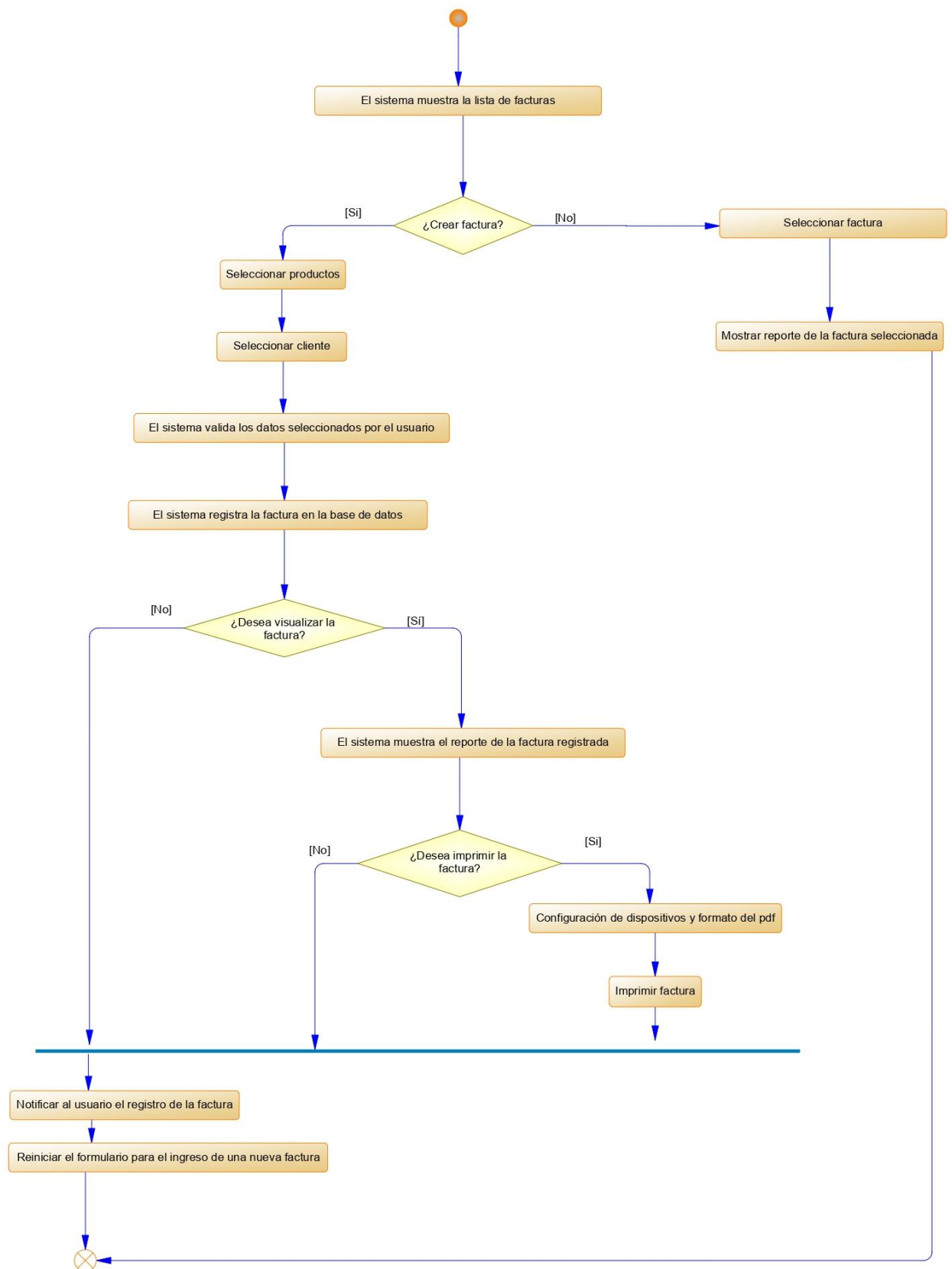


Fig. 10 Diagrama de actividad factura V1.0

1.8. Diagramas de bases de datos

En este artefacto se representan las tablas de la base de datos, sus columnas, tipos de datos y las relaciones entre las diferentes tablas, como las relaciones entre clientes, productos y facturas. Al proporcionar una representación visual detallada de cómo se estructuran y conectan los datos, el diagrama de base de datos facilita la creación de un esquema eficiente y consistente, asegura la integridad referencial y optimiza las consultas y operaciones de la base de datos.

1.8.1. Conceptual

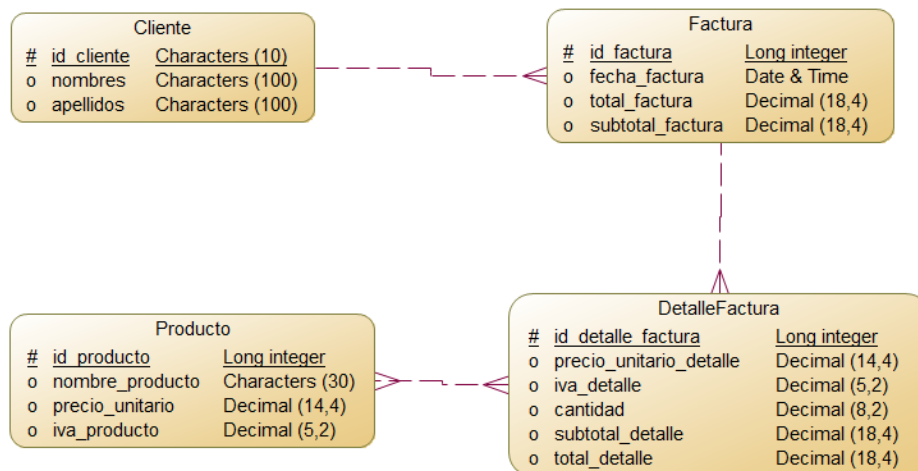


Fig. 11 Diagrama conceptual de base de dato V1.0

1.8.2. Logico

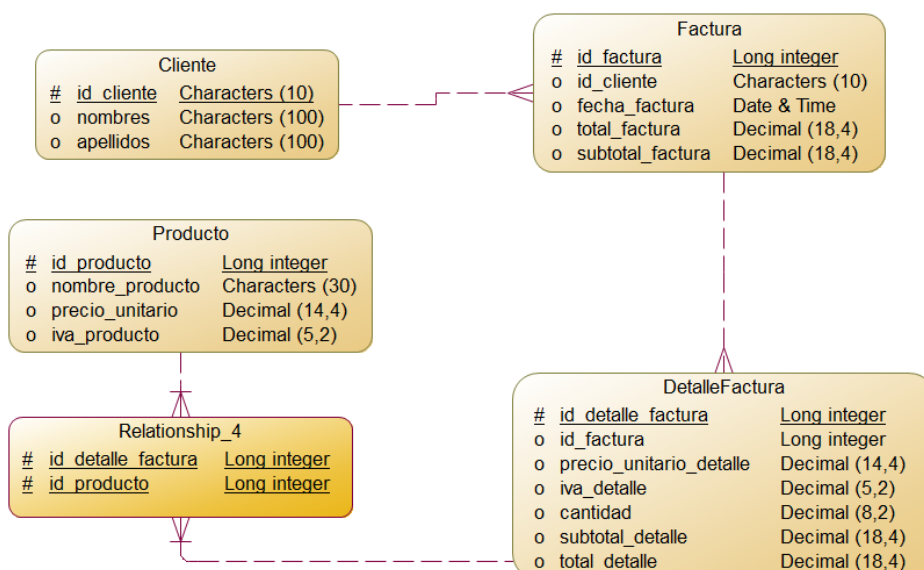


Fig. 12 Diagrama lógico de base de dato V1.0

1.8.3. Físico

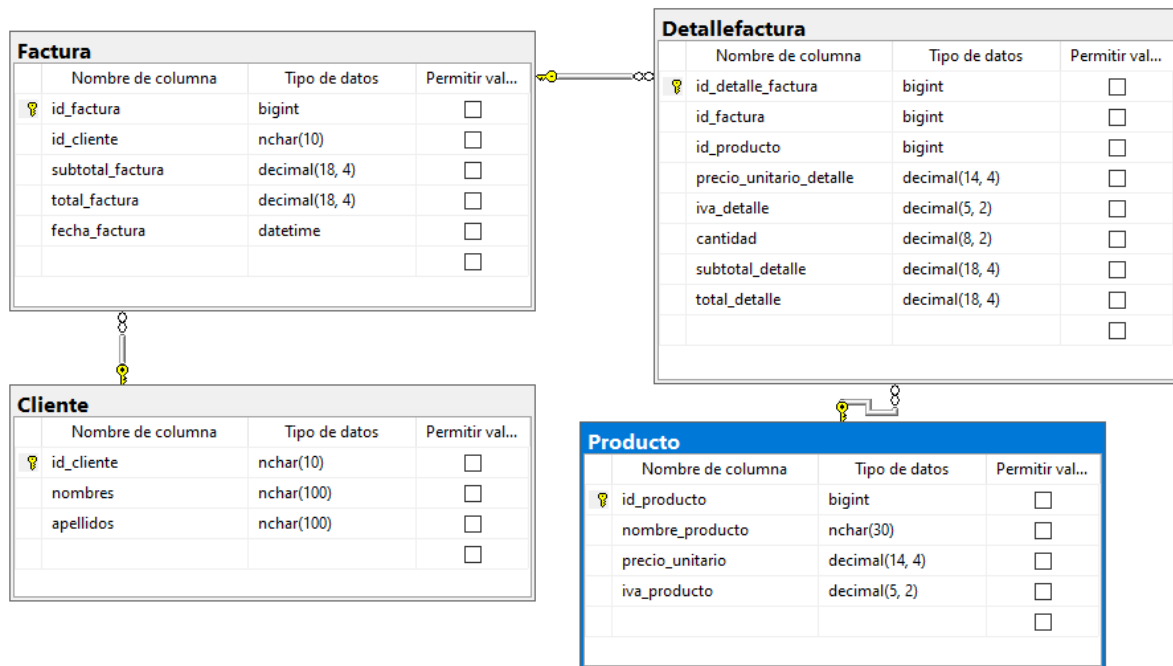


Fig. 8 Diagrama físico de base de datos SQL Server V1.0

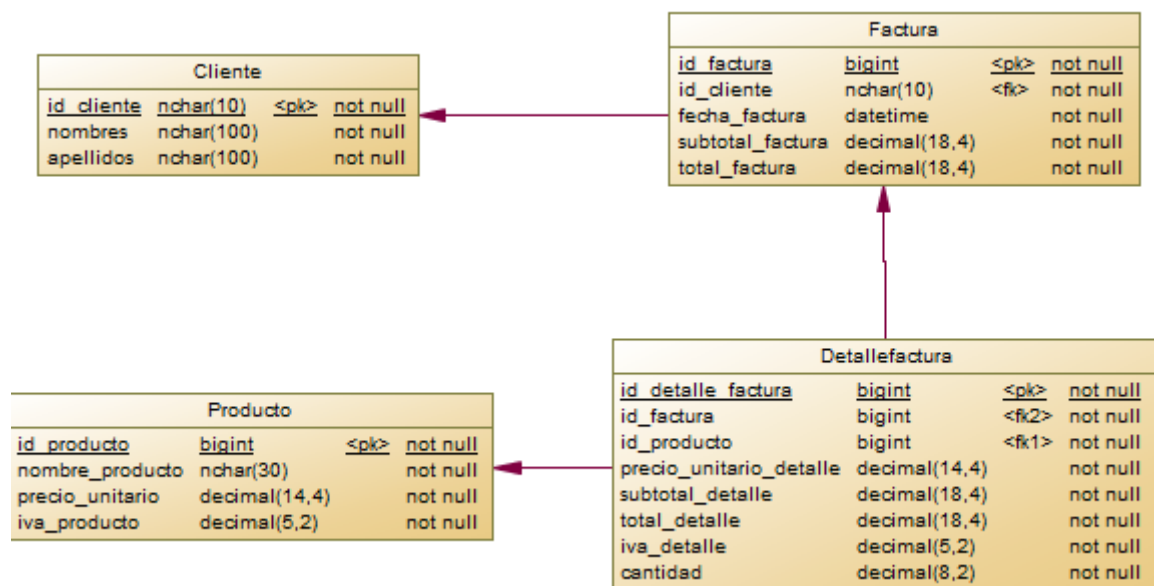


Fig. 9 Diagrama físico de base de datos PowerDesigner V1.0

1.9. Construcción y pruebas

En la fase de construcción, se implementaron las funcionalidades principales del sistema utilizando C# para la aplicación de escritorio y SQL Server como gestor de bases de datos. Se desarrollaron los módulos de gestión de clientes, productos y facturas, asegurando que cada entidad CRUD (Crear, Leer, Actualizar, Eliminar) estuviera correctamente implementada. Durante esta fase, se codificaron las interfaces de usuario, los modelos de datos, siguiendo el diseño detallado previamente elaborado.

1.9.1. Codificación

Se escribió el código para implementar todas las funcionalidades especificadas en los requisitos del sistema. Esto incluyó la creación de formularios para registrar clientes, productos y facturas, así como la lógica para calcular el total de la factura más el IVA. La codificación se realizó en un entorno de desarrollo integrado (IDE) compatible con C#, asegurando el uso de mejores prácticas y patrones de diseño.

1.9.2. Pruebas de integración

Se realizaron pruebas de integración para asegurar que los diferentes módulos del sistema funcionaran correctamente juntos. Se verificó la integración entre la interfaz de usuario, la lógica y la base de datos, asegurando que los datos se procesaran y almacenaran correctamente en SQL Server.

1.9.3. Pruebas de rendimiento

Se llevaron a cabo pruebas de rendimiento para evaluar la eficiencia y capacidad de respuesta del sistema. Se probaron diferentes escenarios de uso para asegurar que el sistema pudiera manejar un número razonable de transacciones sin degradación significativa en el rendimiento.

1.9.4. Creación del ejecutable

Se generó un ejecutable (.exe) de la aplicación de escritorio utilizando las herramientas de compilación de C#. Este ejecutable permite instalar y ejecutar el software en sistemas Windows.

1.10. Instalación

Durante la fase de instalación, se configuraron los computadores del laboratorio para su correspondiente presentación. Se instalaron tanto el ejecutable de la aplicación como SQL Server, y se realizaron las configuraciones necesarias para garantizar la correcta conexión entre la aplicación y la base de datos.

1.10.1. Configuración de computadores

Se configuraron las herramientas necesarias para asegurar que el software funcionara correctamente. Esto incluyó la instalación de dependencias necesarias y la configuración de permisos de usuario.

1.10.2. Despliegue del proyecto

El proyecto se desplegó, haciendo el software accesible para la presentación de este. Se explica su funcionamiento principal sobre el uso del sistema y se realizaron ajustes menores según el feedback inicial.

1.10.3. Dependencias y hardware utilizado

Fecha de uso de la configuración: 21/06/2024

Sistema operativo: Windows 10 Pro. Versión 22H2. Compilación del sistema operativo 19045.4529.

Hardware:

- Procesador: Intel(R) Core(TM) i3-2120 CPU @ 3.30GHz, 3.30 GHz
- RAM: 8.00 GB DDR3
- Núcleos: 2
- Procesadores lógicos: 4
- Disco duro mecánico (500GB) Modelo WDC WDS500G2B0A-00SM50

Plataforma base:

- Windows 10 Pro versión 22H2
- Power Designer v16.7 SP04
- Microsoft Visual Studio Community 2019
 - Versión: 16.11.12
- Microsoft .NET Framework
 - Versión: 4.7.2
- Gestor de paquetes NuGet V3
- Microsoft SQL Server 2019
- Microsoft SQL Server Management Studio 18
- Git bash
 - Versión 2.38.1.windows.1
- Lenguaje de programación C#

Subcomponentes:

- Microsoft.SqlServer.Types
 - Fuente (a): Gestor de paquetes NuGet.
 - Fuente (b): [NuGet Gallery | Microsoft.SqlServer.Types](#)
- Microsoft.ReportingServices.ReportViewerControl.Winforms
 - Fuente (a): Gestor de paquetes NuGet.
 - Fuente (b): [NuGet Gallery | Microsoft.ReportingServices.ReportViewerControl.Winforms](#)
- Diseñador de reportes RDLC: Microsoft.RdlcDesigner.vsix

- Fuente: [Microsoft RDLC Report Designer - Visual Studio Marketplace](#)

Observaciones importantes:

La instalación del IDE de Visual Studio Community debe ser configurada de tal forma que se instalen algunos componentes que se muestran a continuación.

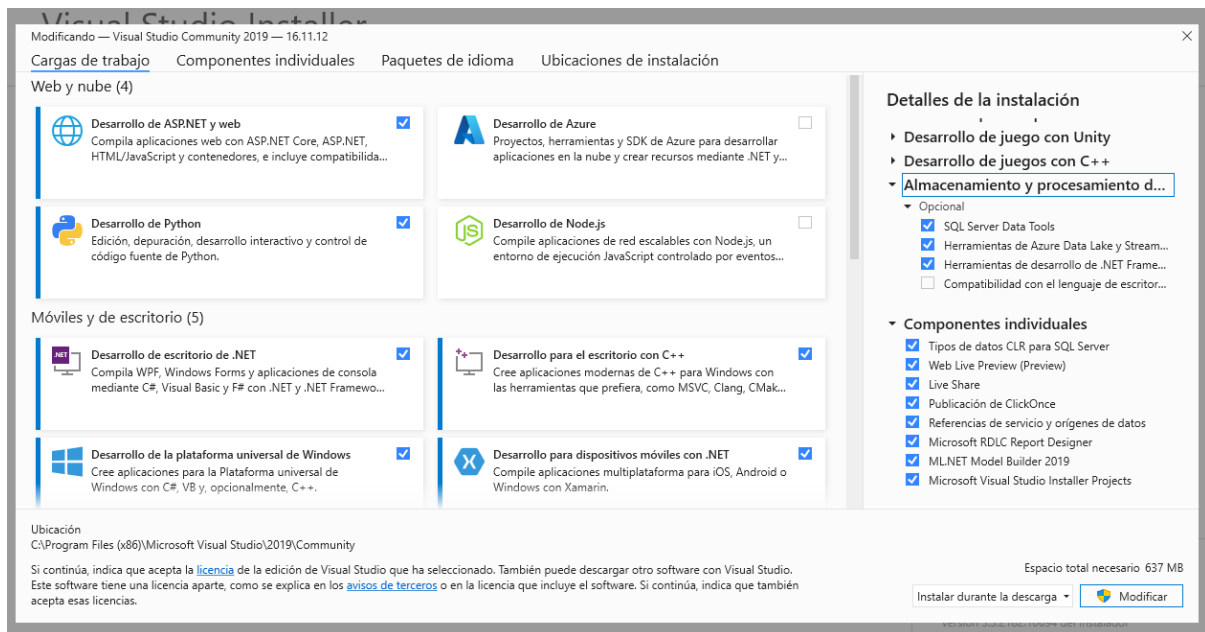


Fig. 10. Componentes requeridos durante la instalación de Visual Studio Community Pt. 1

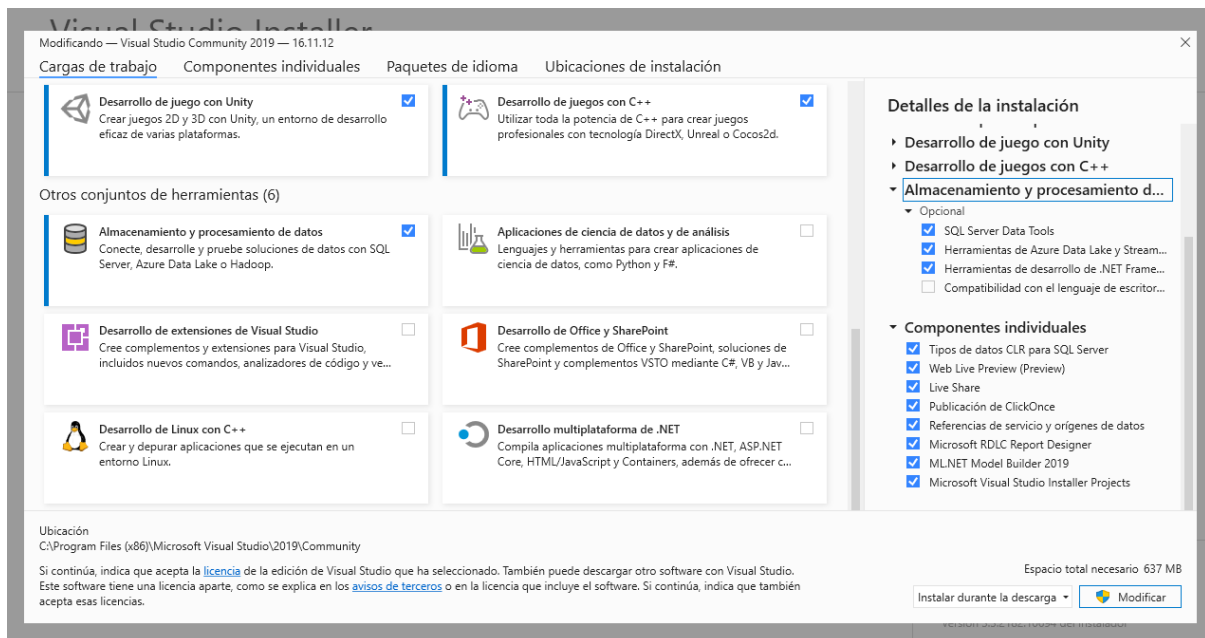


Fig. 11. Componentes requeridos durante la instalación de Visual Studio Community Pt. 2

Versiones de componentes - Gestor de paquetes (NuGet):

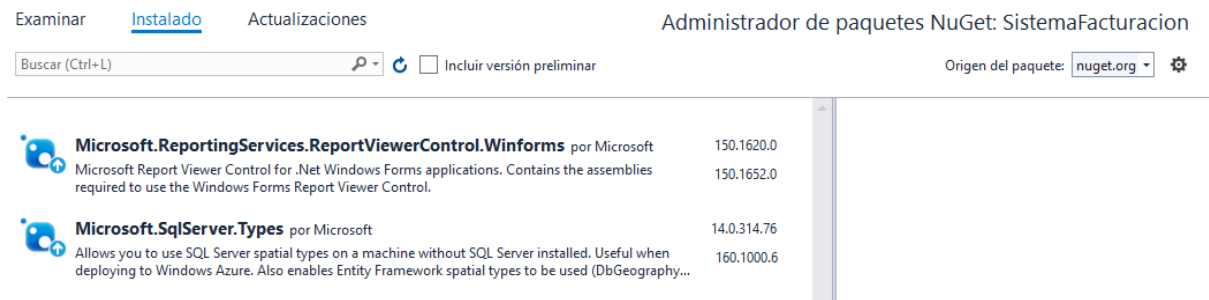


Fig. 12. Paquetes instalados mediante el administrador de paquetes NuGet V1.0

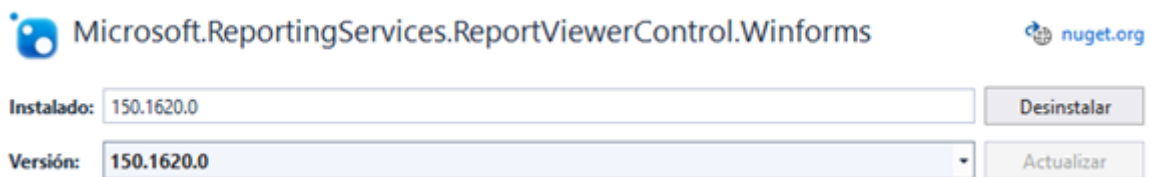


Fig. 13. Versión del paquete de ReportViwerControl instalado en el proyecto V1.0

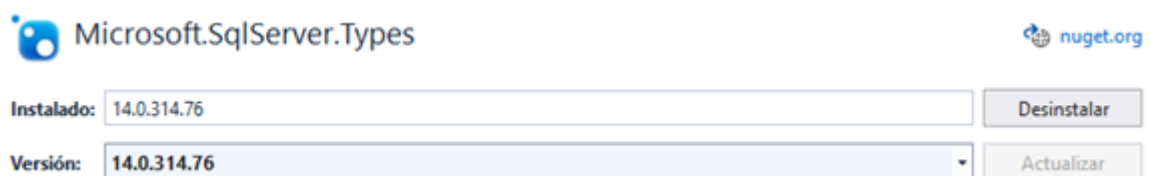


Fig. 14. Versión del paquete de SqlServer.Types instalado en el proyecto V1.0

Verificación

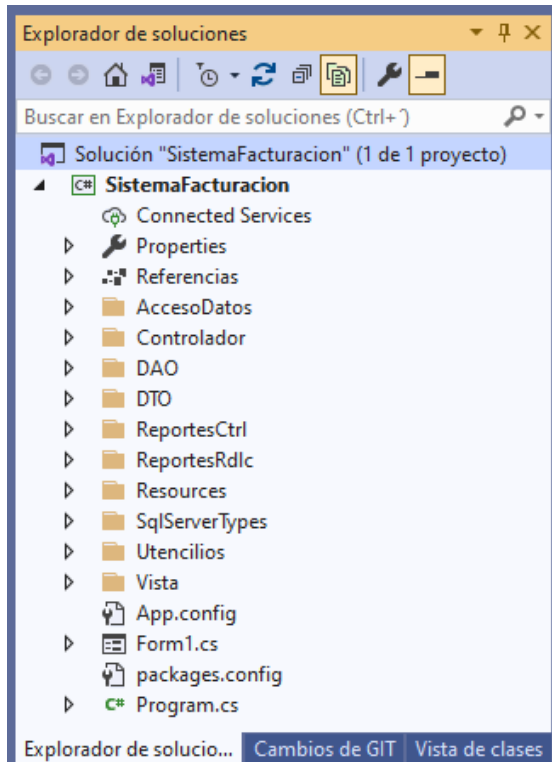


Fig. 16. Estructura del proyecto V1.0

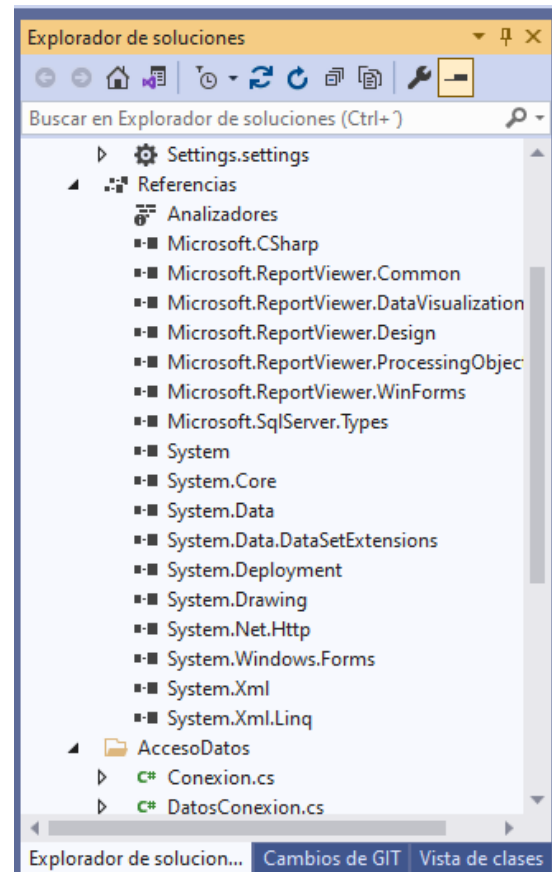


Fig. 15. Referencias instaladas debido a la instalación de los paquetes V1.0

Para verificar que todas las dependencias cargaron correctamente en las Referencias de la solución deberán aparecer aquellas que inician con Microsoft. Adicionalmente, se muestra la estructura de la solución en la imagen ubicada en la derecha.

1.11. Mantenimiento

Conforme pase el tiempo se realizarán las mejoras correspondientes además de incorporar nuevas funcionalidades según sea necesario, con la finalidad de obtener un software más avanzado, sin embargo, a continuación, se explica la proyección del proyecto en cuanto a sus respectivos mantenimientos y lo que queremos alcanzar en cada uno de ellos.

1.11.1. Preventivo

Planeamos realizar mantenimiento preventivo para evitar problemas futuros. Esto incluirá la optimización continua del rendimiento, la revisión periódica del código y la base de datos, y la actualización de componentes de software según sea necesario.

1.11.2. Correctivo

Abordaremos cualquier error o problema que se reporte en la versión actual del software. Esto implicará la corrección de errores y la implementación de soluciones para cualquier fallo que afecte la funcionalidad o la usabilidad del sistema.

1.11.3. Adaptativo

Proyectamos realizar mejoras y agregar nuevas funcionalidades basadas en las necesidades cambiantes según sea necesario. Esto incluirá la adaptación del software a nuevas versiones, la incorporación de nuevas características solicitadas y la mejora de las existentes.

1.12. GitHub Proyecto V1.0

<https://github.com/jhonleturme192005/GCSfacturacion.git>

The screenshot shows the GitHub interface for a repository named 'GCSfacturacion', which is marked as 'Private'. The repository has 5 branches and 0 tags. The commit history shows a recent commit by 'Hecmi' with the message 'Modificación de interfaces, automatización de reportes mediante contr...' at commit hash 'b9be824' 11 hours ago, with 22 total commits. The file list includes folders for '.vs/SistemaFacturacion/v16', 'AccesoDatos', 'Controlador', 'DAO', 'DTO', 'Properties', 'ReportesCtrl', 'ReportesRdlc', 'Resources', 'SqlServerTypes', and 'Utencilios'. Most files were modified 11 hours ago, while 'SqlServerTypes' was updated last week.

File/Folder	Commit Message	Time
.vs/SistemaFacturacion/v16	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
AccesoDatos	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
Controlador	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
DAO	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
DTO	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
Properties	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
ReportesCtrl	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
ReportesRdlc	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
Resources	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago
SqlServerTypes	Versión 2 del proyecto: Estructura, interfaces y codificación	last week
Utencilios	Modificación de interfaces, automatización de reportes mediante contr...	11 hours ago

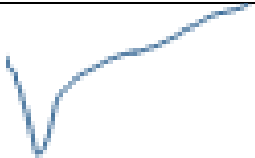
Fig. 15 Proyecto en Github V1.0

2. Proyecto de facturación (V1.1)

Para la versión de este proyecto primero se comenzó con la carta de solicitud del cliente en la cual mostraba la descripción de los cambios que se debían realizar además con la fecha límite para su presentación. También se mostraban las razones de porque se realizaban los cambios (Errores, Mejor manejo en la facturación etc.) También se presentaban el impacto que tendrían en el proyecto es decir en que partes afectarían a las funcionalidades ya implementadas, formularios clases etc. Finalmente mostrando las restricciones que se deberían seguir cuando se comiencen a realizar las modificaciones al software. A continuación, se presenta la orden de cambio de ingeniería formulada con los requisitos solicitados.

OCI – Software de Facturación de línea de referencia	
Identificación: OCI_001_2024	Referencia de solicitud de cambio: 001_2024
Fecha de emisión de la OCI: 08/07/2024	Fecha de la solicitud de cambio: 01/07/2024
Línea de referencia: Sistema de facturación UTEQ v1.0. Versión resultante: Sistema de facturación UTEQ v1.1.	Solicitando del cambio: Elías Portilla Olvera (rol de cliente)
Descripción de los cambios: • Agregar opciones para registrar los vendedores que realizan la facturación para conocer quién creo una determinada factura. Cada vendedor debe tener sus credenciales (usuario y contraseña) para acceder al sistema. • Para seleccionar un cliente en la facturación el sistema debe permitir hacerlo al dar doble clic. • En facturación, buscar el producto deseado por el texto de interés a buscar y coincida con el nombre inicial (de izquierda a derecha) de algún producto. • Las búsquedas deben realizarse según el campo que sea de interés para el usuario y solo debe aplicarse una vez presione “enter” en el textbox correspondiente a la búsqueda. • Permitir realizar una búsqueda de las Facturas por fechas. • No permitir al usuario marcar varias filas al intentar seleccionar un cliente en la facturación (no permitir subrayar más de un cliente). • Permitir crear empleados con sus credenciales y roles para el acceso al sistema y uso de funciones específicas del mismo. Se requiere que los cambios estén listo máximo el 15/Julio/2024	
Razón de cambio: El sistema de facturación no cumplen con algunos de los requerimientos definidos por el cliente; por esta razón, se adjuntan los siguientes errores y cambios requeridos: • Para evitar la carga del sistema durante las búsquedas, las mismas se realizarán cuando el usuario presione “enter” en el textbox de búsqueda y no al escribir una letra. • Al seleccionar un cliente para la factura solo debería seleccionar una única celda o fila, ya que seleccionar varias filas causa confusión sobre cuál es el que se ha seleccionado realmente. • No hay un control de usuarios que permita la creación de vendedores y administradores, que incluya sus datos y credenciales para acceder al sistema. • Los trabajadores tienen diferentes funciones que pueden realizar en el sistema, por eso se requiere que las acciones y permisos se den en base a roles de cada usuario. • Al crear una factura es necesario reconocer al vendedor que realizó determinada factura.	
Impacto en el proyecto: El impacto que tendrá estos cambios en el software son significantes ya que implican modificar y agregar nuevas funcionalidades. En la siguiente lista se encuentran los elementos que serán afectados o creados por los cambios: • Modificación de los diagramas desarrollados en PowerDesigner (diagrama de casos de uso, diagrama de clases y base de datos) debido al requisito de insertar usuarios y roles. • Modificación de los DataGridViews para realizar búsquedas según el campo que sea de interés por el usuario. • Formulario de visualización o lista de facturas para permitir al usuario realizar la búsqueda de facturas por fechas. • DataGridView de la lista de facturas para mostrar el vendedor que realizó las facturas. • Agregar un formulario para el registro de empleados con sus respectivos roles, datos y credenciales. • Modificación de la estructura de la base de datos y procedimientos almacenados para el cumplimiento de los cambios.	

• Modificación de clases existentes y creación de nuevas clases en el sistema. • CRUD en conjunto a su formulario para administrar los usuarios. • Formulario de inicio de sesión en el software.	
Restricciones: 1. En todos los formularios con DataGridView: a. Restricción: No cargar todos los datos por defecto en el DataGridView; cargar los datos solo cuando el usuario busque algún registro. 2. En los formularios que listan Clientes, Productos y Facturas: a. Restricción: Mejorar la barra de búsqueda permitiendo al usuario seleccionar el campo por el cual buscar. La búsqueda debe activarse mediante un botón y no en base al evento de cambio de texto. 3. En el proceso de selección de usuarios: a. Restricción: La selección de un usuario debe realizarse mediante el evento de doble clic, eliminando el botón para seleccionar. 4. En el formulario de impresión de factura: a. Restricción: Permitir la impresión de factura una vez guardada en la base de datos. Agregar el botón de impresión en el mismo formulario.	
Criterio para revisar y auditar: • Revisar la nueva versión del software con los cambios solicitados. • Revisar que los DataGridView no muestre información que no es requerida. • Evaluar la usabilidad del software, considerando la experiencia del usuario. • Revisar los modelos (casos de uso, clases y base de datos) del sistema anterior y actual. • Revisar el código fuente del sistema anterior y actual. • Inspeccionar la compatibilidad entre los modelos y el código fuente.	
Número de sprint que se tomaran: 2 sprint (cada sprint tiene una duración de una semana).	
Responsables de la implementación: • Casanova Morante Héctor Miguel. • Cedeño Salvatierra Orlando Jesús. • Jaime Pesantez Anderson David. • Leturne Pluas Jhon Byron. • Tenecela Intriago Angélica Michelle.	
Recursos necesarios: • SQL Server Management Studio 20. • SQL Server 2019 Express • IDE – Visual Studio Community 2019 (Versión: 16.11.12). • Microsoft .NET Framework (Versión: 4.7.2) • Power Designer (Versión 16.7 SP05). • Lenguaje de programación C#. • Git Bash (Versión 2.38.1. windows.1) • 5 ordenadores con los softwares previamente especificados instalados.	
Aprobado por: Casanova Morante Héctor Miguel (ACC)	
Fecha Inicio de la Implementación: 01/07/2024	Fecha a entregar los cambios: 15/07/2024
Estatus: Por implementar	Prioridad: Alta
Comentarios adicionales: ninguno.	

 Firma del solicitante (Elías Portilla Olvera)	 Firma del aprobador – ACC (Héctor Casanova)
--	---

2.1. Requisitos

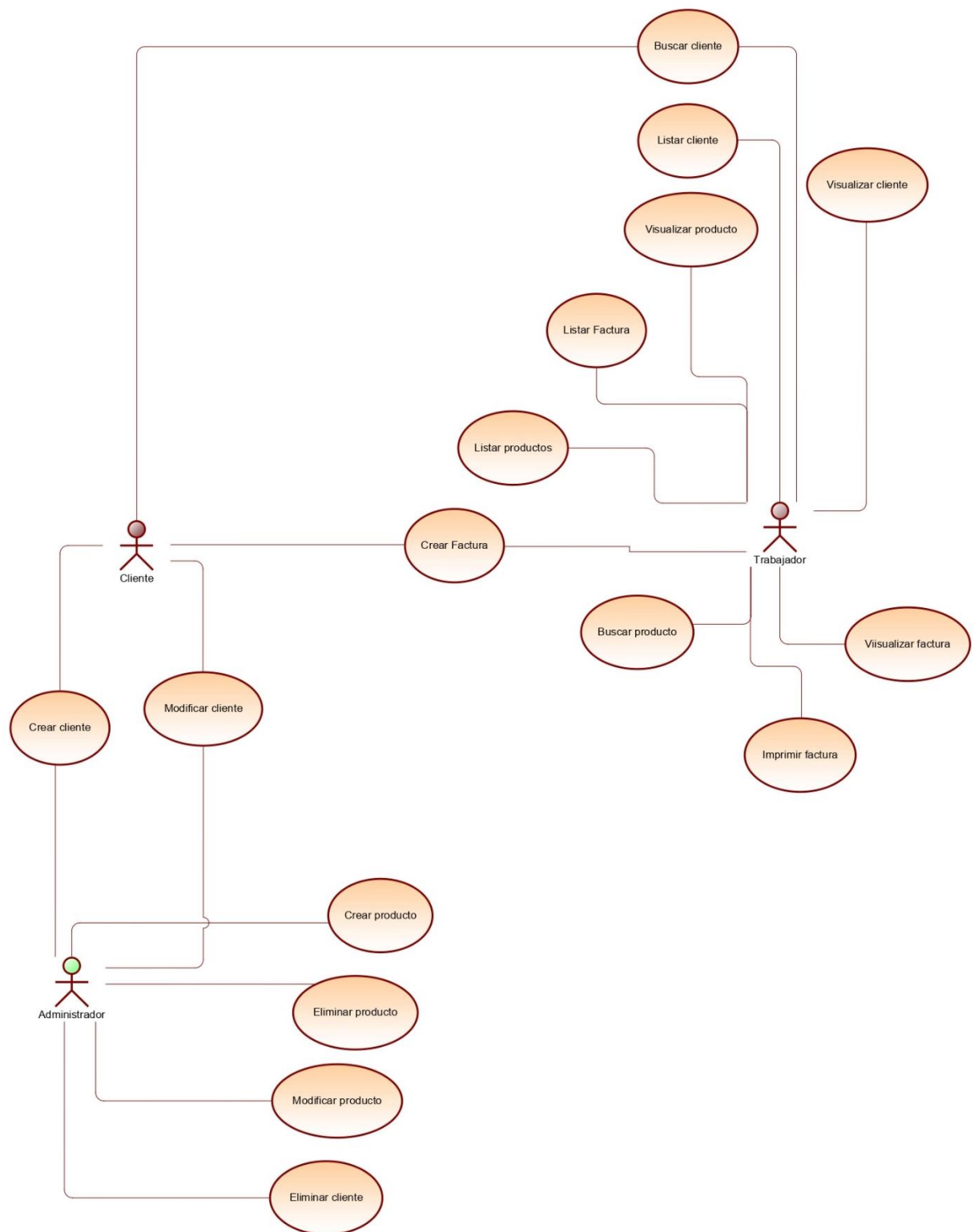
R1: Gestión de usuarios

- El sistema debe permitir crear nuevos usuarios asignando su rol (ADM (ADMINISTRADOR), VEN (VENDEDOR)).
- El sistema debe permitir guardar la factura con el trabajador que la está registrando.

Una vez aprobado la solicitud de cambio se procedió a hacer un análisis en los diagramas de la V1.0 para con ello implementar correctamente los nuevos cambios. Para ello las partes modificadas se marcó de color verde para saber que serían implementación o modificaciones nuevas.

2.2. Diagramas de casos de uso

2.2.1. Diagrama general



2.2.2. Diagrama de CU (Cliente)

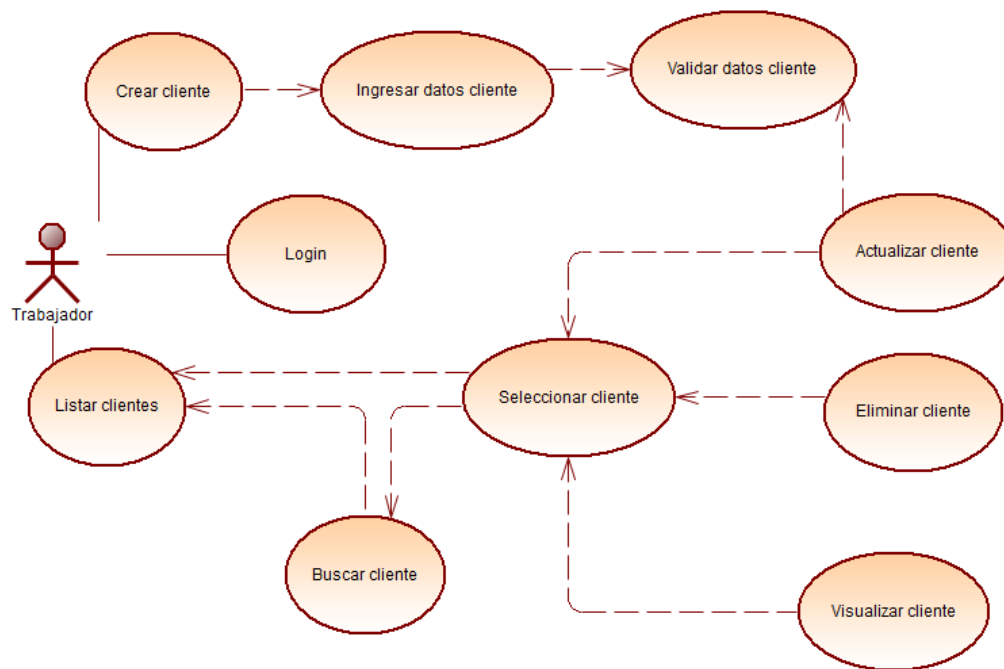


Fig. 17. Diagrama de casos de uso de cliente V1.1

2.2.3. Diagrama de CU (Empleado)

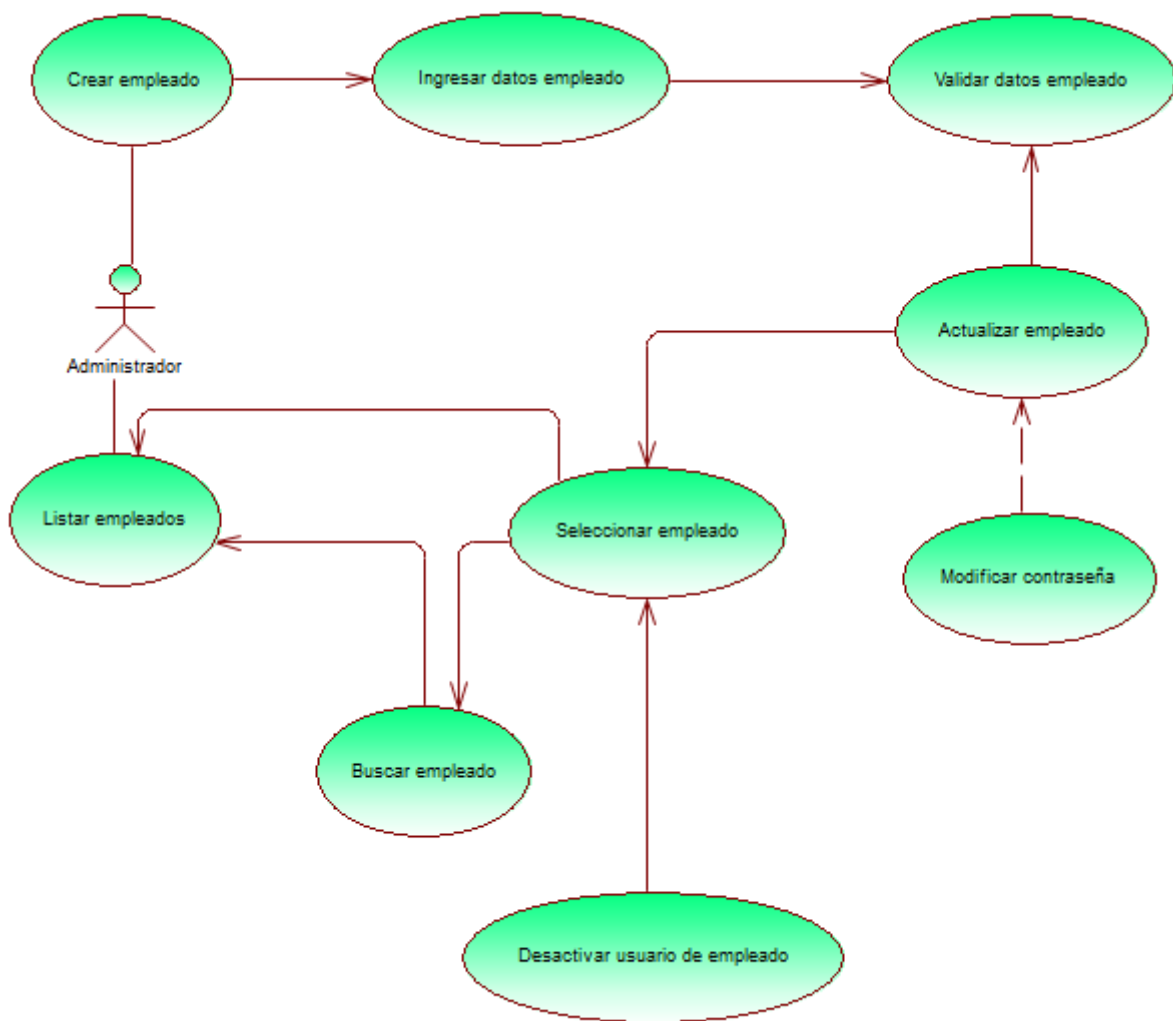


Fig. 18. Diagrama de casos de uso de empleados V1.1

2.2.4. Diagrama de CU (Producto)

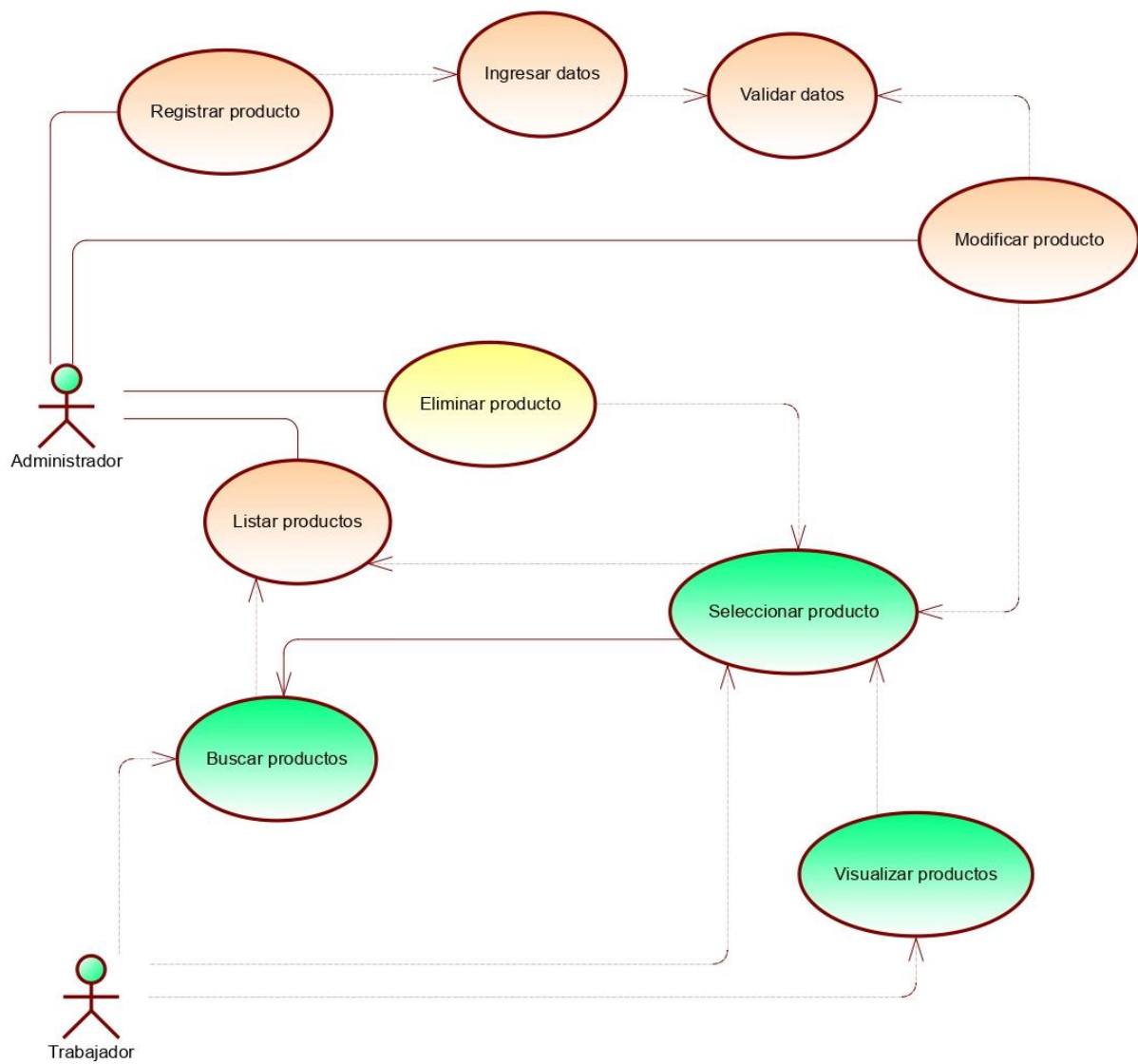


Fig. 19. Diagrama de casos de uso de productos V1.1

2.2.5. Diagrama de CU (Factura)

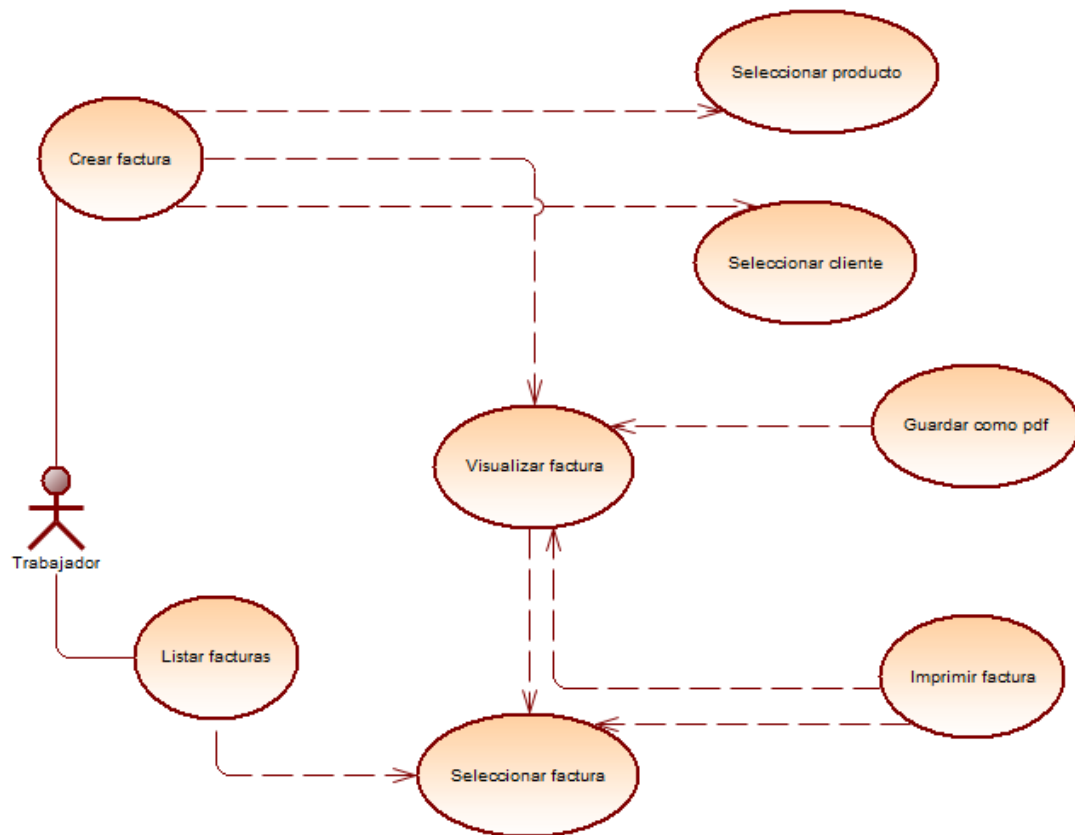


Fig. 20. Diagrama de casos de uso de factura V1.1

2.3. Casos de uso

2.3.1. Caso de uso Registrar Factura (Modificación por nueva versión)

Caso de uso	Registrar factura	Excepciones	
Iteración	2, última modificación: 14/07//2024	6.1. Posibles cálculos incorrectos si los precios de los productos no son coherentes. 7.1. No existen productos seleccionados para la factura. 12.1. El reporte no funciona correctamente debido a una mal configuración.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Registrar la factura de los productos que fueron seleccionados por un cliente específico.	Frecuencia de uso	Alta
Precondiciones	• El sistema debe estar funcionando correctamente. • El cliente al que va dirigida la factura debe estar registrado en el sistema. • El trabajador debe haber iniciado sesión en el sistema. • Deben existir productos registrados en el sistema.	Canal al actor	A través de un computador con conexión a internet.

Disparador	Un cliente desea realizar la compra de una serie de productos.	Actores secundarios	Cliente
Escenario		Canales a los actores secundarios	
Actores	Sistema	Cliente: Interacción directa con el trabajador.	
1. El cliente debe mencionar al trabajador los productos que desea.			
2. El trabajador accede a la sección de facturas presionando el botón con el mismo nombre.			
3. El trabajador presiona el botón “Seleccionar cliente” (ver el caso de uso Listar clientes) y selecciona al cliente correspondiente.			
4. El trabajador presiona el botón “Seleccionar productos” (ver el caso de uso Listar productos)			
5. El trabajador busca el/los producto(s) que el cliente desea.		Asuntos pendientes	
6. El trabajador agrega el producto a la lista de productos en factura.			

	7. El sistema calcula el IVA, subtotal y el total a pagar.	
8. El trabajador presiona el botón “Facturar”		
	9. El sistema envía los productos de la factura a la base de datos a la base de datos y calcula tanto el IVA, total entre otros valores.	
	10. El sistema guarda el encabezado de la factura (datos del cliente y del empleado con la sesión iniciada), el detalle (productos) y los totales.	
11. El trabajador decide si desea imprimir la factura generada por el sistema.		
	12. Sí el vendedor desea ver la factura se muestra en forma de reporte; caso contrario, no lo hace.	
13. El sistema actualiza el formulario de registro de factura, limpiándolo.		

2.3.2. Caso de uso Registrar empleado

Caso de uso	Registrar empleado	Excepciones	
Iteración	1, última modificación: 14/07//2024	3.1. Al calcular la edad basado en la fecha de nacimiento resulta en que el empleado no es mayor de edad. 3.2. La contraseña no cumple con las restricciones o mecanismos de seguridad para garantizar una clave válida. 4.1. El sistema al procesar los datos detecta que el usuario ya se ha registrado anteriormente.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Registrar un usuario en el sistema para que cada vendedor pueda tener su cuenta dentro del software.	Frecuencia de uso	Alta
Precondiciones	- El sistema debe estar funcionando correctamente. - La sesión actual debe ser de un usuario de tipo administrador. - El usuario no debe estar registrado en el sistema	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un administrador desea realizar el registro de un nuevo trabajador.	Actores secundarios	Trabajador
Escenario		Canales a los actores secundarios	

Actores	Sistema	Administrador: Interacción directa con el trabajador.
1. El trabajador proporciona los datos al administrador del sistema.		
2. El administrador accede al menú de “Empleados” y presiona el botón de “Agregar” y envía los datos al sistema.		
	3. El sistema valida que la fecha de nacimiento de como resultado que el empleado sea mayor de edad, valida que el usuario sea único y la contraseña sea válida con respecto a mecanismos de seguridad.	
	4. El sistema procesa los datos del trabajador encriptando la contraseña.	
	5. El sistema guarda los datos procesados en la base de datos.	Asuntos pendientes
6. El administrador avisa al trabajador que su usuario se ha creado correctamente.		

2.3.3. Caso de uso: Actualizar empleado

Caso de uso	Actualizar empleado	Excepciones	
Interacción	1, última modificación: 14/07//2024	6.1. Los datos definidos por el empleado son incorrectos.	
		7.1. La contraseña ingresada es incorrecta, por lo que, no se realizan los cambios y se notifica al trabajador.	
Actor principal	Trabajador	Prioridad	Alta
Objetivo en contexto	Permitir al trabajador modificar sus datos según lo requiera.	Cuando estará disponible	Al 1er incremento
Precondiciones	- El sistema debe estar funcionando correctamente. - El usuario debe estar registrado en el sistema. - El usuario debe haber iniciado sesión.	Frecuencia de uso	Alta
Canal al actor		Actores secundarios	A través de un computador con conexión a internet.
Disparador	Un trabajador desea cambiar sus datos personales o de acceso.		Trabajador
Escenario		Canales a los actores secundarios	
Actores	Sistema	Trabajador: Interacción directa con el sistema.	
1. El trabajador inicia sesión en el sistema (ver el caso de uso “LogIn”).			

2. El trabajador presiona el botón con el ícono representativo a “usuario” para modificar sus datos.		
	3. El sistema obtiene los datos correspondientes al empleado y los muestra en la interfaz.	
4. El trabajador ingresa los datos que desea modificar.		
5. El trabajador ingresa su contraseña para confirmar que los cambios sean aplicados y presiona el botón de “Editar”.		
	6. El sistema verifica que la información proporcionada es correcta.	Asuntos pendientes
	7. El sistema verifica que la contraseña sea correcta para proceder con la actualización de los datos del usuario.	
	8. El sistema indica al trabajador que sus datos han sido actualizados correctamente.	

2.3.4. Caso de uso: Log In

Caso de uso	Log In	Excepciones
Interacción	1, última modificación: 14/07//2024	3.1. Existen datos faltantes, por lo tanto, el sistema notifica al trabajador.

		4.1. Los datos proporcionados son incorrectos por lo tanto no inicia sesión y notifica al usuario de las credenciales erróneas.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir a los trabajadores iniciar sesión en el sistema.	Frecuencia de uso	Alta
Precondiciones	• El sistema debe estar funcionando correctamente. • El usuario debe estar registrado en el sistema. • No debe haber ningún usuario con la sesión iniciada.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un trabajador desea iniciar sesión en el sistema.	Actores secundarios	Trabajador
Escenario		Canales a los actores secundarios	
Actores	Sistema	Trabajador: Interacción directa con el sistema.	
1. El trabajador ejecuta la aplicación e ingresa sus credenciales (usuario y contraseña)			
2. El trabajador presiona el botón con el texto “Iniciar sesión”.			
	3. El sistema valida que los campos no estén vacíos.		

	4. El sistema verifica que el usuario que ingreso existe en los registros del sistema.	
	5. El sistema inicia sesión con el usuario proporcionado.	Asuntos pendientes
	6. El sistema retorna al usuario a la pantalla principal y muestra sus datos de acceso en la interfaz (nombre de usuario y rol).	

2.3.5. Caso de uso: Cambiar contraseña

Caso de uso	Cambiar contraseña	Excepciones	
Interacción	1, última modificación: 14/07//2024	6.1. Si el empleado tiene rol de vendedor y no ingresó correctamente su contraseña notificará al usuario que la contraseña actual no coincide con la registrada. 6.2. La nueva contraseña no cumple con las restricciones de seguridad de contraseña; es decir, combinación de caracteres y longitud de contraseña. 7.1. Ocurre un error durante la modificación, por lo que se notifica al usuario.	
		Prioridad	Alta

Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir a los trabajadores modificar su contraseña.	Frecuencia de uso	Alta
Precondiciones	- El sistema debe estar funcionando correctamente. - El usuario debe estar registrado en el sistema.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un trabajador desea modificar su contraseña.	Actores secundarios	Trabajador
Escenario		Canales a los actores secundarios	
Actores	Sistema	Trabajador: Interacción directa con el sistema.	
1. El trabajador inicia sesión (ver caso de uso “ Log In ”)			
2. El trabajador accede al apartado de modificación de empleado (ver caso de uso “ Modificar empleado ”)			
3. El empleado presiona el botón con el texto “Modificar contraseña”			
	4. El sistema muestra el nombre de usuario del empleado.	Asuntos pendientes	
5. El empleado con rol de vendedor ingresa su contraseña actual y la nueva contraseña. Sí			

tiene rol de administrador solo especifica la nueva contraseña-		
	6. El sistema valida que la contraseña actual sea igual a la cifrada en la base de datos. Sí tiene rol de administrador no se realiza la verificación de contraseña.	
	7. El sistema muestra un mensaje notificando al usuario que la contraseña fue modificada correctamente.	

2.3.6. Listar empleados

Caso de uso	Listar empleados	Excepciones	
Interacción	1, última modificación: 26/05//2024	2.1. No existen facturas.	
		Prioridad	Alta
Actor principal	Administrador	Cuando estará disponible	Al 4to incremento
Objetivo en contexto	Listar los empleados o trabajadores registrados en el sistema.	Frecuencia de uso	Media
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.

	La sesión iniciada debe ser de un administrador.		
Disparador	El administrador desea visualizar los empleados que han sido registrados en el sistema.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El administrador ingresa al apartado de “Empleados”.			
	2. El sistema lista todos los empleados registrados en el sistema.		
3. El usuario ingresa un texto a buscar y selecciona un campo de búsqueda.		Asuntos pendientes	
	4. El sistema muestra los empleados que coincidan con los criterios de búsqueda.		

2.3.7. Caso de uso: Visualizar empleado

Caso de uso	Visualizar factura	Excepciones
Iteración	1, última modificación: 26/05/2024	1.1. No existen facturas registradas en el sistema. Por lo tanto, no es posible realizar una visualización.

		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Visualizar los datos de un empleado en específico.	Frecuencia de uso	Alta
Precondiciones	- El sistema debe estar funcionando correctamente. - La sesión iniciada debe ser de un administrador.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El administrador accede a la lista de empleados.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador accede a la lista de Empleados (ver caso de uso Listar empleados) y presione el botón con el ícono representativo a “visualizar”			
	2. El sistema muestra los datos del empleado seleccionado		
		Asuntos pendientes	
		N/A	

2.3.8. Caso de uso: Registrar producto (Modificación)

Caso de uso	Registrar producto	Excepciones	
Interacción	2, última modificación: 14/07//2024	4.1. Los datos ingresados no son válidos. Por lo tanto, se retroalimenta al usuario de los errores.	
		5.1. Al registrar los datos en la base de datos existe algún error que evita la inserción.	
Actor principal	Administrador	Prioridad	Alta
Actor principal	Administrador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al administrador registrar productos al sistema.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente	Canal al actor	A través de un computador con conexión a internet.
Disparador	El Administrador desea registrar un producto en el sistema.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
7. El administrador accede a la opción “Registrar producto” en la interfaz.			
	8. El sistema muestra un formulario para el ingreso de los datos del producto.		

9. El administrador ingresa los datos solicitados en el formulario.		
	10. El sistema valida los datos ingresados (longitud y tipos).	Asuntos pendientes 2. ¿Debería permitirse utilizar diferentes tipos de moneda?
	11. El sistema registra el producto en la base de datos.	
	12. El sistema notifica al trabajador que el producto ha sido registrado correctamente.	

2.3.9. Caso de uso: Modificar producto (Modificación)

Caso de uso	Modificar producto	Excepciones	
Interacción	2, última modificación: 14/07//2024	2.1. La lista de productos no ha sido actualizada y el producto seleccionado ha sido eliminado. Por lo tanto, al editarlo se produce un error. 5.1. Los datos ingresados no son válidos. Por lo tanto, se retroalimenta al usuario de los errores. 6.1. Al registrar los datos en la base de datos existe algún error que evita la modificación del producto.	
		Prioridad	Alta

Actor principal	Administrador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al administrador modificar los datos de un producto específico.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El administrador desea modificar los datos de un producto.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El administrador accede a la opción “Modificar” en la lista de productos desplegada (ver el caso de uso Listar productos).			
	2. El sistema realiza una consulta a la base de datos para obtener el producto seleccionado.		
	3. El sistema muestra los datos detallados del producto seleccionado por el trabajador.	Asuntos pendientes	
		2. ¿Qué medidas deberían considerarse para evitar errores	

4. El administrador modifica los datos de interés permitidos por el sistema.		en el caso especificado en la excepción 2.1?
	5. El sistema valida los datos ingresados (longitud y tipos).	
	6. El sistema modifica los datos del producto seleccionado en la base de datos.	
	7. El sistema notifica al trabajador que el producto ha sido modificado correctamente.	

2.3.10. Caso de uso: Eliminar producto (Modificación)

Caso de uso	Eliminar producto	Excepciones	
Interacción	2, última modificación: 15/07//2024	2.1. El producto seleccionado no puede ser eliminado por que está asociado con los registros de otra tabla.	
		Prioridad	Alta
Actor principal	Administrador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Permitir al administrador eliminar un producto específico.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un

			computador con conexión a internet.
Disparador	El administrador desea eliminar un producto.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
5. El administrador accede a la opción “Eliminar” en la lista de productos desplegada (ver el caso de uso Listar productos).			
	6. El sistema solicita confirmación para la eliminar el producto seleccionado.		
	7. El sistema elimina el registro del producto de la base de datos.		
	8. El sistema actualiza la lista de productos.		
		Asuntos pendientes	
		1. ¿Qué mecanismos podrían aplicarse para que todos los usuarios mantengan la lista de productos actualizada?	

2.4. Diagramas de actividades

2.4.1. Diagrama de actividad (Empleados)

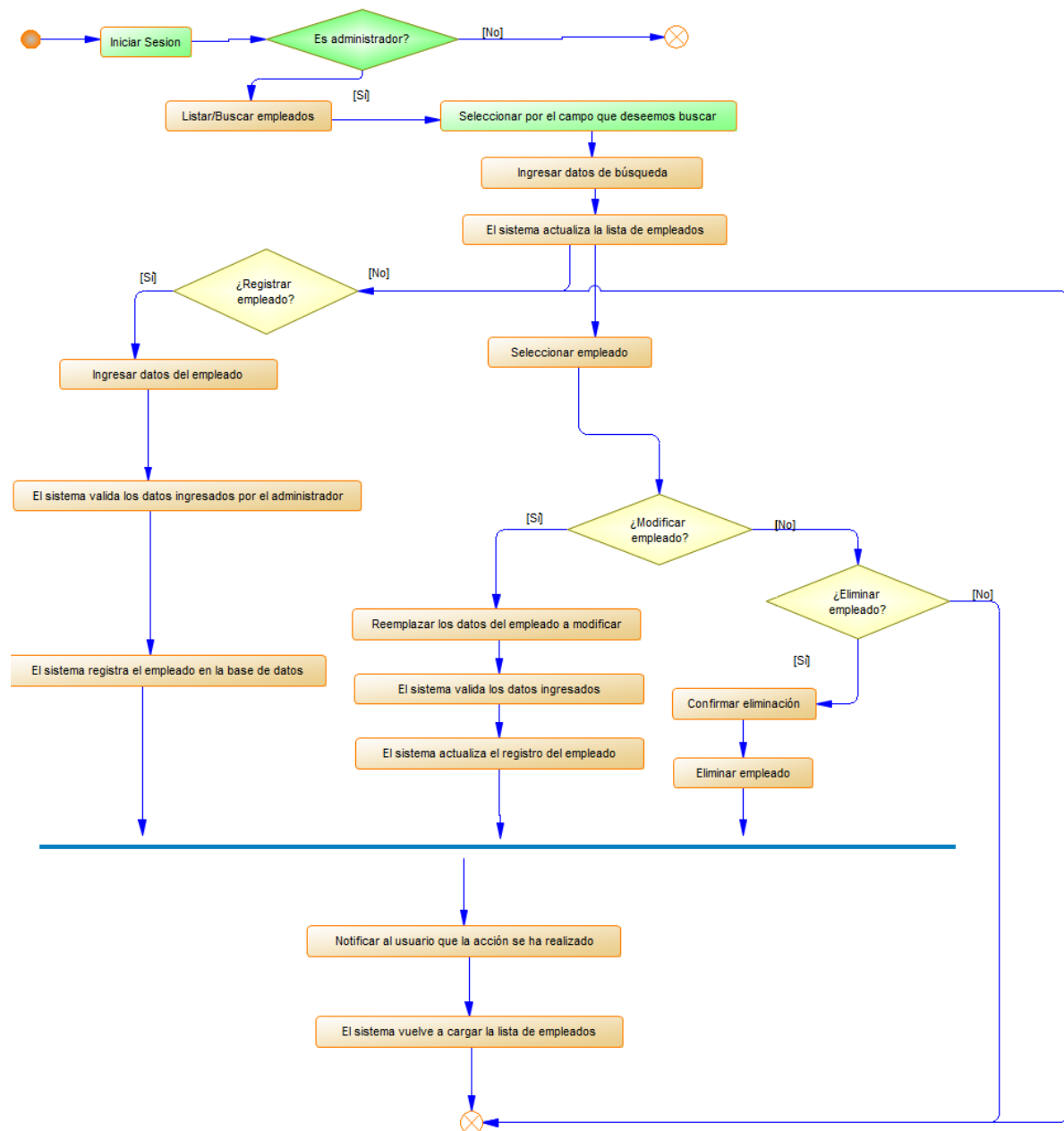


Fig. 21. Diagrama de actividades de empleados V1.1

2.4.2. Diagrama de actividad (Cliente)

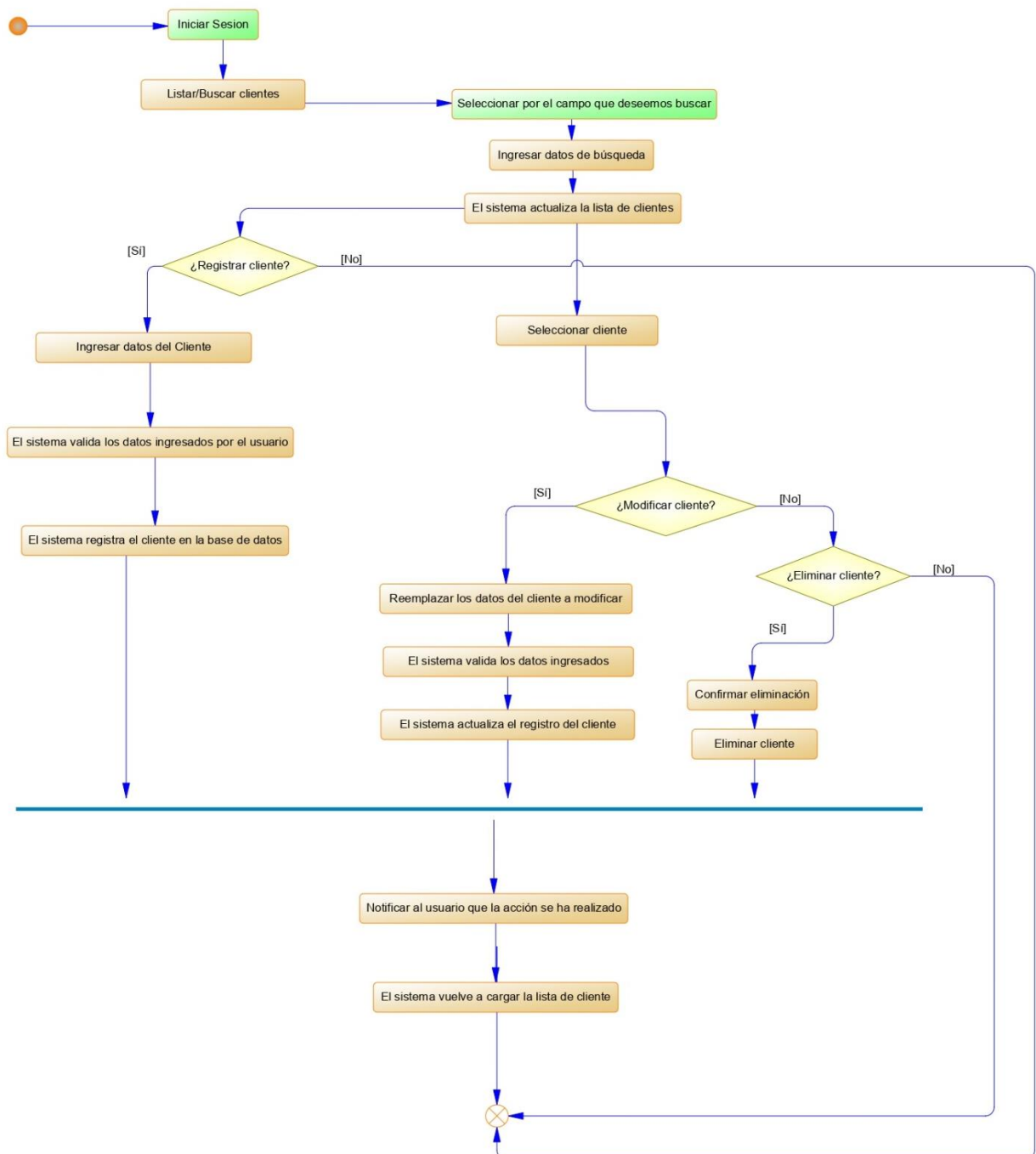


Fig. 22. Diagrama de actividades de clientes V1.1

2.4.3. Diagrama de actividad (Productos)

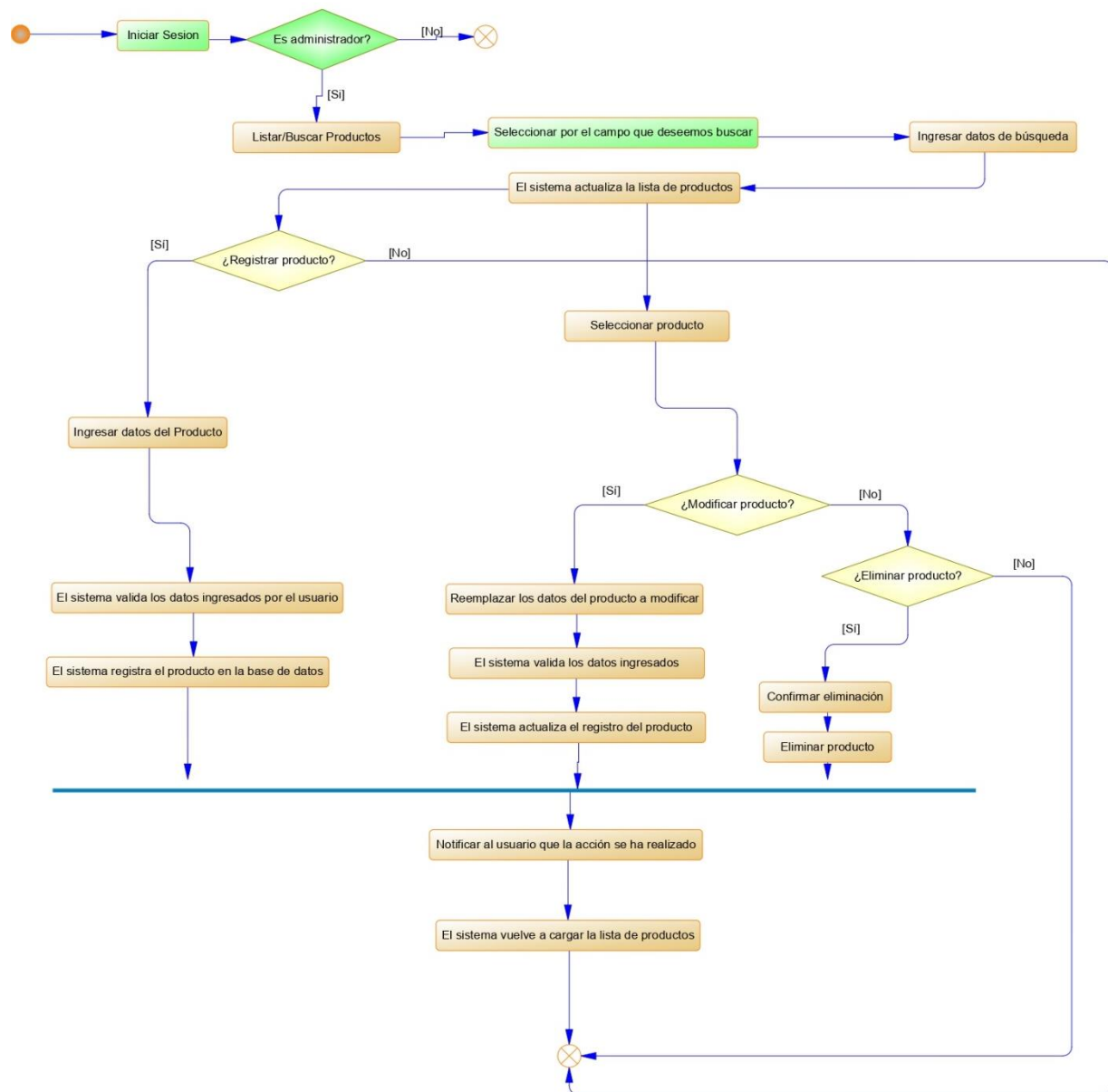


Fig. 23. Diagrama de actividades de productos V1.1

2.4.4. Diagrama de actividad (Factura)

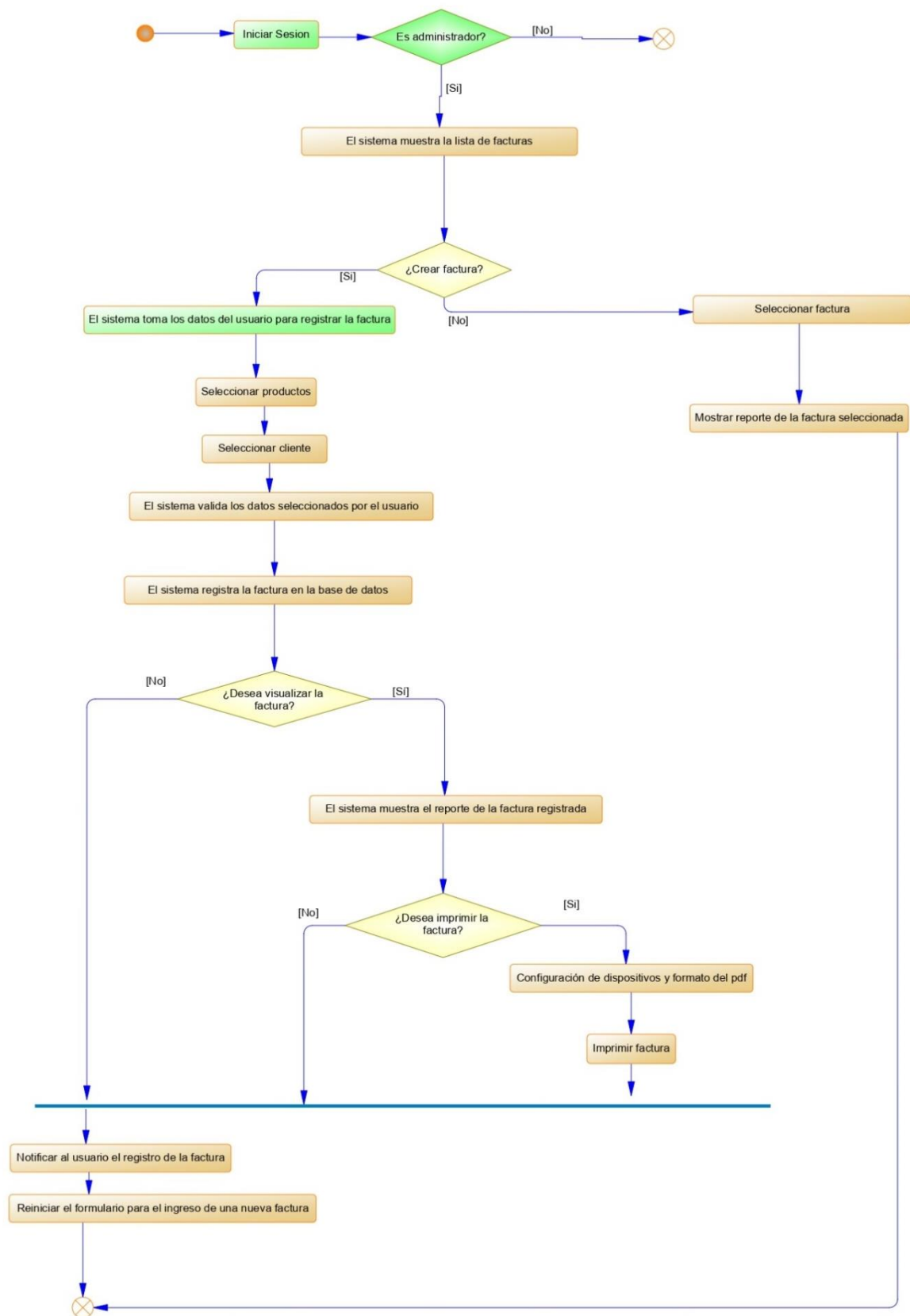


Fig. 24. Diagrama de actividades de factura V1.1

2.5. Diagrama de flujo de datos

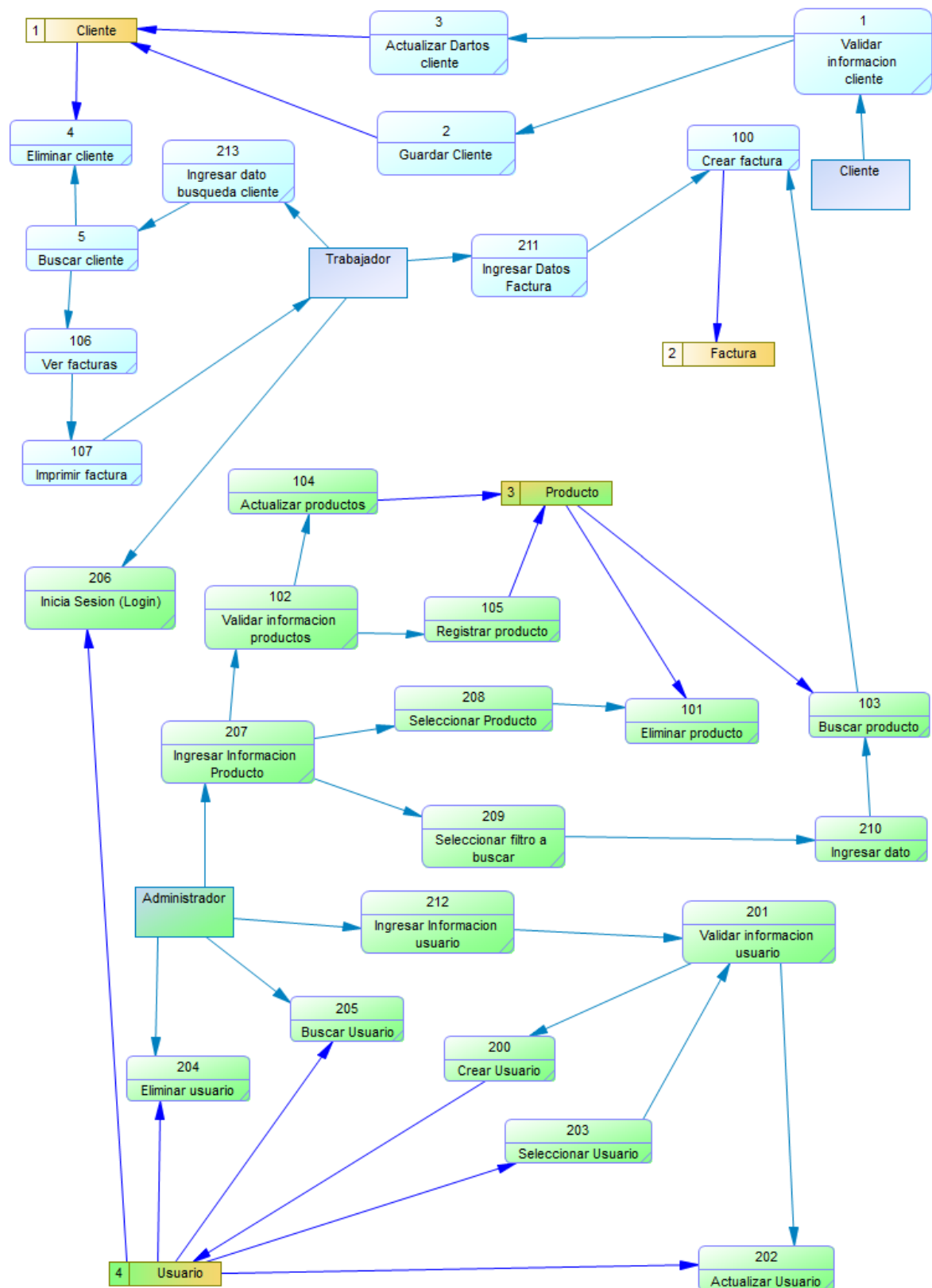


Fig. 25. Diagrama de flujo de datos V1.1

2.6. Diagrama de clases

2.6.1. Paquete DTO (Objeto de transferencia de datos)

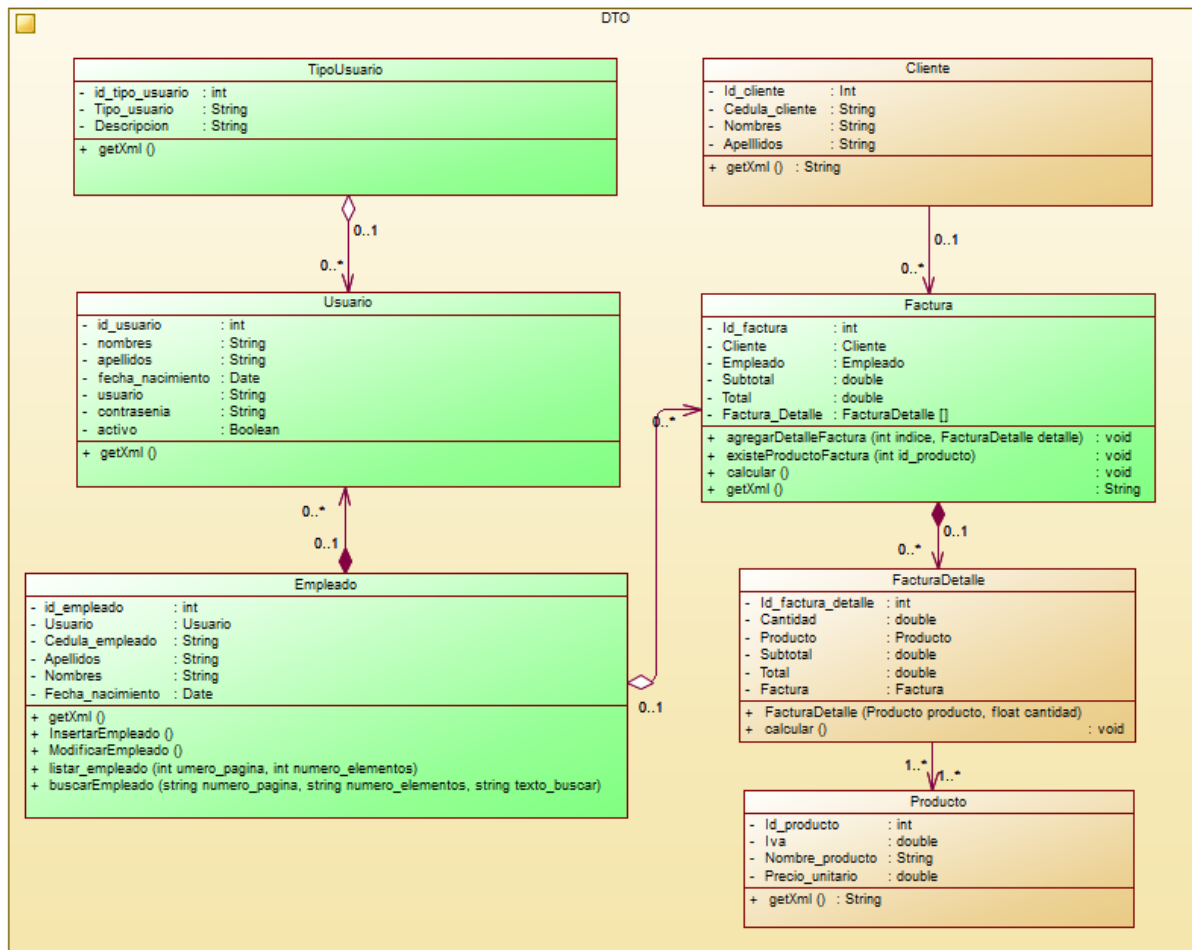


Fig. 26. Diagrama de clases – Paquete Dto V1.1

2.6.2. Paquete de Conexión con la base de datos

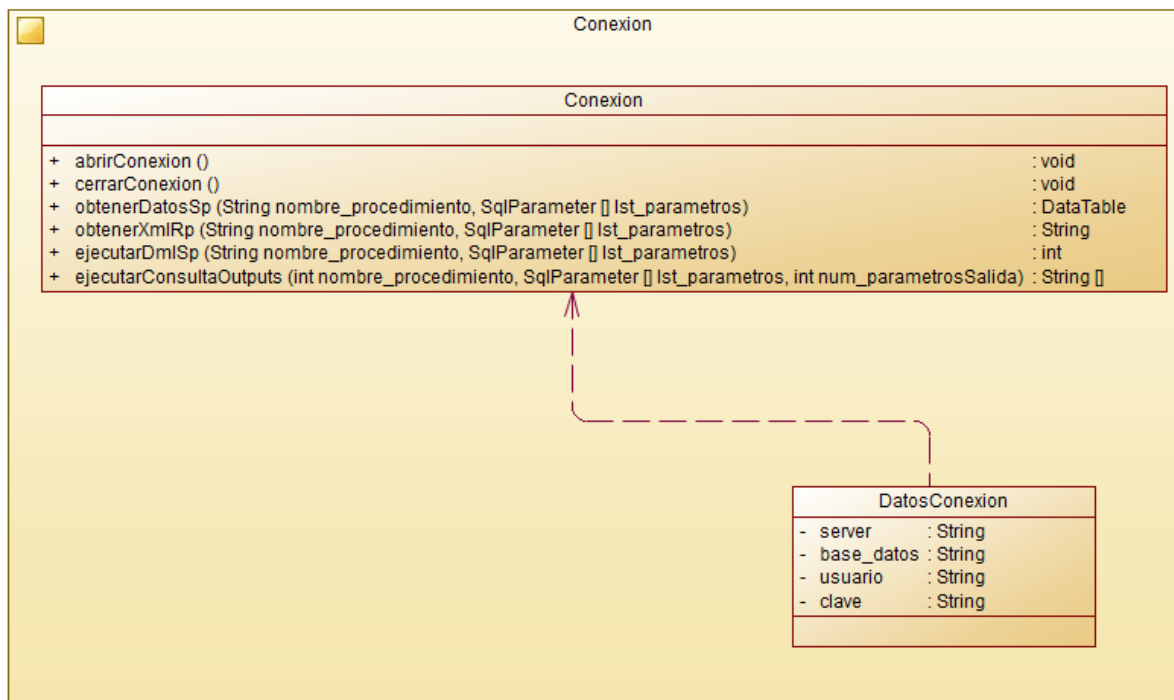


Fig. 27. Diagrama de clases – Paquete de conexión V1.1

2.6.3. Paquete de controladores

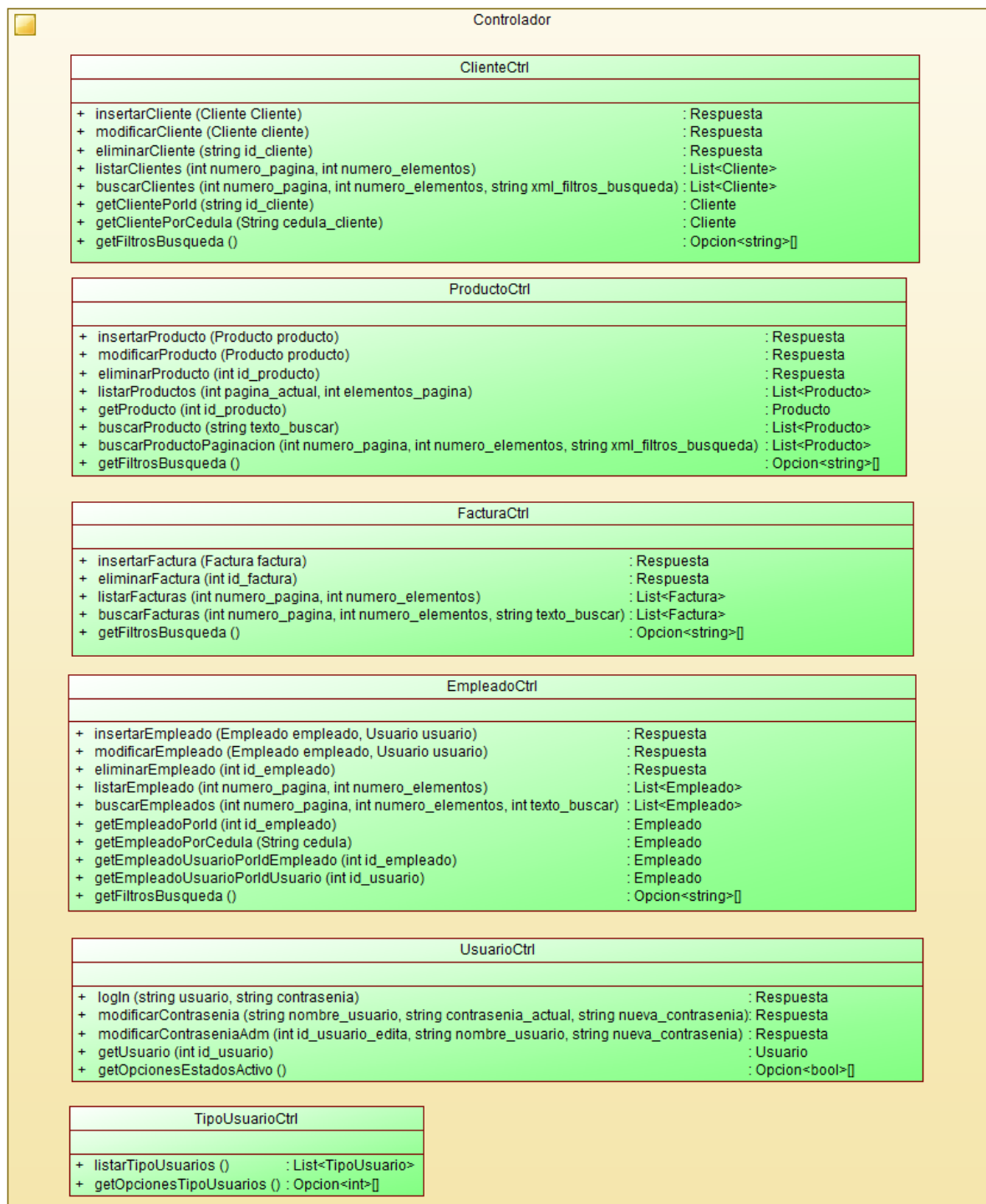
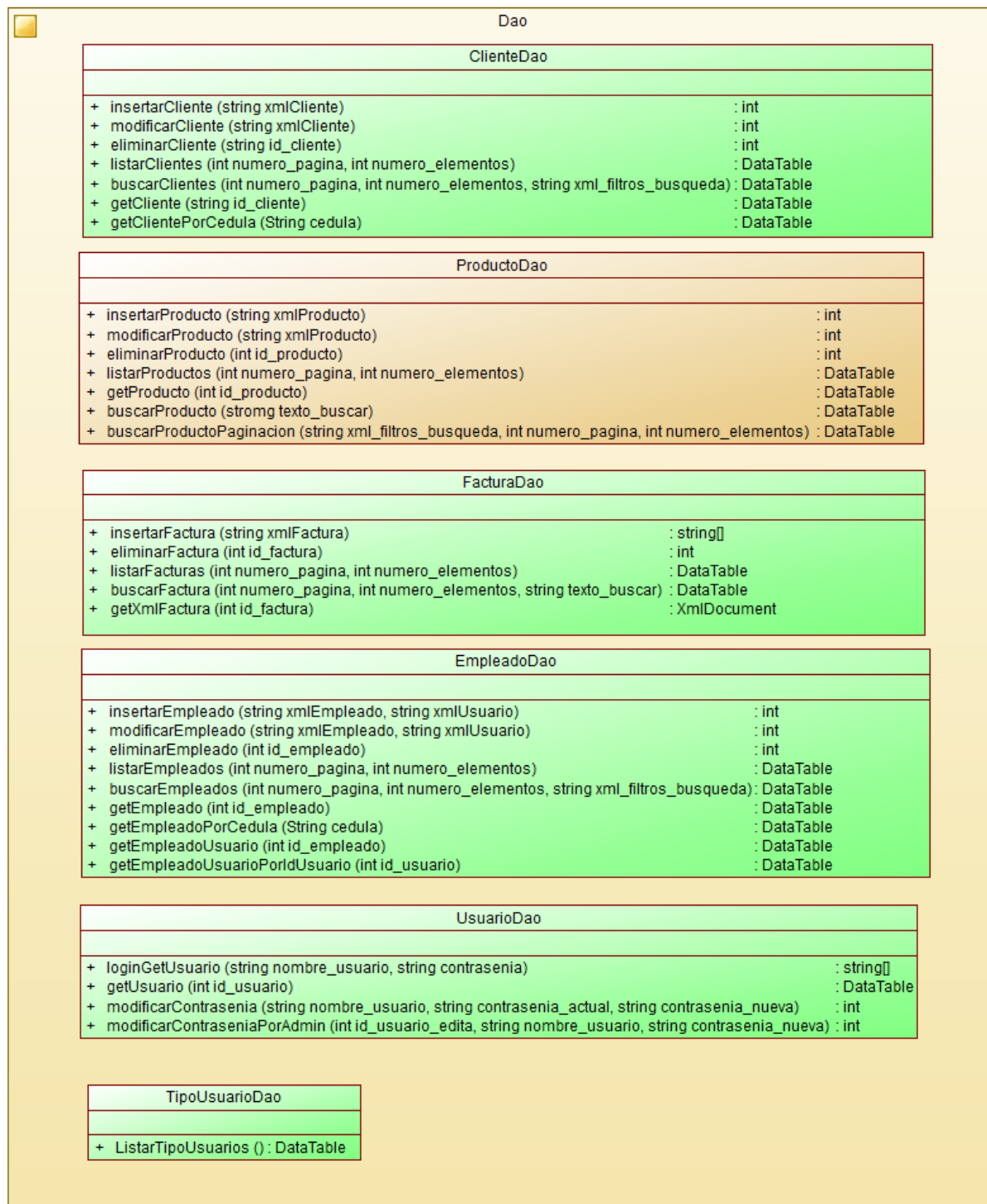


Fig. 28. Diagrama de clases – Paquete de Controladores V1.1

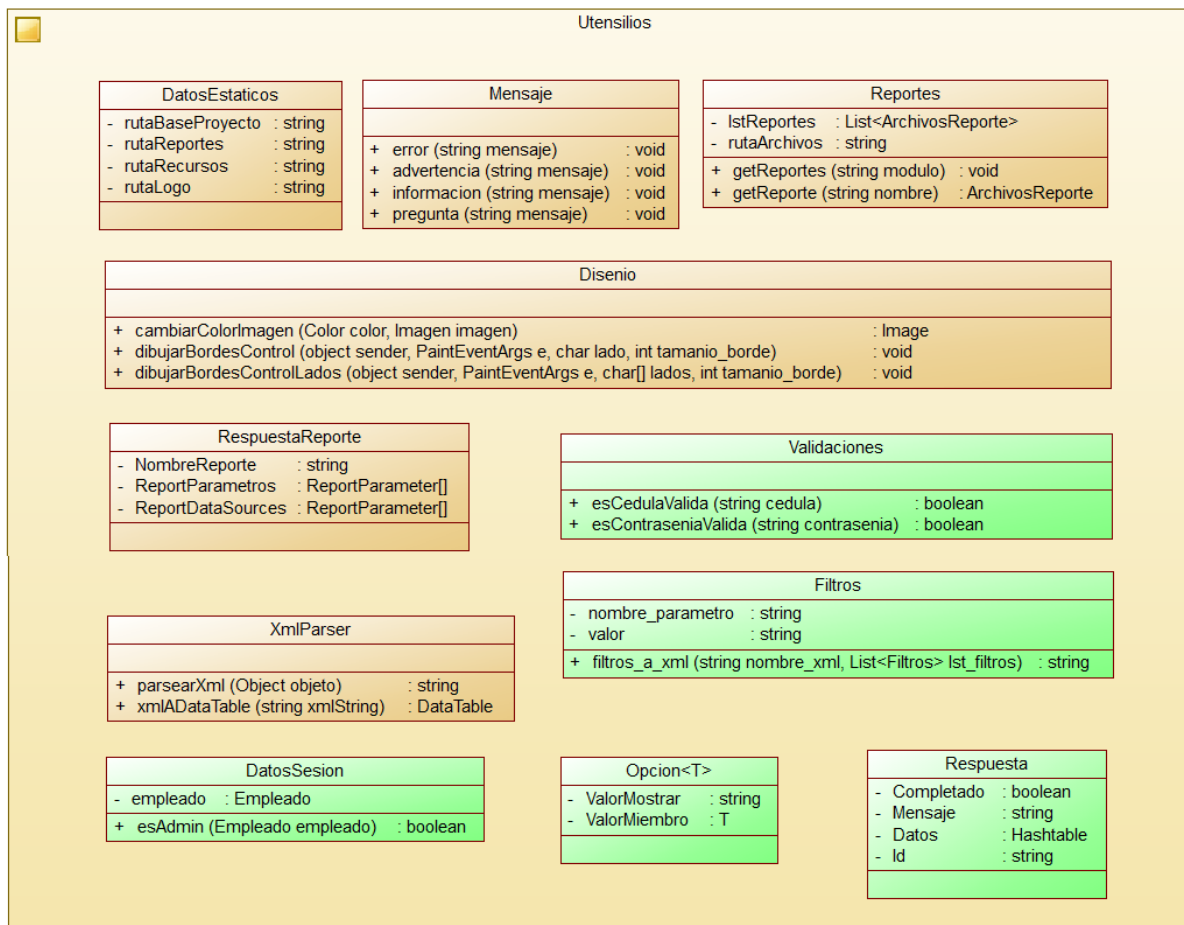


Fig. 29. Diagrama de clases – Paquete de controlador de reportes V1.1

2.6.4. Paquete DAO (Objeto de acceso a datos)



2.6.5. Paquete de utensilios

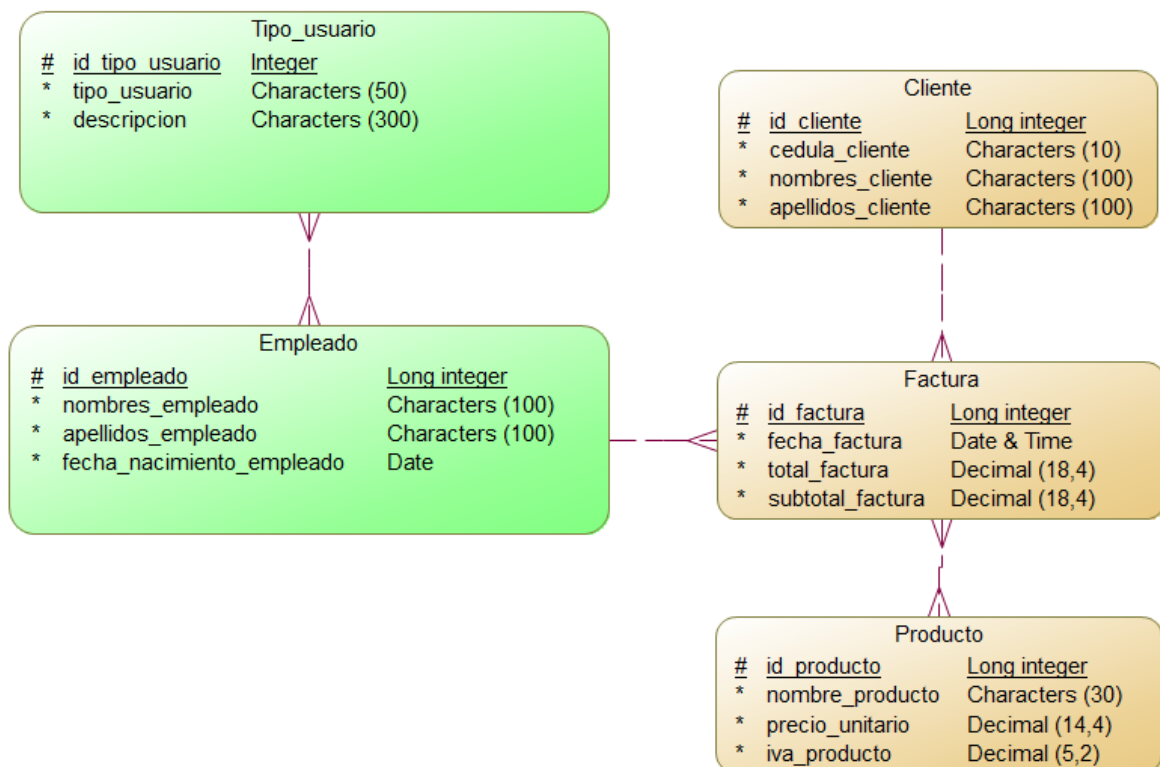


2.7. Diagramas de bases de datos

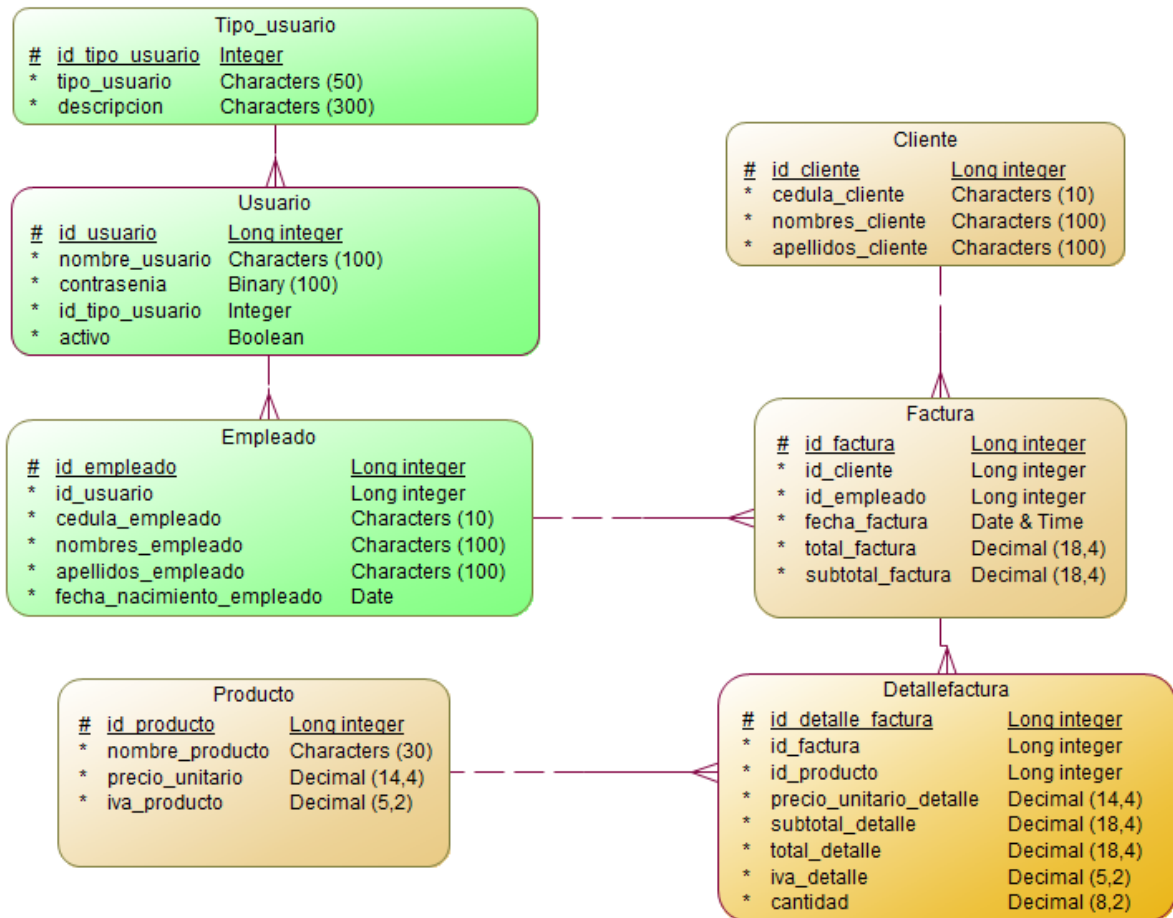
Debido a las modificaciones solicitadas por el cliente se notó la necesidad de modificar los diagramas de base de datos. De forma específica, se deben de agregar nuevas tablas, las cuales contienen información relacionada con los trabajadores o empleados, para especificar su rol y la relación que tienen en las facturas; es decir, visualizar al vendedor que realiza una venta y genera una factura.

Además, cada uno de los empleados tendrán sus credenciales (usuario, contraseña y tipo de usuario) para tener acceso a funciones específicas. Así, los diagramas conceptual, lógico y físico de la base de datos se muestran a continuación.

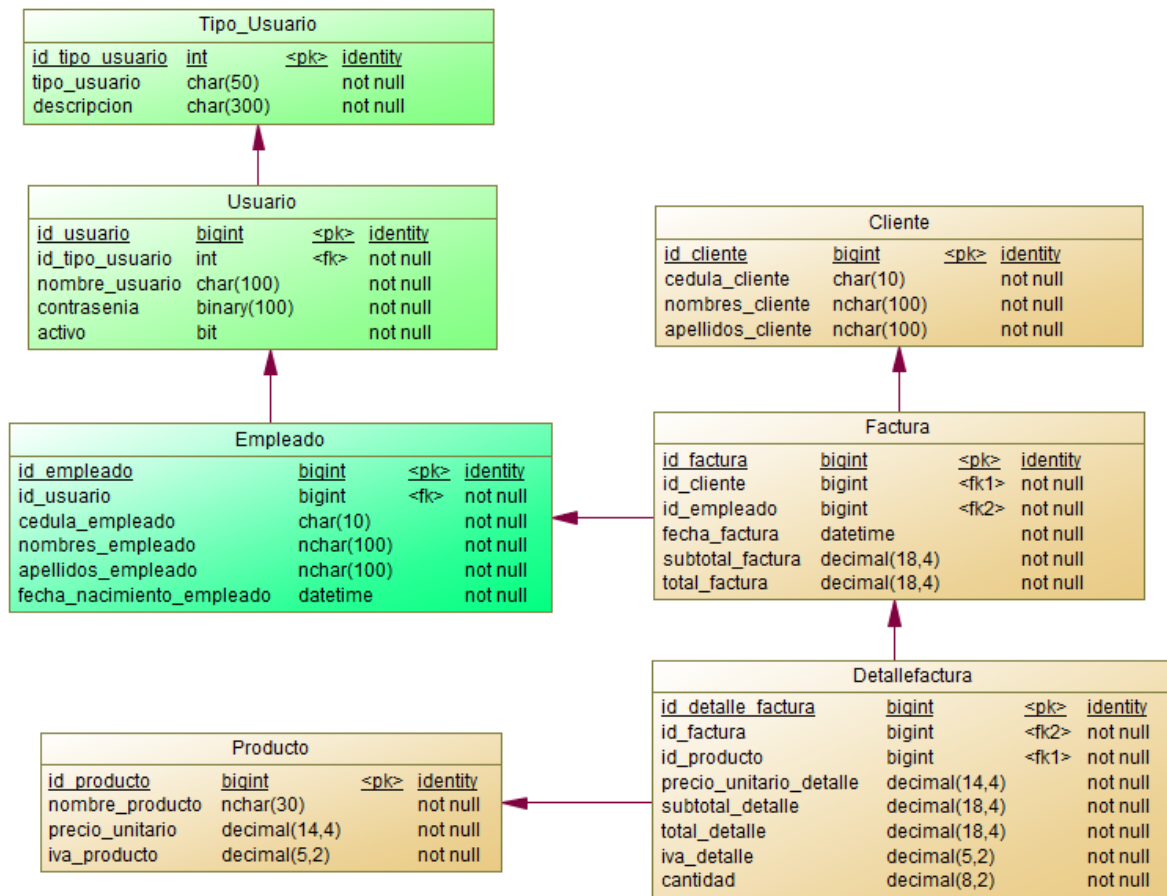
2.7.1. Conceptual



2.7.2. Lógico



2.7.3. Físico



2.7.4. Diagrama en SQL Server



3. Sistema de facturación V1.2

Esta versión del sistema surgió de la solicitud del cliente en la cual mostraba la descripción de los cambios que se debían realizar además con la fecha límite para su presentación. También se mostraban las razones por las que se requerían los cambios, se hizo un análisis del impacto del desarrollo de los cambios en el proyecto; o sea, que ICS se verían afectados y podrían afectar funciones ya implementadas, formularios clases, etc. La principal funcionalidad que se desea agregar es incluir proveedores, en la que cada producto esta relacionado directamente a un proveedor. A continuación, se presentan los documentos que sirven de soporte para desarrollar e implementar esta versión.

3.1. Solicitud de cambios en el software

AGILE SOFTWARE SOLUTIONS

Quevedo Shopping Center 2do piso, local N° 9.

Email: ags@solutions.com.ec.

Teléfono: 052-785-547

Solicitud de Cambios en el Software N°: 0003-2024

Fecha: 22/08/2024 14:48

Datos del cliente

Nombres y apellidos: Ing. Elías Portilla Olvera **Cédula:** 1203838566 **Teléfono:** 0996545783

Email: eportilla@uteq.edu.ec

Nombre del software a modificar: Sistema de Facturación UTEO V1.1

DESCRIPCIÓN DE LOS CAMBIOS

A. Descripción de los cambios:

- Implementar la funcionalidad de registrar proveedores en el sistema.
- La implementación de proveedores en el sistema debe de incluir las funcionalidades de agregar, modificar y eliminar proveedores en los productos
- Cada producto debe estar asociado con los proveedores para identificar a que proveedor corresponde cada uno de ellos.

Observaciones:

Lo que principalmente se requiere asegurar de que cada producto se encuentre relacionado directamente con un proveedor, cabe mencionar que no se está solicitando que se realice el proceso de compra de productos a proveedores, si no que cada uno de los productos registrados se encuentren asociados directamente con los proveedores.

B. Desarrolladores que realizan el estudio:

Apellidos y nombres de los revisores técnicos dela solicitud.

- Casanova Morante Héctor Miguel.
- Cedeño Salvatierra Orlando Jesús.
- Jaime Pesantez Anderson David.
- Leturne Plúas Jhon Byron.
- Piña Paredes Miguel Ángel.
- Tenecela Intriago Angélica Michelle.

B. Resumen de estudio de viabilidad:

El cambio solicitado si es viable. En cuestiones de recursos se cuenta con el personal necesario, equipos y software para el desarrollo de la nueva versión del software. Con respecto a la viabilidad técnica, la incorporación de esta funcionalidad no afecta de manera considerable a los otros módulos o funciones ya existentes. Sin embargo, este cambio provoca realizar modificaciones en los diagramas representantes del sistema. Se estima que el tiempo requerido para la implementación y las pruebas requeridas de esta función es de dos semanas (siendo hoy 22/08/2024), misma fecha que coincide con la requerida.

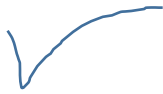
Resultados:

A: Se aprueba totalmente [☒]

B: Se aprueba parcialmente [☐]

C: Se rechaza totalmente [☐]

Otra: _____



Cliente: Ing. Elías Portilla Olvera



Recibido por: Casanova Morante Héctor Miguel

3.2. Orden de cambio de ingeniería (OCI)

OCI – Software de Facturación de línea de referencia	
Identificación: OCI_002_2024	Referencia de solicitud de cambio: 002_2024
Fecha de emisión de la OCI: 22/08/2024 16:24	Fecha de la solicitud de cambio: 22/08/2024 14:11
Línea de referencia: Sistema de facturación UTEQ v1.1. Versión resultante: Sistema de facturación UTEQ v1.2.	Solicitante del cambio: Elías Portilla Olvera (rol de cliente)
Descripción de los cambios: • Agregar opciones para registrar los proveedores de cada producto para identificar a que proveedor le corresponde cada producto • Agregar las funcionalidades de registrar, modificar y eliminar proveedores en el producto • Relacionar todos los productos registrados con sus correspondientes proveedores. Se requiere que los cambios estén listo máximo el 05/09/2024	
Razón de cambio: En el sistema de facturación se necesita implementar la funcionalidad que permita registrar proveedores, para tener la capacidad de identificar a que proveedor le corresponde cada producto dentro del sistema; por esta razón, se adjuntan las siguientes razones de los cambios requeridos: • Al registrar un nuevo producto se debe de especificar a que proveedor le pertenece, para relacionarlo y poder identificarlo en el caso de que sea necesario. • Si se registró un producto y los datos del proveedor se ingresaron de manera incorrecta el sistema tiene que tener la funcionalidad que me permita corregir la información y guardarla nuevamente. • En el caso de que se quiera eliminar un proveedor en específico este solo se eliminará cuando no tenga ningún producto asociado. • No hay un control en los productos en los que permita crear una relación de proveedores con sus correspondientes productos, que incluya agregar sus datos necesarios en el sistema, por eso es necesario implementar una nueva funcionalidad que incluya relacionarlos. • Cada producto tiene que tener especificado directamente solo con un proveedor, no puede existir más de un proveedor para un mismo producto. • Es necesario implementar una sección de visualización de proveedores en donde se forme una lista con toda la información necesaria. • Es necesario implementar la búsqueda de proveedores para reducir el tiempo de búsqueda de un proveedor en específico.	
Impacto en el proyecto: El impacto que tendrá estos cambios en el software son medianamente significantes ya que implican modificar y agregar nuevas funcionalidades. En la siguiente lista se encuentran los elementos que serán afectados o creados por los cambios: • Modificación de los diagramas desarrollados en PowerDesigner (diagrama de casos de uso, diagrama de clases y base de datos) debido al requisito de insertar proveedores. • Formulario de visualización o lista de proveedores para permitir al usuario realizar la búsqueda de manera rápida. • DataGridView de la lista de proveedores para mostrar el producto que esta relacionado con el proveedor. • Agregar un formulario para el registro de proveedores con sus respectivos datos. • Modificación de la estructura de la base de datos y procedimientos almacenados para el cumplimiento de los cambios. • Modificación de clases existentes y creación de nuevas clases en el sistema. • CRUD en conjunto a su formulario para administrar los proveedores.	

Restricciones: - En todos los formularios con DataGridView: - Restricción: No cargar todos los datos por defecto en el DataGridView; cargar los datos solo cuando el usuario busque algún registro. - En los formularios que listan proveedores: - Restricción: Mejorar la barra de búsqueda permitiendo al usuario seleccionar el campo por el cual buscar. La búsqueda debe activarse mediante un botón y no en base al evento de cambio de texto. - En el proceso de selección de proveedores: - Restricción: La selección de un proveedor debe realizarse mediante el evento de doble clic. - En el proceso de eliminación de proveedor: - Restricción: en el caso de querer eliminar un proveedor y este tiene productos relacionados el sistema no debe de permitir eliminarlo, solo debe de eliminarse en el caso de que no tenga ningún producto relacionado. - Productos y proveedores: - Restricción: cada producto solo podrá estar relacionado con un proveedor, no pueden existir productos que estén relacionados con mas de un proveedor.	
Criterio para revisar y auditar: - Revisar la nueva versión del software con los cambios solicitados. - Evaluar la usabilidad del software, considerando la experiencia del usuario. - Revisar los modelos (casos de uso, clases y base de datos) del sistema anterior y actual. - Revisar el código fuente del sistema anterior y actual. - Inspeccionar la compatibilidad entre los modelos y el código fuente.	
Número de sprint que se tomaran: 2 sprint (cada sprint tiene una duración de una semana).	
Responsables de la implementación: - Casanova Morante Héctor Miguel. - Cedeño Salvatierra Orlando Jesús. - Jaime Pesantez Anderson David. - Leturne Pluas Jhon Byron. - Tenecela Intriago Angélica Michelle. - Piña Paredes Miguel Ángel	
Recursos necesarios: - SQL Server Management Studio 20. - SQL Server 2019 Express - IDE – Visual Studio Community 2019 (Versión: 16.11.12). - Microsoft .NET Framework (Versión: 4.7.2) - Power Designer (Versión 16.7 SP05). - Lenguaje de programación C#. - Git Bash (Versión 2.38.1. windows.1) 5 ordenadores con los softwares previamente especificados instalados.	
Aprobado por: Casanova Morante Héctor Miguel (ACC)	
Fecha Inicio de la Implementación: 22/08/2024	Fecha a entregar los cambios: 05/08/2024
Estatus: Por implementar	Prioridad: Alta
Comentarios adicionales: ninguno.	
 Firma del solicitante (Elías Portilla Olvera)	 Firma del aprobador – ACC (Héctor Casanova)

3.3. Línea de referencia

LÍNEA DE REFERENCIA	
Nombre del proyecto:	Sistema de facturación UTEQ
Referencia a la solicitud de cambio:	0002_2024
Referencia a la OCI:	OCI_002_2024
Versión de la línea de referencia:	V1.1
Repositorio de la línea de referencia:	https://github.com/jhonleturne192005/GCSfacturacion/tree/V1.1
Versión resultante:	V1.2
Repositorio de la versión resultante:	
ICS Modificados: La lista de elementos modificados o creados puede visualizarse en la celda de la derecha. La versión especificada al lado de cada componente es la base utilizada para su modificación.	• Diagramas de casos de uso (CU): 1. Diagrama general de CU (V1.1) 2. Diagramas de CU (Producto) (V1.1) • Casos de uso extendido: 1. Registrar producto (V1.1) 2. Modificar producto (V1.1) 3. Eliminar producto (V1.1) • Diagrama de actividades 1. Diagrama de actividades (Proveedor) 2. Diagrama de actividades (Producto) (V1.1) • Diagrama de flujo de datos (DFD) (V1.1) • Diagrama de base de datos 1. Conceptual (V1.1) 2. Lógico (V1.1) 3. Físico (V1.1) • Diagrama de clases 1. Paquete de conexión a base de datos (V1.1) 2. Paquete DTO (V1.1) 3. Paquete DAO (V1.1) 4. Paquete de Controladores (V1.1) 5. Paquete de utensilios (V1.1) • Documentación de la plataforma de desarrollo (V1.1) • Código fuente (V1.1)
Nuevos ICS: Lista de elementos de configuración de software creados para cumplir con los cambios requeridos por el cliente.	• Diagramas de casos de uso (CU): 1. Diagramas de CU (Proveedor) • Casos de uso extendido: 1. Registrar proveedor (Nuevo) 2. Actualizar proveedor (Nuevo) 3. Listar proveedor (Nuevo) 4. Eliminar proveedor (Nuevo) 5. Visualizar proveedor (Nuevo) 6. Relacionar producto con proveedor (Nuevo) • Diagrama de actividades

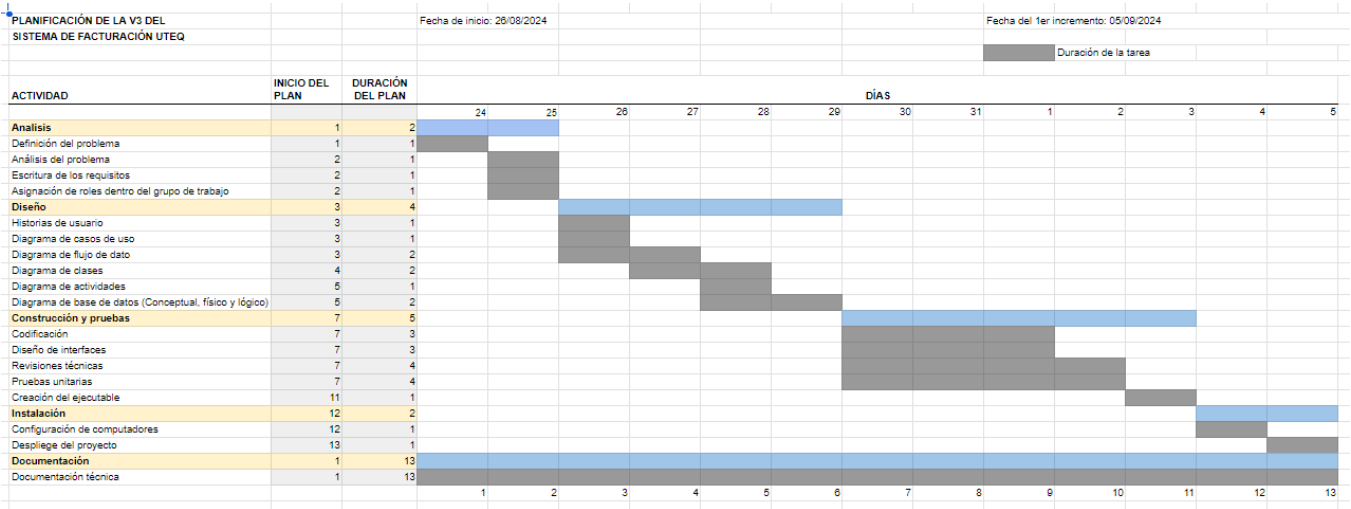
1. Diagrama de actividades (Proveedor)
--

3.4. Requisitos

Para esta nueva versión del software se ha solicitado agregar una sección de proveedores, la cual principalmente va a estar relacionadas con los productos, cabe mencionar que la solicitud del cliente tiene como propósito general relacionar cada producto con un proveedor, los requisitos que giran en torno a esta nueva solicitud se detallan a continuación:

- Agregar opciones para registrar los proveedores de cada producto para identificar a que proveedor le corresponde cada producto
- Agregar las funcionalidades de registrar, modificar y eliminar proveedores en el producto
- Relacionar todos los productos registrados con sus correspondientes proveedores.

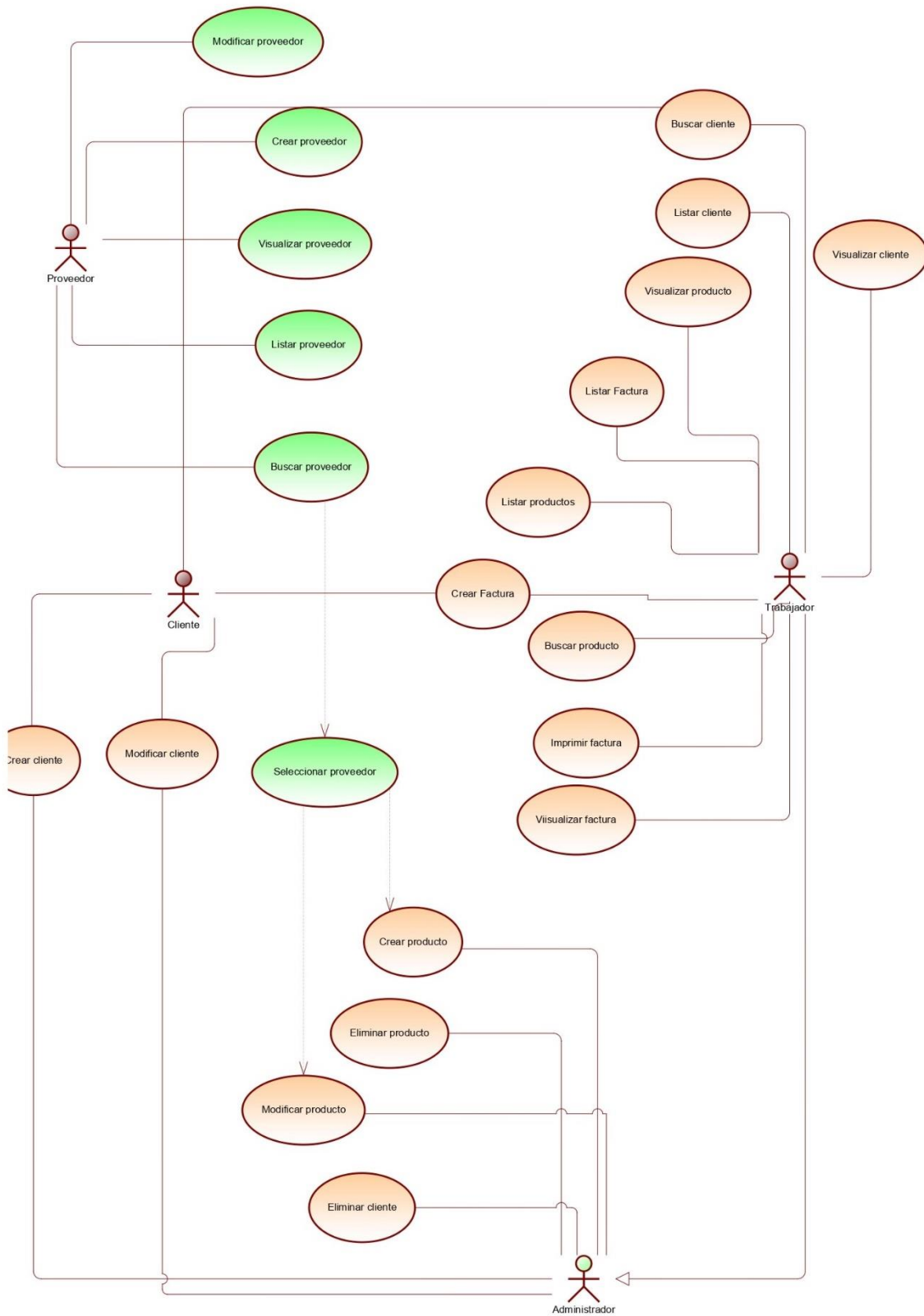
3.5. Planificación



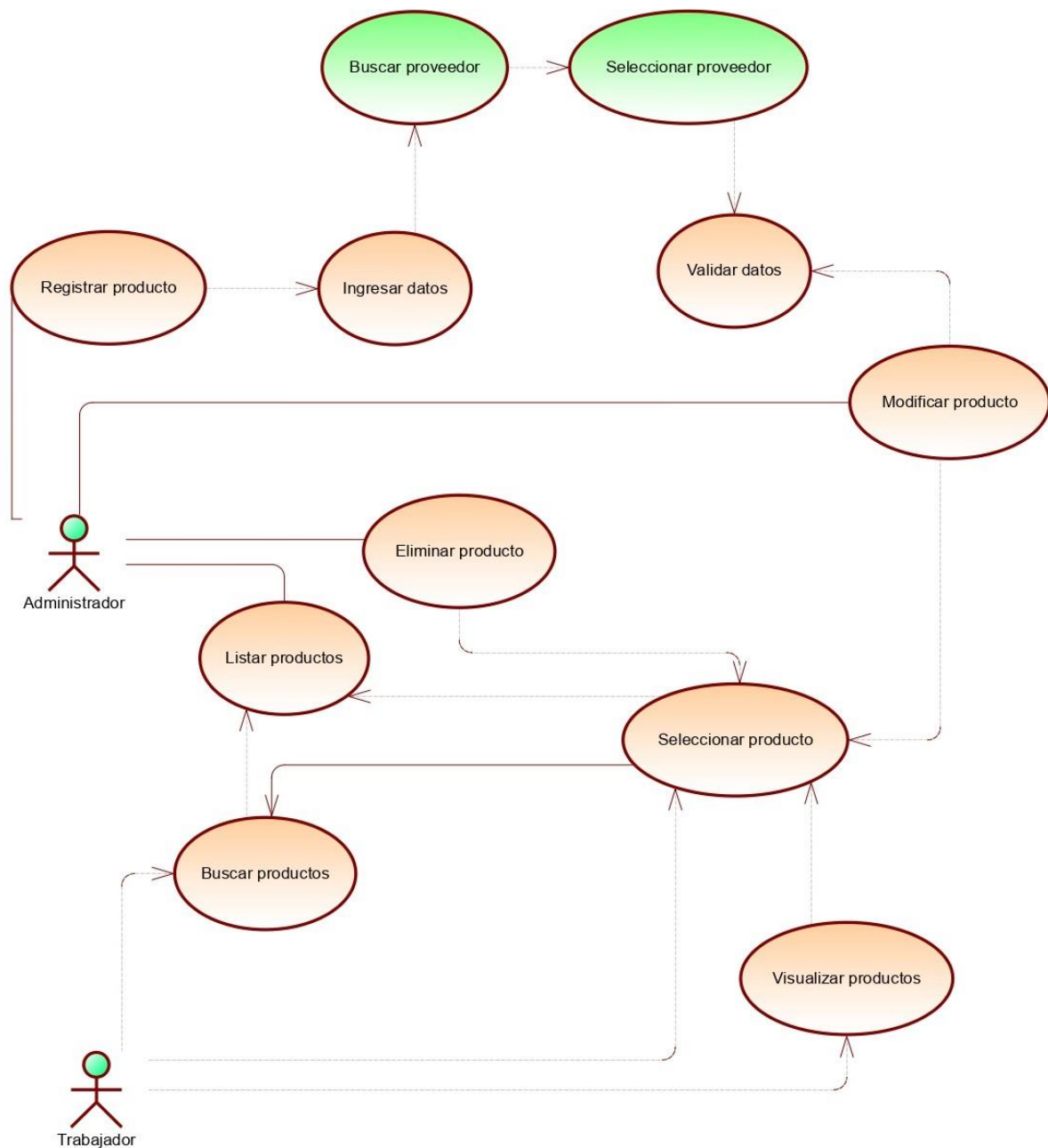
3.6. Diagramas de casos de uso

3.6.1. Diagrama general

Al diagrama de casos de uso general, se le ha implementado proveedor, el cual está relacionado con productos, en las que cada producto está relacionado con un proveedor, en este sentido también el proveedor se lo puede agregar, modificar, visualizar y listar. Cada una de estas acciones se las ha incorporado a partir de la solicitud como nuevo requisito del cliente.



3.6.2. Diagrama de CU (Producto)



El caso de uso de producto ha sido modificado a partir de la incorporación de proveedor, lo cual conlleva a incluir, dos actividades más al momento de registrar un nuevo producto, en la que al registrar un producto se enlistan los proveedores registrados para posteriormente seleccionarlo y de esta forma vincularlo con el producto que se está registrando.

3.7. Casos de uso

3.7.1. Caso de uso: Registrar proveedor

Caso de uso	Registrar proveedor	Excepciones	
Iteración	1, última modificación: 04/09/2024	5.1. El sistema retroalimenta al usuario mostrando una notificación con los campos erróneos en el registro de proveedor.	
		Prioridad	Alta
Actor principal	Trabajador.	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Registrar un proveedor en el sistema con los datos brindados por el mismo.	Frecuencia de uso	Alta, debe implementarse de inmediato.
Precondiciones	El sistema debe estar funcionando correctamente, deben obtenerse los datos del proveedor.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un proveedor no existente en el sistema y se desea asignarlo a un producto.	Actores secundarios	Trabajador
Escenario		Canales a los actores secundarios	
Actores	Sistema	Proveedor: Interacción directa con el trabajador.	
1. El trabajador seleccione el botón “Crear proveedor”			

2. El trabajador procede a pedirle los datos al proveedor para registrarlo en el sistema.		
3. El proveedor brinda los datos solicitados por el trabajador.		Asuntos pendientes 1. ¿Qué mecanismos son los correctos para proteger que los datos ingresados en la base de datos sean íntegros?
4. El trabajador presiona el botón “Guardar”.		
	5. El sistema verifica que los datos cumplen los parámetros necesarios (longitud, campos vacíos, entre otras validaciones)	
	6. El sistema guarda los datos.	
	7. Muestra una notificación de “Datos guardados correctamente”.	

8. El trabajador notifica al proveedor que sus datos fueron registrados de manera correcta.		
---	--	--

3.7.2. Caso de uso: Modificar proveedor

Caso de uso	Modificar proveedor	Excepciones	
Iteración	1, última modificación: 04/09/2024	2.1. El proveedor no está registrado en el sistema, por lo que procede a crearlo (ver el caso de uso Registrar proveedor). 7.1. El sistema retroalimenta al usuario mostrando una notificación con los campos erróneos en la modificación de los datos del proveedor.	
		Prioridad	Media
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Actualizar los datos del cliente con los nuevos proporcionados por el mismo.	Frecuencia de uso	Media
Precondiciones	El sistema debe estar funcionando correctamente. Además, el cliente debe estar registrado en el sistema.	Canal al actor	A través de un computador con conexión a internet.
Disparador	El cliente decide modificar sus datos en el sistema.	Actores secundarios	Proveedor

Escenario		Canales a los actores secundarios
Actores	Sistema	Proveedor: Interacción directa con el trabajador.
1. El proveedor se acerca al trabajador para actualizar sus datos en el sistema.		
2. El trabajador busca al proveedor por sus nombres o cédula.		
3. El trabajador presiona “Lista de proveedor” (ver el caso de uso Listar proveedor) y selecciona el proveedor a modificar y presiona el botón “Editar”.		
4. El trabajador procede a pedirle los datos al proveedor para modificarlos en el sistema.		
5. El trabajador presiona el botón “Guardar”.		Asuntos pendientes 1. ¿Qué mecanismos son los correctos para proteger que los datos modificados

6. El trabajador presiona el botón “Actualizar”.		en la base de datos mantengan su integridad?
	7. El sistema verifica que los datos cumplen los parámetros necesarios (longitud, campos vacíos)	
	8. El sistema guarda los datos.	
	9. Muestra una notificación de “Datos modificados correctamente”.	
10. El trabajador notifica al proveedor que sus datos fueron modificados.		

3.7.3. Caso de uso: Eliminar proveedor

Caso de uso	Eliminar proveedor	Excepciones
Iteración	1, última modificación: 04/09/2024	3.1. El proveedor no está registrado en el sistema, por lo que procede a crearlo (ver el caso de uso Registrar proveedor). 5.1. El proveedor no puede ser eliminado ya el registro

		correspondiente está relacionada con otras tablas.	
		Prioridad	Media
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Eliminar (desactivar) el proveedor del sistema.	Frecuencia de uso	Baja
Precondiciones	El sistema debe estar funcionando correctamente. Además, el proveedor debe estar registrado en el sistema.	Canal al actor	A través de un computador con conexión a internet.
Disparador	Un proveedor desea darse de baja del sistema.	Actores secundarios	Proveedor
Escenario		Canales a los actores secundarios	
Actores	Sistema	Cliente: Interacción directa con el trabajador.	
1. El proveedor se acerca al trabajador para eliminar sus datos del sistema.			
2. El trabajador presiona el botón “Listar proveedor” (ver el caso de uso Listar proveedor).			
3. El trabajador busca al proveedor por sus nombres o cédula			
		Asuntos pendientes	

4. El trabajador selecciona el proveedor y presiona el botón “Eliminar”.		
	5. El sistema procede a eliminar el proveedor del sistema.	
	6. Muestra una notificación de “Proveedor eliminado correctamente”.	
	7. El sistema redirige al trabajador a la lista de proveedores.	
8. El trabajador notifica al proveedor que ha sido dado de baja del sistema.		

3.7.4. Caso de uso: Listar proveedores

Caso de uso	Listar proveedores	Excepciones	
Iteración	1, última modificación: 04/09//2024	3.1. El sistema no retorna ningún dato.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento

Objetivo en contexto	Listar los proveedores registrados en el sistema.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador accede a la lista de proveedores registrados.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El trabajador presiona el botón “Listar proveedores”.			
	2. El sistema consulta a la base de datos los registros de la tabla proveedores.		
	3. El sistema retorna los datos de la base de datos.		
	4. El sistema muestra los datos de los proveedores en forma de lista.	Asuntos pendientes	
5. El usuario visualiza todos los datos cargados.		1. ¿Cómo debería diferenciarse los clientes dados de baja del sistema de los activos?	

3.7.5. Caso de uso: Visualizar proveedor

Caso de uso	Visualizar proveedor	Excepciones	
Iteración	1, última modificación: 04/09//2024	3.1. El sistema no retorna ningún dato.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Visualizar un proveedor específico que haya sido seleccionado por el trabajador.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador desea ver los datos detallados de un proveedor.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
El trabajador presiona el botón “Listar proveedores” (ver el caso de uso Listar proveedores).			
El trabajador selecciona un proveedor específico y presiona el botón “Ver”.	.		
	El sistema consulta a la base de datos los registros solicitados del proveedor seleccionado.		

	El sistema muestra los datos detallados del proveedor.	Asuntos pendientes
El trabajador visualiza los datos del proveedor seleccionado.		1. ¿Cómo debería diferenciarse los clientes dados de baja del sistema de los activos?

3.7.6. Caso de uso: Buscar proveedor

Caso de uso	Buscar proveedor	Excepciones	
Iteración	1, última modificación: 04/09//2024	3.1. El sistema no retorna ningún dato.	
		Prioridad	Alta
Actor principal	Trabajador	Cuando estará disponible	Al 1er incremento
Objetivo en contexto	Buscar un proveedor de interés por el trabajador.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente.	Canal al actor	A través de un computador y conexión a internet.
Disparador	El trabajador desea buscar dentro de la lista de clientes alguno que cumpla con características definidas por él (datos).	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
El trabajador presiona el botón “Listar proveedores” (ver el caso de uso Listar proveedores).			

El trabajador ingresa en un cuadro de texto que desea buscar en la lista de proveedores.	.	
	El sistema consulta a la base de datos los proveedores que cumplan con las especificaciones de búsqueda.	
	El sistema muestra los proveedores que cumplen las características de búsqueda.	Asuntos pendientes 1. ¿Cómo debería diferenciarse los clientes dados de baja del sistema de los activos?
El trabajador visualiza la lista de proveedores.		

3.7.7. Caso de uso: Crear productos (Modificación)

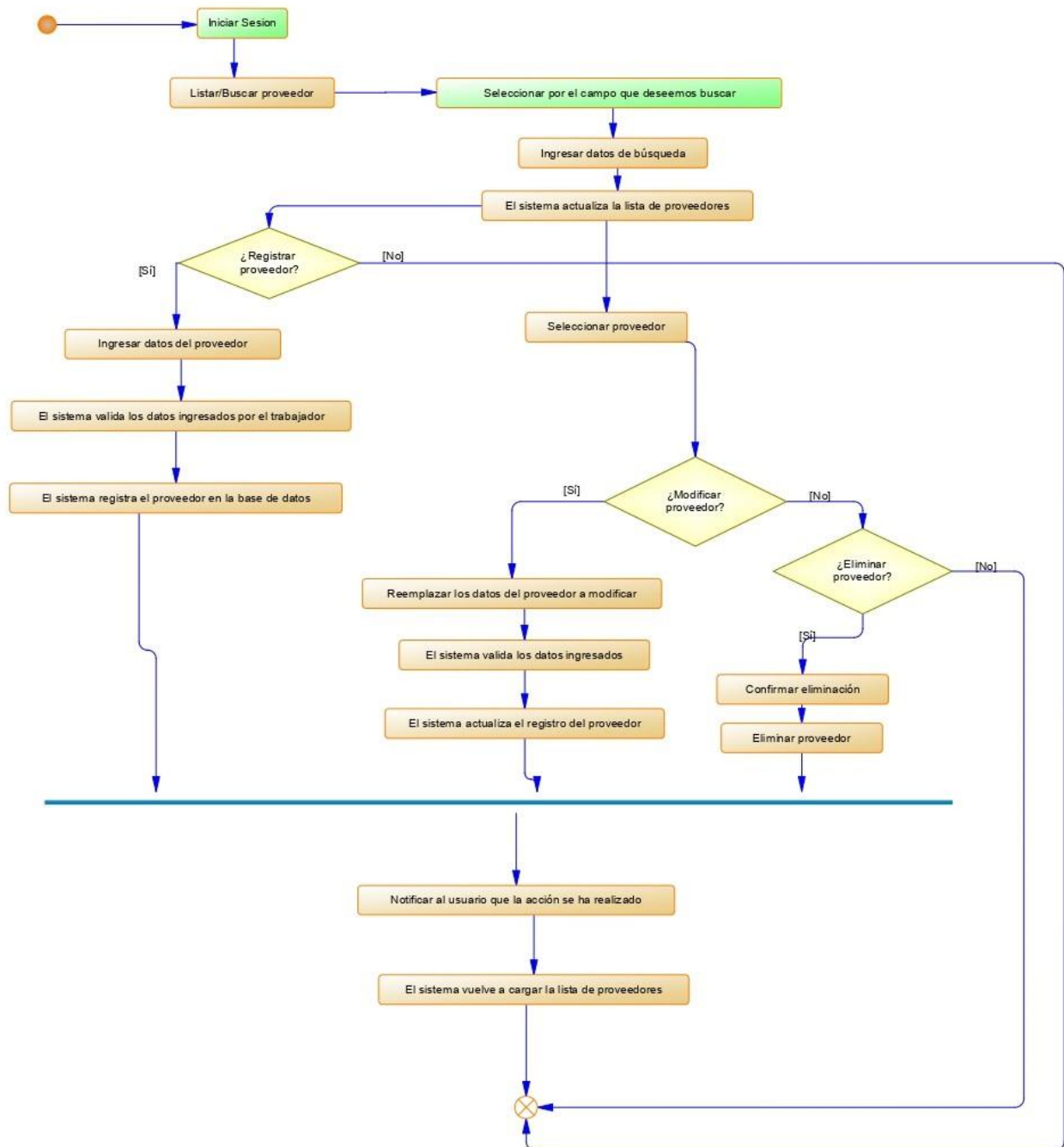
Caso de uso	Registrar producto	Excepciones	
Interacción	3, última modificación: 04/09/2024	4.1. Los datos ingresados no son válidos. Por lo tanto, se retroalimenta al usuario de los errores. 5.1. Al registrar los datos en la base de datos existe algún error que evita la inserción.	
		Prioridad	Alta
Actor principal	Administrador	Cuando estará disponible	Al 1er incremento

Objetivo en contexto	Permitir al administrador registrar productos al sistema.	Frecuencia de uso	Alta
Precondiciones	El sistema debe estar funcionando correctamente	Canal al actor	A través de un computador con conexión a internet.
Disparador	El Administrador desea registrar un producto en el sistema.	Actores secundarios	N/A
Escenario		Canales a los actores secundarios	
Actores	Sistema	N/A	
1. El administrador accede a la opción “Registrar producto” en la interfaz.			
	2. El sistema muestra un formulario para el ingreso de los datos del producto.		
3. El administrador ingresa los datos solicitados en el formulario.			
4. El administrador selecciona el proveedor del producto.			
	5. El sistema valida los datos ingresados (longitud y tipos).	Asuntos pendientes	
		3. ¿Debería permitirse utilizar diferentes tipos de moneda?	

	6. El sistema registra el producto en la base de datos.	
	7. El sistema notifica al trabajador que el producto ha sido registrado correctamente.	

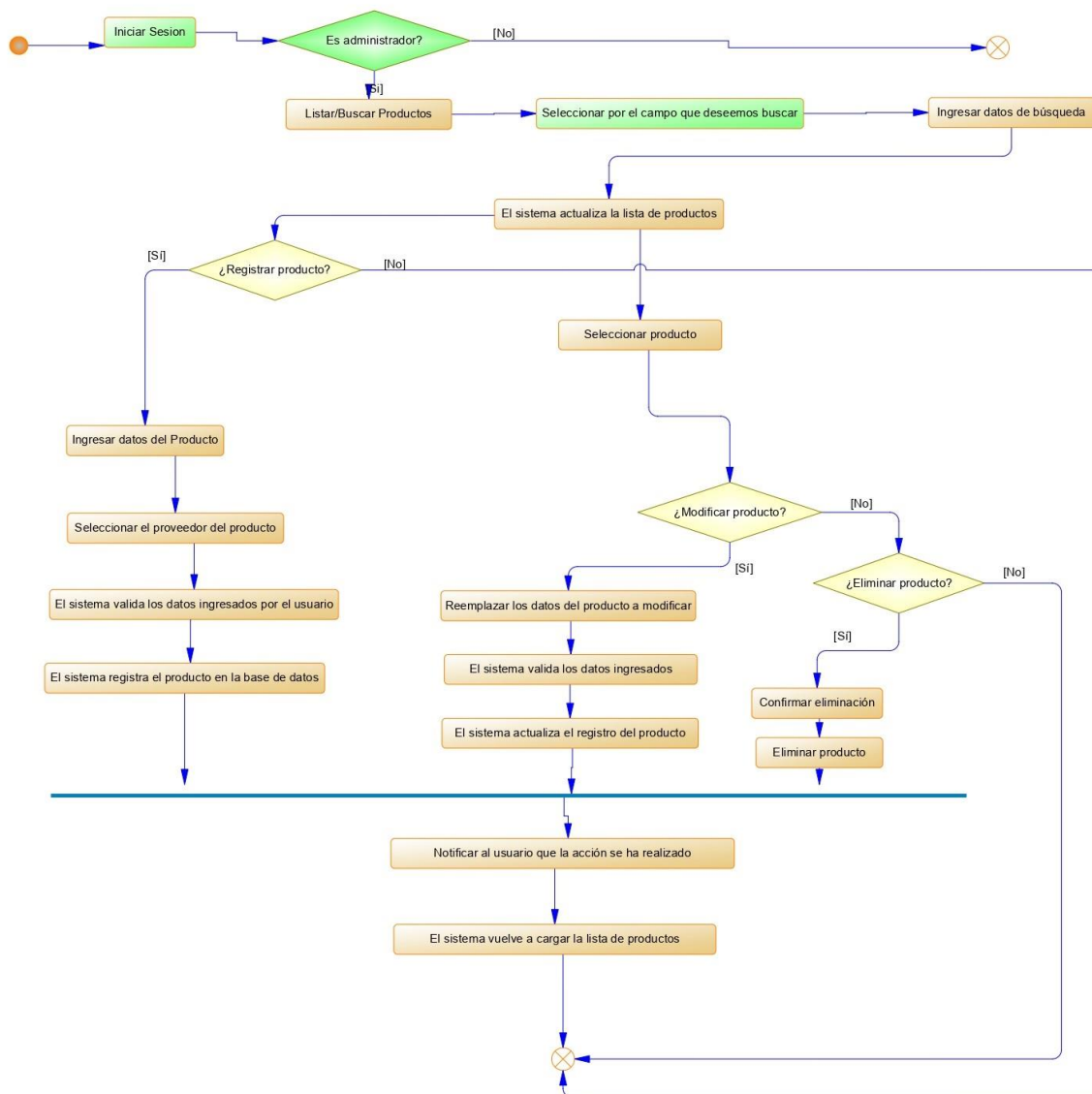
3.8. Diagramas de actividades

3.8.1. Diagrama de actividad (Proveedor)

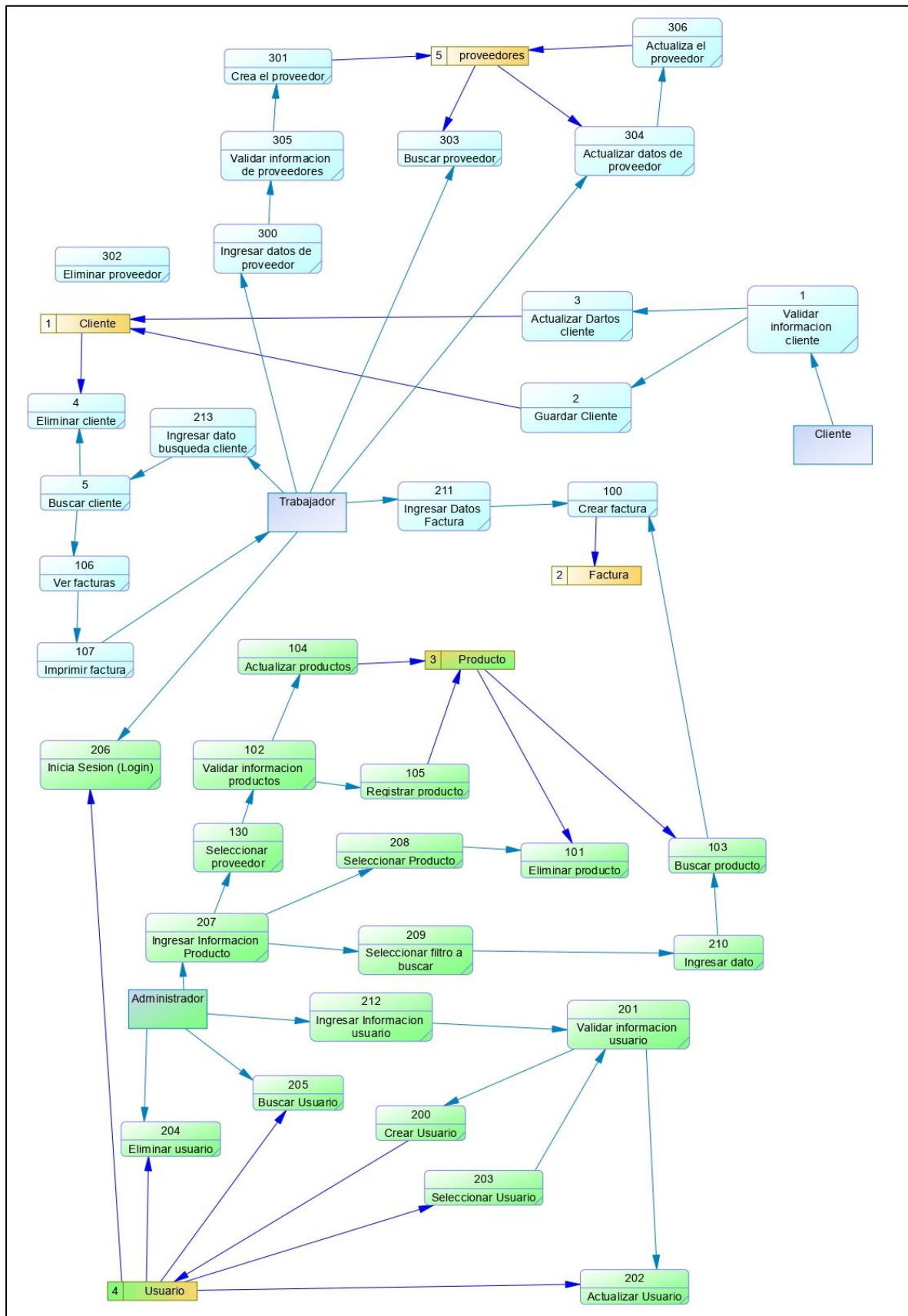


Para vincular a un producto con proveedor, primero se debe de registrar al proveedor, en el cual se deben de adjuntar los datos correspondientes al proveedor, una vez registrados, ahora pasa al proceso de registrar producto en la que se especifican los datos del producto y se enlistan los proveedores registrados actualmente, se selecciona el proveedor que le corresponde y se guarda el producto, en caso de que se necesite modificar los datos del proveedor o eliminarlo por completo el usuario puede seleccionarlo desde la lista de proveedores, para realizar las actividades mencionadas y posteriormente guardar los cambios.

3.8.2. Diagrama de actividad (Productos)

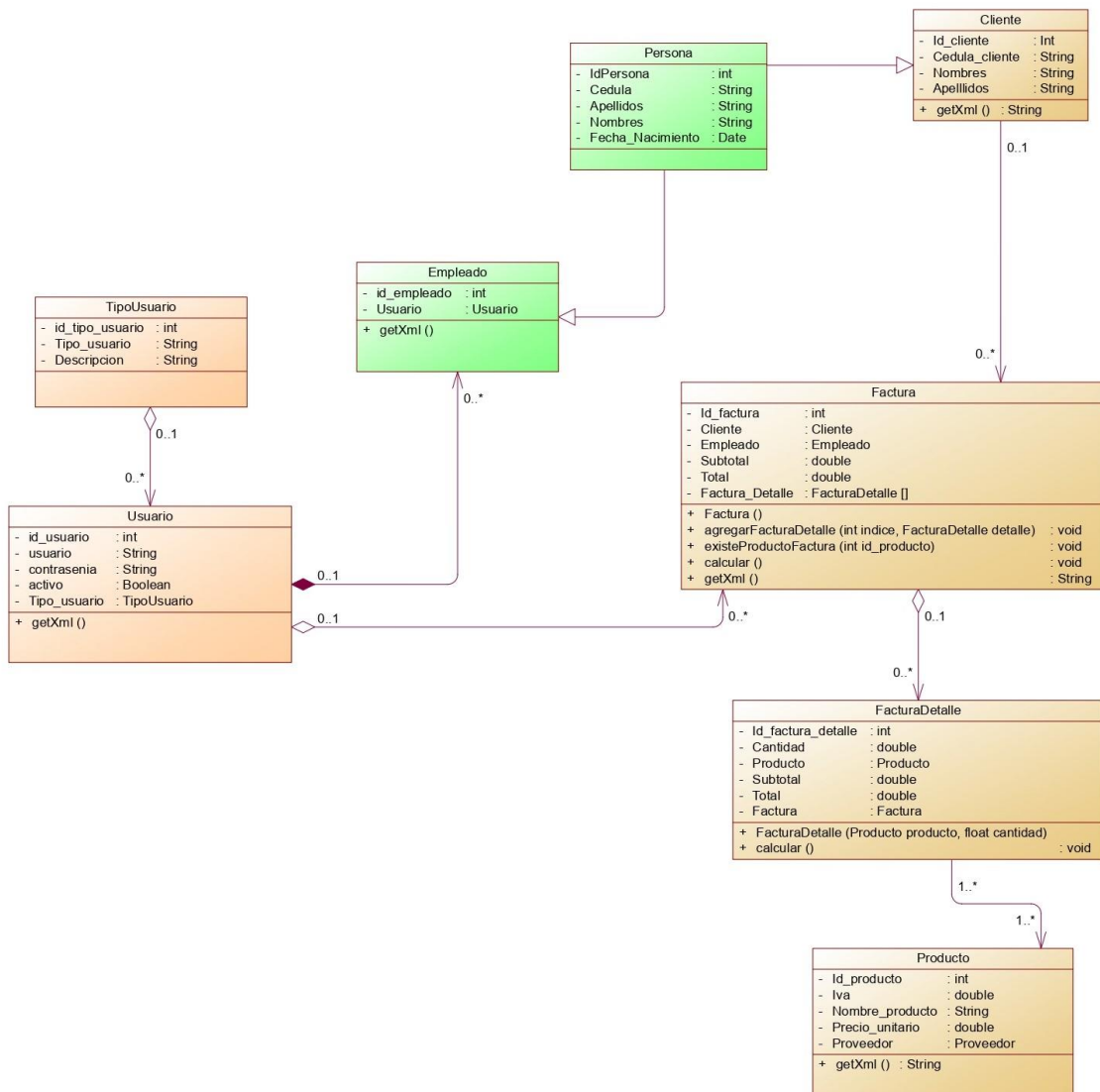


3.9. Diagrama de flujo de datos

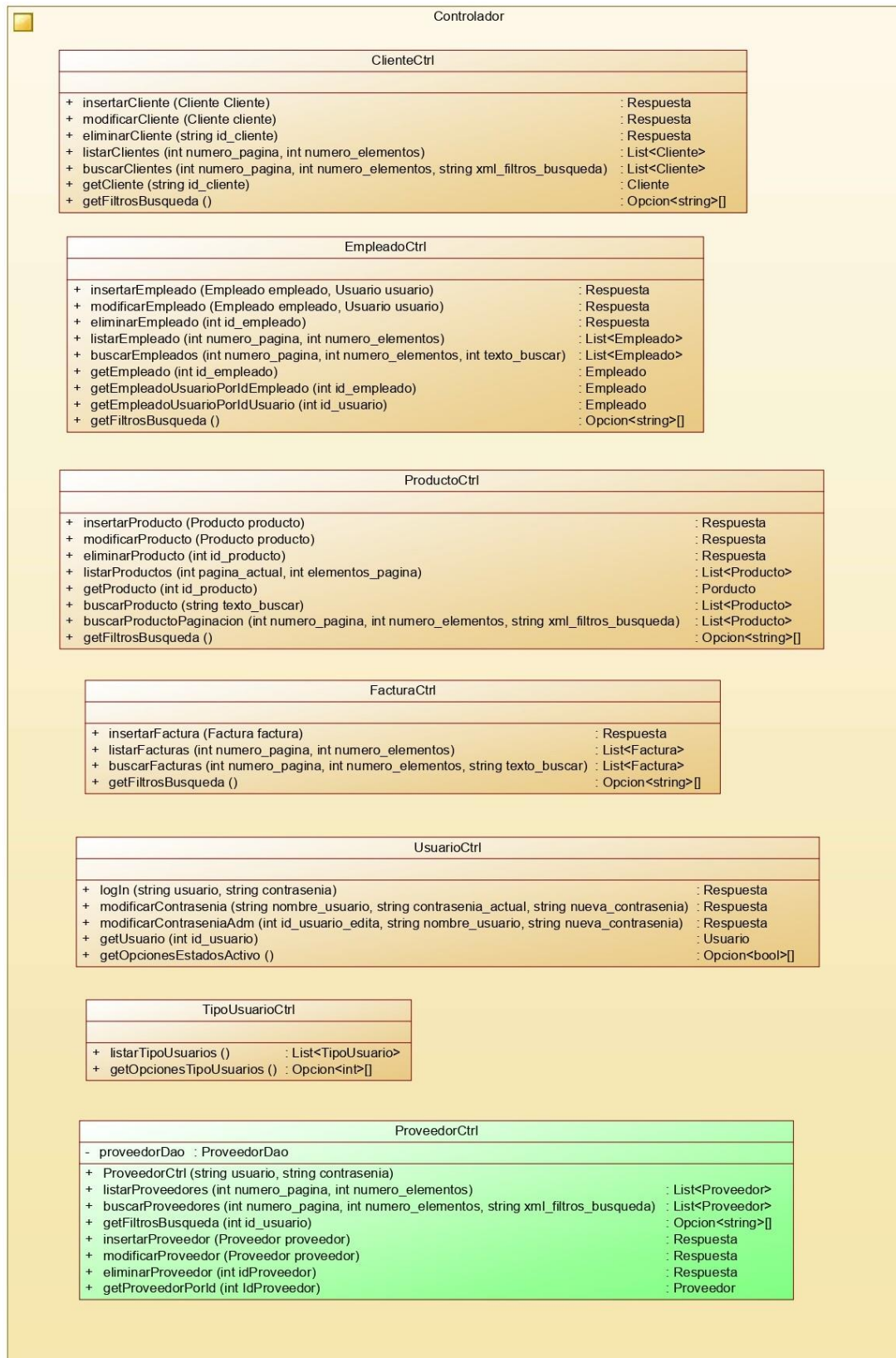


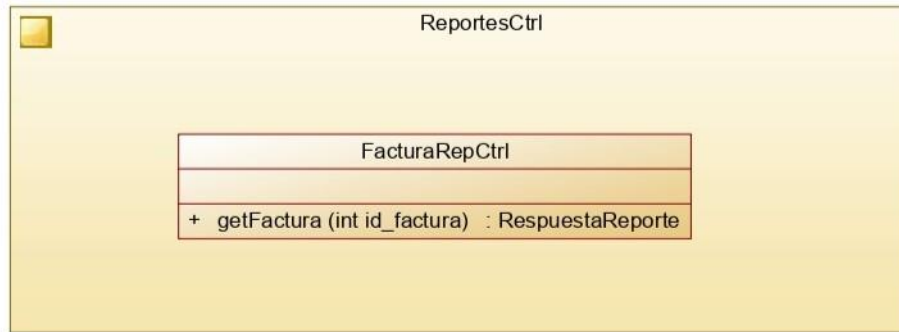
3.10. Diagrama de clases

3.10.1. Paquete DTO (Objeto de transferencia de datos)

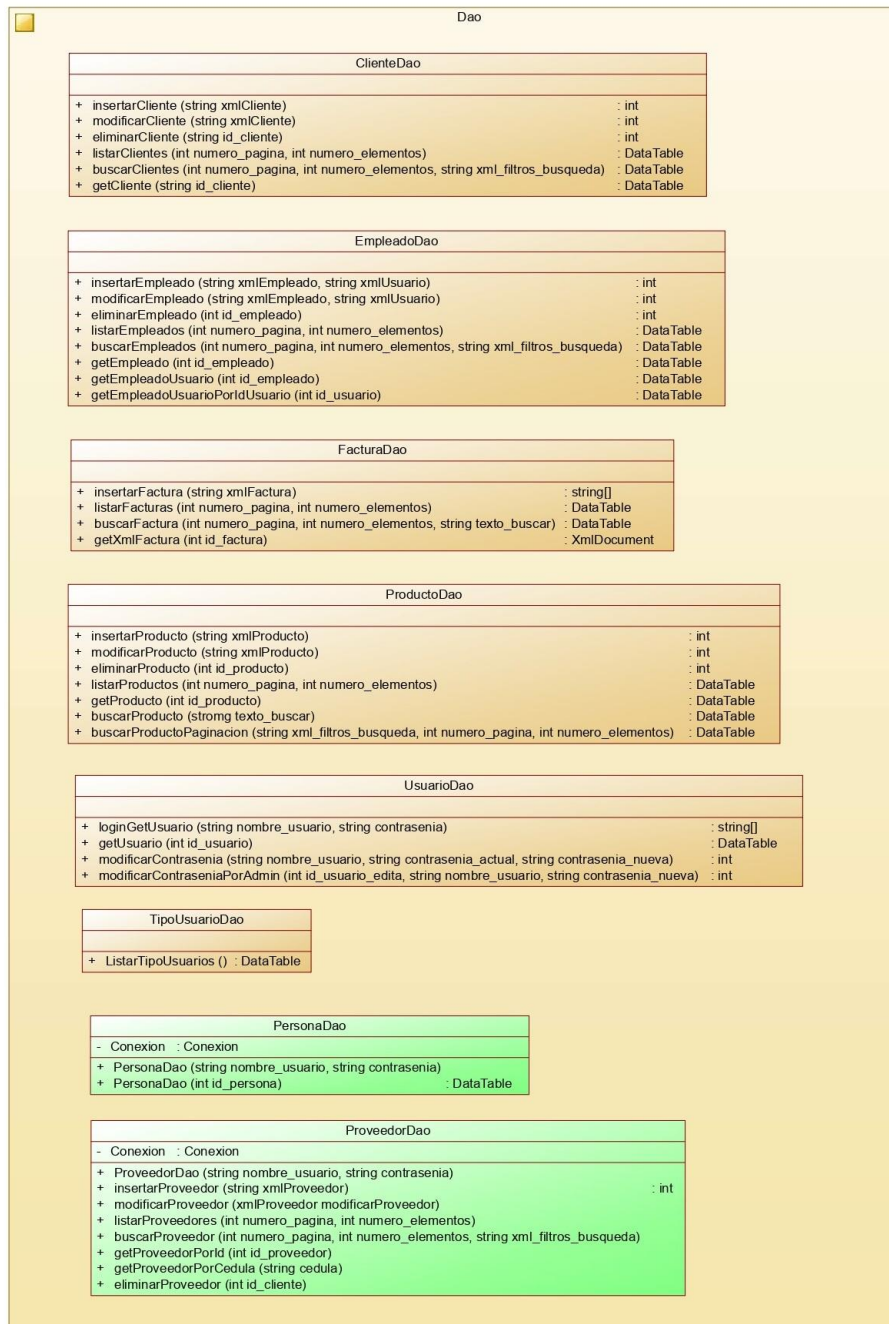


3.10.2. Paquete de controladores

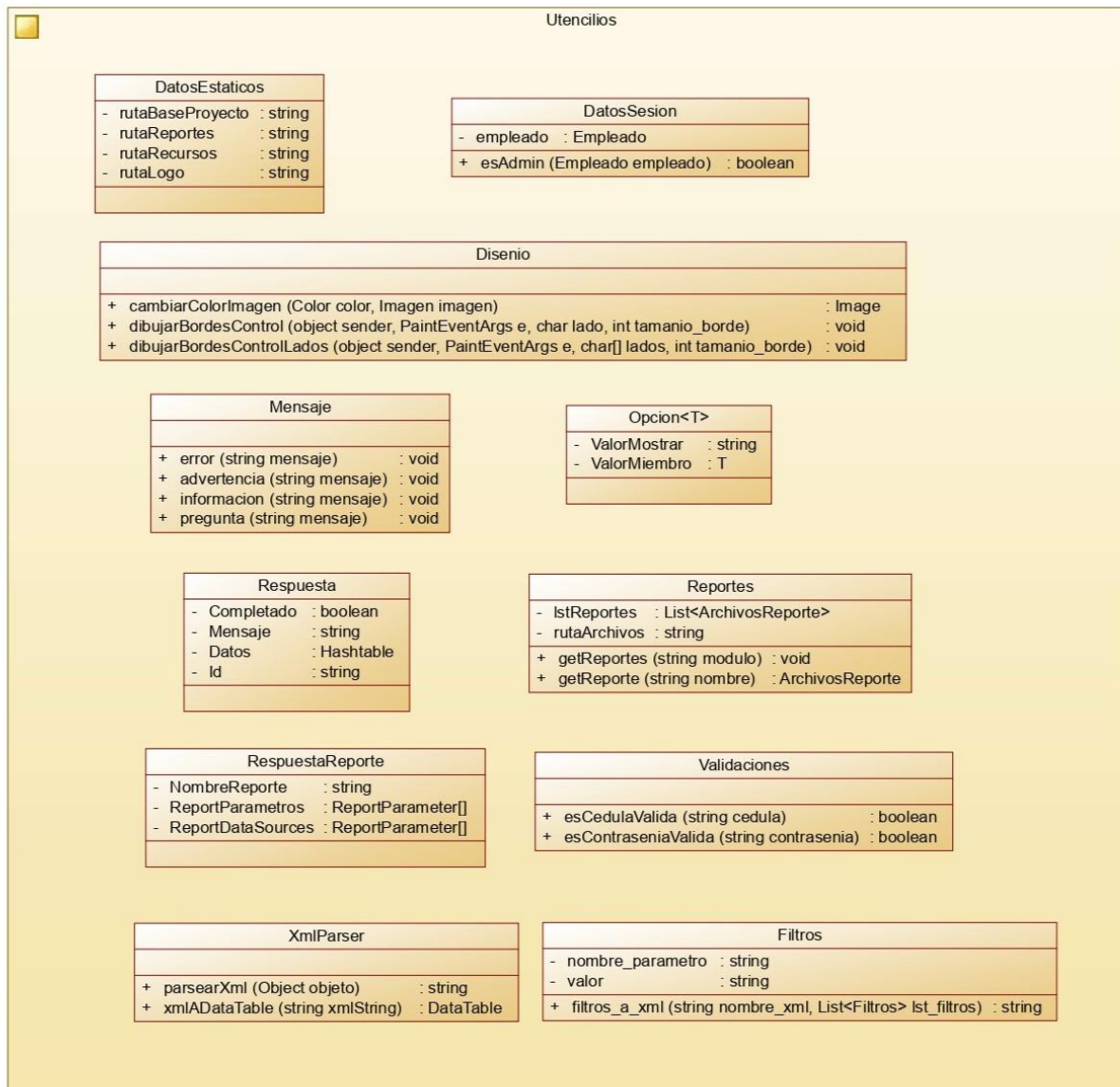




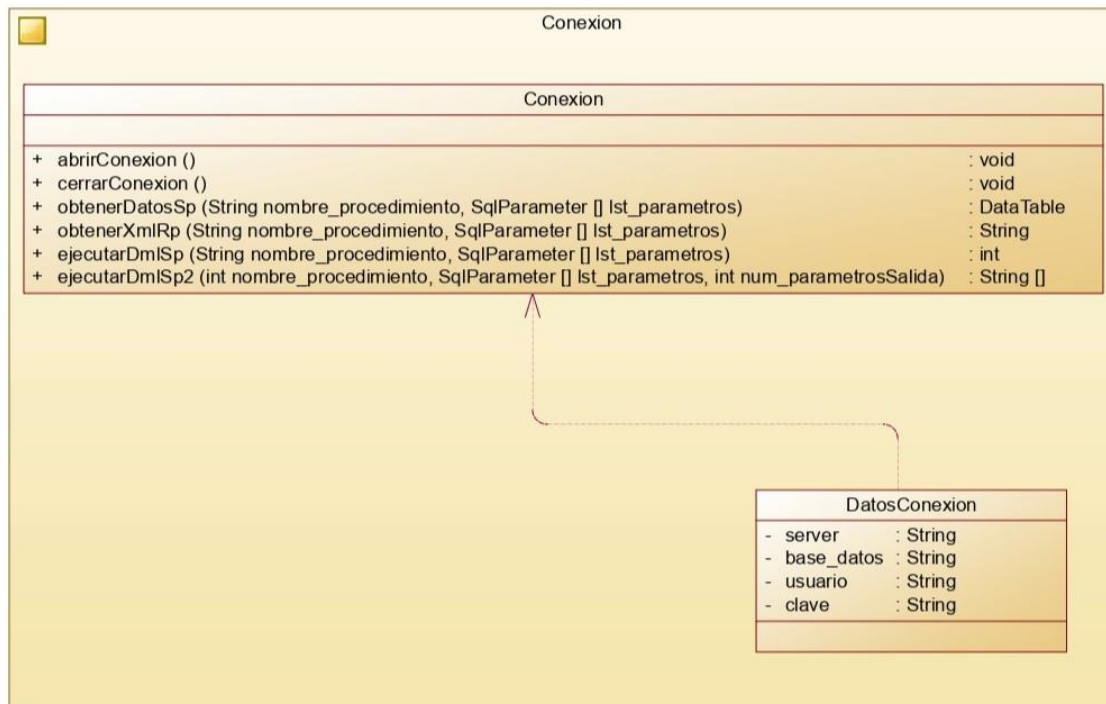
3.10.3. Paquete DAO (Objeto de acceso a datos)



3.10.4. Paquete Utensilios

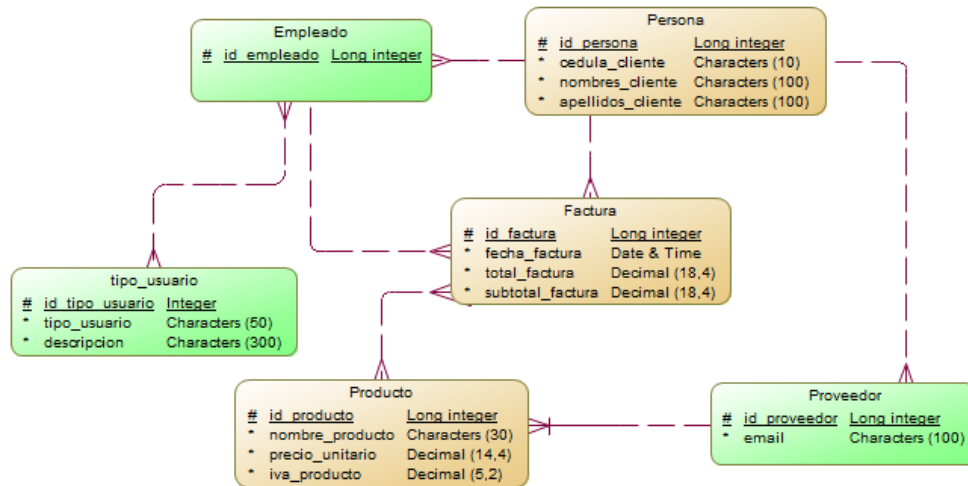


3.10.5. Paquete conexión

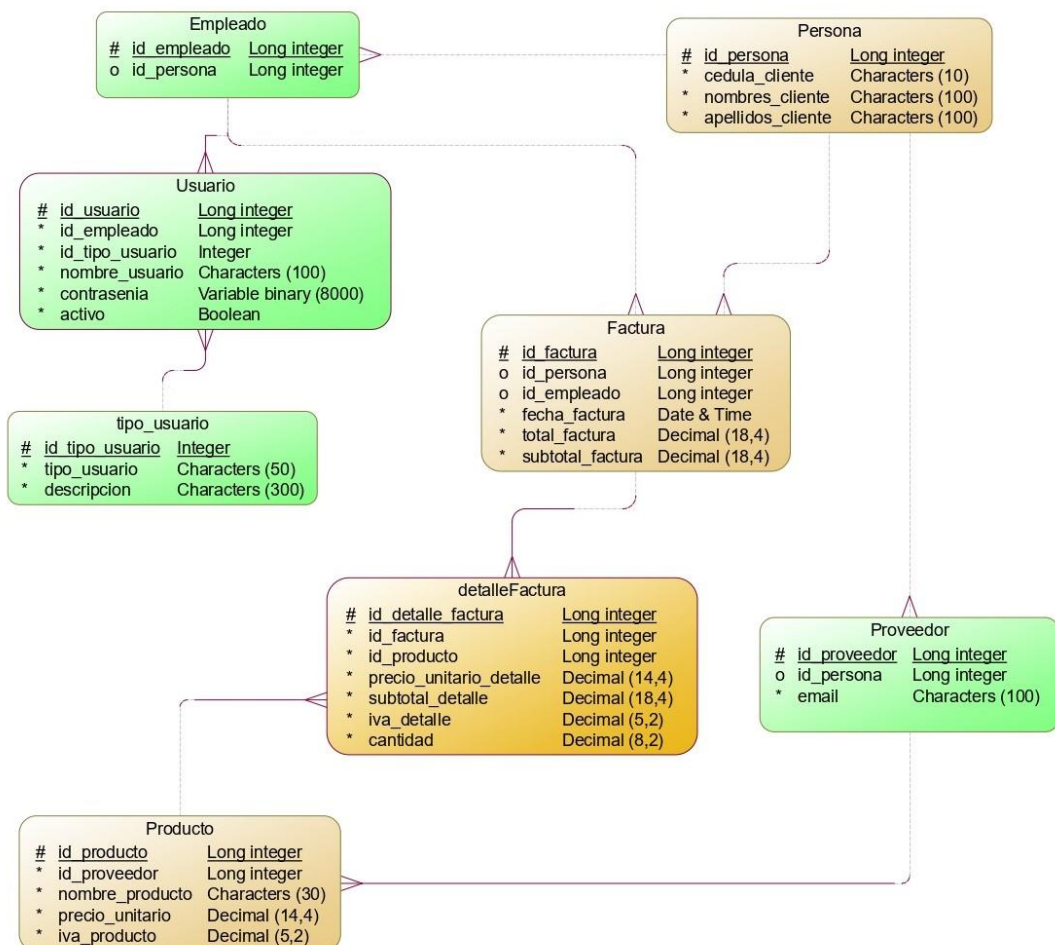


3.11. Diagramas de bases de datos

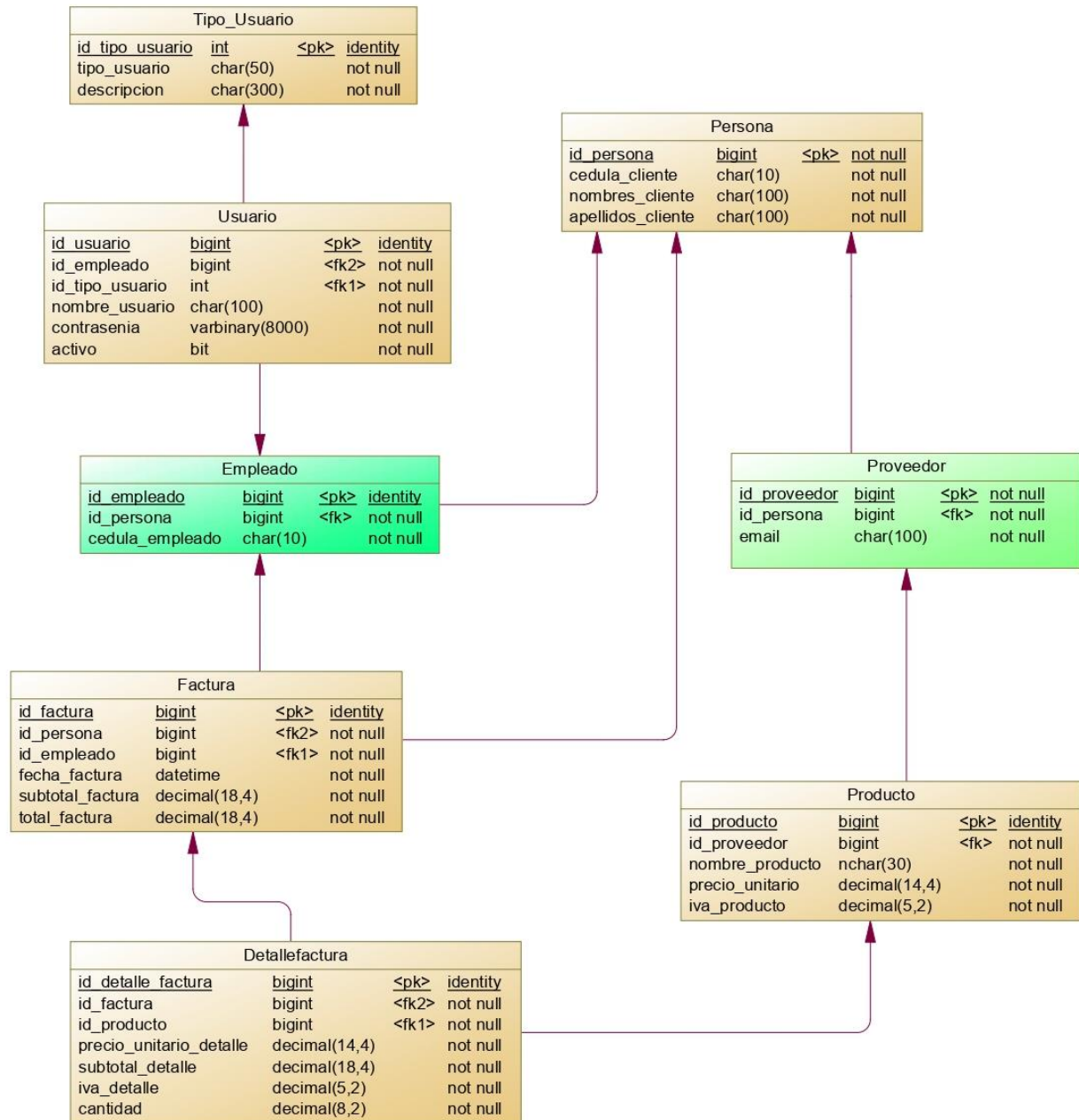
3.11.1. Conceptual



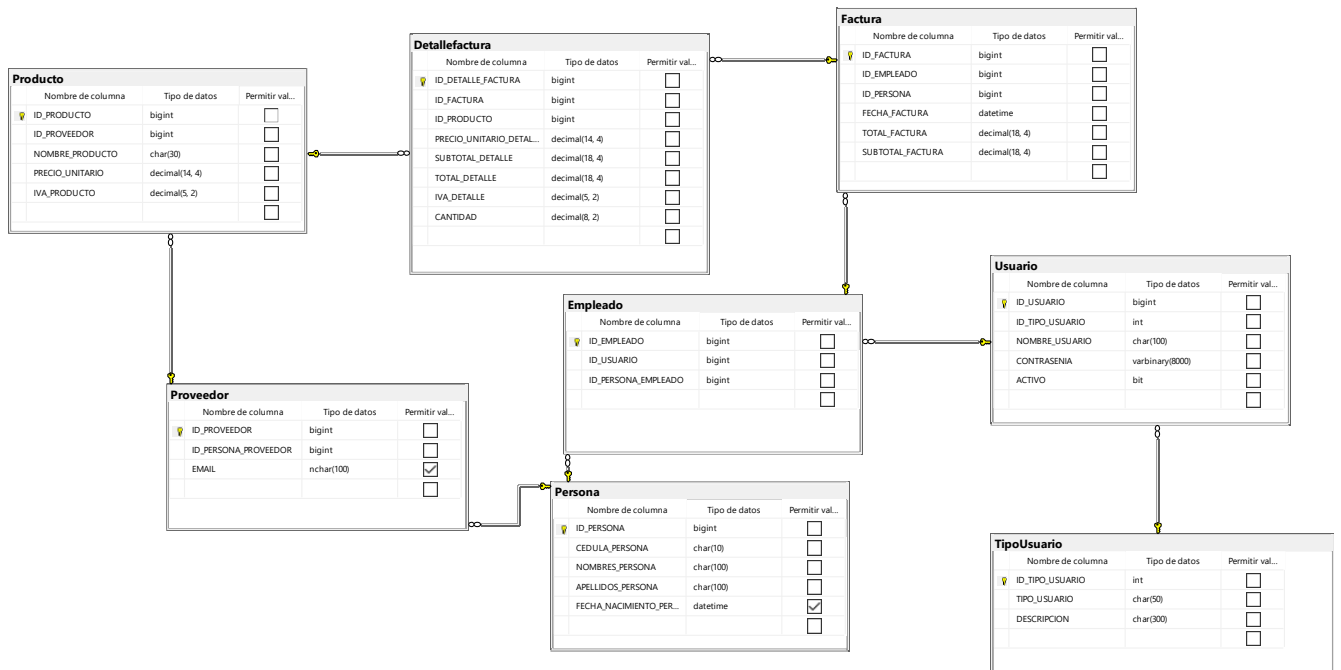
3.11.2. Lógico



3.11.3. Físico

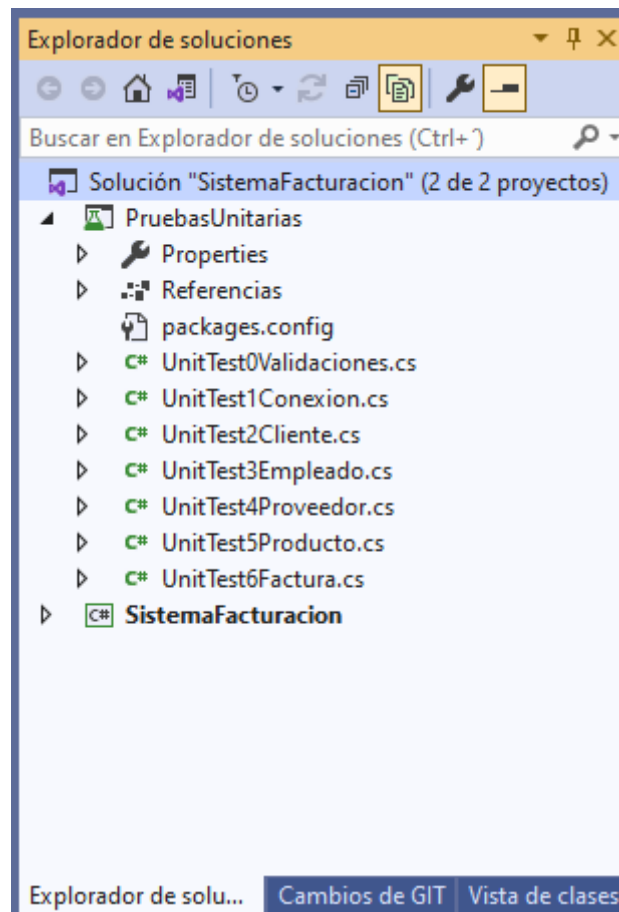


3.11.4. Diagrama en SQL Server



3.12. Pruebas unitarias

Se realizaron una prueba de unitarias para comprobar el correcto funcionamiento de las diferentes funciones que pueden ser realizadas dentro del sistema. Para ejecutar cada una de las actividades de tal forma que se simule el trabajo real se definieron pruebas para cada componente. En la siguiente imagen se puede visualizar la estructura de la solución de “MsTestFramework” denominada “PruebasUnitarias”.



Existe una clase de pruebas unitarias específicas para cada una de las entidades de alta importancia en el sistema. Entre ellas, cliente, empleado, **proveedor**, producto y factura. Además, de pruebas de conexión y validación de datos como la cédula y contraseñas.

Con lo ya especificado se procede a especificar el contenido de cada una de estas clases y el objetivo de cada una de las pruebas.

3.12.1. Pruebas unitarias para validación de datos

La implementación de las pruebas unitarias para validación de datos en el proyecto se centra en evaluar el cumplimiento de las restricciones de la cédula y contraseñas seguras, como se muestra a continuación.

```

[TestClass]
0 referencias
public class UnitTest0Validaciones
{
    [TestMethod]
    [DataRow("Password123!")]
    [DataRow("Contra:2023")]
    [DataRow("ContraseniaD2#")]
    0 referencias
    public void MT_Contrasenia_valida(string contrasenia)
    {
        Assert.IsTrue(Validaciones.esContraseniaValida(contrasenia));
    }

    [TestMethod]
    [DataRow("Password123")]
    [DataRow("Contra2023")]
    [DataRow("ContraseniaD2")]
    [DataRow("!!223123124@")]
    0 referencias
    public void MT_Contrasenia_no_valida(string contrasenia)
    {
        Assert.IsFalse(Validaciones.esContraseniaValida(contrasenia));
    }
}

```

Los dos métodos de prueba presentados en la ilustración anterior evalúan una serie de contraseñas para determinar su validez según los métodos definidos en la clase Validaciones, misma que contiene métodos estáticos. Con esto validamos si los datos que se están enviando al sistema son válidos o no.

```

[TestMethod]
[DataRow("1234569874")]
[DataRow("2350703385")]
[DataRow("1705236521")]
[DataRow("1705236522")]
[DataRow("1705236523")]
[DataRow("1725231524")]
0 referencias
public void MT_Cedula_valida(string cedula)
{
    Assert.IsTrue(Validaciones.esCedulaValida(cedula));
}

[TestMethod]
[DataRow("123456987")]
[DataRow("23507033865")]
[DataRow("17052E6521")]
[DataRow("1@05236522")]
[DataRow("170Q23652E")]
[DataRow("17231524")]
0 referencias
public void MT_Cedula_no_valida(string cedula)
{
    Assert.IsFalse(Validaciones.esCedulaValida(cedula));
}

```

Por su contraparte los dos métodos presentados en esta figura verifican un conjunto de cédulas de identidad para determinar su validez. Eso se realiza mediante las expresiones regulares teniendo una buena verificación antes de que los datos se envíen al sistema.

3.12.2. Pruebas unitarias de conexión

La prueba unitaria para la conexión tiene como objetivo comprobar que la conexión pueda abrirse y cerrarse correctamente evitando posibles excepciones debido a una mala gestión de la apertura y cierre de las conexiones.

```
[TestClass]
0 referencias
public class UnitTest1Conexion
{
    [TestMethod]
    0 referencias
    public void TM_Conexion_abierta()
    {
        Conexion con = new Conexion();
        con.AbrirConexion();

        Assert.IsTrue(con.ConexionAbierta());
        con.CerrarConexion();
    }

    [TestMethod]
    0 referencias
    public void TM_Conexion_cerrada_sin_iniciar()
    {
        Conexion con = new Conexion();
        Assert.IsFalse(con.ConexionAbierta());
    }

    [TestMethod]
    0 referencias
    public void TM_Conexion_cerrada()
    {
        Conexion con = new Conexion();
        con.AbrirConexion();
        con.CerrarConexion();

        Assert.IsFalse(con.ConexionAbierta());
    }
}
```

3.12.3. Pruebas unitarias para entidad clientes

Las pruebas unitarias para cada una de las entidades contienen el flujo completo para insertar, modificar, buscar, obtener y eliminar un registro de la base de datos. A continuación, se muestra los métodos de prueba desarrollados para la entidad cliente.

```

[TestMethod]
0 | 0 referencias
public void TM_1_Insertar_cliente()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = new Cliente();

        cliente.Cedula_cliente = "0000000000";
        cliente.Apellidos = "Apellidos";
        cliente.Nombres = "Nombres";

        Respuesta r = clienteCtrl.insertarCliente(cliente);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Este método de prueba simplemente crea un cliente con una serie de datos ya definidos esperando que el resultado o respuesta sea un valor de verdad, si es así, entonces se cumplirá correctamente el método.

```

[TestMethod]
0 | 0 referencias
public void TM_2_Buscar_cliente()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "cedula_cliente"));
        filtros.Add(new Filtros("texto_buscar", "0000000000"));

        List<Cliente> lstCliente = clienteCtrl.buscarClientes(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstCliente.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}

```

Por otro lado, el método de prueba 2 de la entidad Cliente, realiza una búsqueda del cliente recién creado, como resultado se espera que la lista de clientes resultantes de la búsqueda sea equivalente a 1, especialmente debido a que es una búsqueda por cédula.

```

[TestMethod]
0 | 0 referencias
public void TM_3_Get_cliente_por_cedula()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000000");

        Assert.IsNotNull(cliente);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El método de prueba 3 obtiene el mismo cliente registrado, pero ahora mediante un método específico que garantiza retornar un único objeto de tipo cliente si es que existe. Por esa razón su verificación es considerando si el objeto no es nulo.

```

[TestMethod]
0 | 0 referencias
public void TM_4_Get_cliente_por_id()
{
    bool encontrado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000000");

        if (cliente != null)
        {
            Cliente c = clienteCtrl.getClientePorId(cliente.Id_cliente);
            encontrado = c != null;
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Por contraparte al método anterior, este busca al cliente por el id asociado, debido a que se desconoce el que sea asignado por la base de datos (a pesar de ser incremental), primero se realiza una búsqueda por la cédula que devolverá los datos del cliente y el id se enviará para verificar si existe realmente.

```

[TestMethod]
✓ | 0 referencias
public void TM_5_Modificar_cliente()
{
    bool modificado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000000");

        if (cliente != null)
        {
            cliente.Nombres = "NombresMod";
            cliente.Apellidos = "ApellidosMod";

            Respuesta r = clienteCtrl.modificarCliente(cliente);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El método 5 de la entidad Clientes, realiza la modificación del cliente creado en el método 1. Simplemente cambia el nombre y apellido para luego evaluar si realmente existió alguna modificación. Además, para ello se obtiene el objeto completo mediante la obtención del cliente por su cédula.

```

[TestMethod]
0 | Referencias
public void TM_6_Eliminar_cliente()
{
    bool eliminado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000000");

        if (cliente != null)
        {
            Respuesta r = clienteCtrl.eliminarCliente(cliente.Id_cliente);
            eliminado = r.Completado;
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Finalmente, el último método de prueba de la entidad Cliente, es eliminar, en ella sigue el proceso de los métodos anteriores; que es, obtener el cliente por la cédula y ejecutar la acción correspondiente a eliminarlo.

3.12.4. Pruebas unitarias para entidad empleado

Al igual que los métodos de prueba definidos en las pruebas unitarias de Cliente, en esta entidad se desarrollan las transacciones de consulta y de manipulación de datos. Es importante mencionar que para crear un empleado se requiere definir las credenciales y roles correspondientes según lo especificado en la clase Usuario.

```

public void TM_1_Insertar_empleado()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        //Verificar si ya está registrado
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000001");
        if (cliente != null)
        {
            clienteCtrl.eliminarCliente(cliente.IdPersona);
        }

        Empleado empleado = new Empleado();
        empleado.Cedula = "0000000001";
        empleado.Apellidos = "Apellidos";
        empleado.Nombres = "Nombres";
        empleado.Fecha_Nacimiento = new DateTime(1955, 1, 1).ToString();

        Usuario usuario = new Usuario();
        usuario.Tipo_usuario.Id_tipo_usuario = 1;
        usuario.Nombre_usuario = "_Usuario_";
        usuario.Contrasenia = "123";
        usuario.Activo = true;

        Respuesta r = empleadoCtrl.insertarEmpleado(empleado, usuario);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception e)
    {
        Assert.IsTrue(false);
        throw new Exception(e.Message);
    }
}

```

El primer método de prueba realiza la inserción de un empleado con parámetros ya definidos y con la inicialización de un objeto de tipo usuario para establecer las credenciales asociadas al mismo.

```

[TestMethod]
0 referencias
public void TM_2_Get_empleado_por_cedula()
{
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        Assert.IsNotNull(empleado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Por otro lado, el segundo método de prueba realiza la búsqueda del empleado que coincida con la cédula especificada y creada en el método anterior. creado en el método anterior, como resultado se espera que el objeto empleado no sea nulo.

```

[TestMethod]
0 referencias
public void TM_3_Get_empleado_por_id()
{
    bool encontrado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            Empleado e = empleadoCtrl.getEmpleadoPorId(empleado.Id_empleado);
            encontrado = (e != null);
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El tercer método de prueba permite obtener los datos de un empleado según el id que tenga asociado. Para ello, se utiliza el método de obtener por cédula y así enviar el parámetro requerido al método. Como resultado se espera que el objeto no sea nulo y se haya encontrado.

```

[TestMethod]
0 referencias
public void TM_4_Modificar_empleado()
{
    bool modificado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            empleado = empleadoCtrl.getEmpleadoUsuarioPorIdEmpleado(empleado.Id_empleado);

            empleado.Nombres = "NombresMod";
            empleado.Apellidos = "ApellidosMod";
            empleado.Usuario.Contrasenia = "123";

            Respuesta r = empleadoCtrl.modificarEmpleado(empleado, empleado.Usuario);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El método de prueba 4, modifica los datos de un empleado, cambiando su nombre, apellido y estableciendo su contraseña, puesto que es requerido para validar que los datos sean modificados.

```

[TestMethod]
0 referencias
public void TM_6_Modificar_contrasenia_empleado()
{
    try
    {
        UsuarioCtrl usuarioCtrl = new UsuarioCtrl();
        Respuesta r = usuarioCtrl.modificarContrasenia("_Usuario_", "123", "1234");

        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Así mismo, para modificar la contraseña se requiere de la contraseña actual y la nueva contraseña. Por esta razón se requiere evaluar la validez del método. En los anteriores se

estableció que la contraseña es 123 por lo tanto se espera que el resultado de este método de prueba sea un valor de verdad (true).

```
[TestMethod]
0 referencias
public void TM_6_Modificar_empleado_contraseña_incorrecta()
{
    bool modificado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000001");

        if (empleado != null)
        {
            empleado = empleadoCtrl.getEmpleadoUsuarioPorIdEmpleado(empleado.Id_empleado);

            empleado.Nombres = "NombresMod";
            empleado.Apellidos = "ApellidosMod";
            empleado.Usuario.Contrasenia = "123";

            Respuesta r = empleadoCtrl.modificarEmpleado(empleado, empleado.Usuario);
            modificado = r.Completado;
        }

        Assert.IsFalse(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

También se evalúa la modificación de datos cuando la contraseña es incorrecta. Es decir, en el método de prueba anterior (5) se cambió la contraseña a 1234. Por lo tanto, este método será válido solo si la modificación de los datos no se realiza; o, en otras palabras, es falso.

```

[TestMethod]
0 referencias
public void TM_7_Eliminar_empleado()
{
    bool eliminado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("000000001");

        if (empleado != null)
        {
            Respuesta r = empleadoCtrl.eliminarEmpleado(empleado.Id_empleado);
            eliminado = r.Completado;
        }

        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("000000001");
        if (cliente != null)
        {
            clienteCtrl.eliminarCliente(cliente.IdPersona);
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Finalmente, se realiza el método de prueba para eliminar el registro con el que se trabajó durante el flujo de esta prueba unitaria. Se podrá visualizar que primero se obtiene el empleado y se elimina. Inmediatamente, se requiere del controlador de Cliente puesto que el empleado también es registrado como un cliente, y al eliminar el objeto de empleado es necesario eliminar el objeto de tipo Cliente.

3.12.5. Pruebas unitarias para entidad proveedor

La nueva entidad proveedor lleva una relación directa con productos; por lo tanto, se realizarán y detallarán en primera instancia los métodos de consulta y de manipulación de datos de esta entidad.

```

[TestMethod]
0 | 0 referencias
public void TM_1_Insertar_proveedor()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = new Proveedor();

        proveedor.Cedula = "0000000002";
        proveedor.Apellidos = "Apellidos";
        proveedor.Nombres = "Nombres";
        proveedor.Email = "prueba_proveedor@gmail.com";
        proveedor.Fecha_Nacimiento = new DateTime(2002, 10, 12).ToString();

        Respuesta r = proveedorCtrl.insertarProveedor(proveedor);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El primer método de prueba es la inserción del proveedor, en este se definen datos fijos como la cédula, apellidos, nombres, email y la fecha de nacimiento. Posterior a ello, se procede a realizar la inserción mediante el método correspondiente. Como resultado, se espera que se haya insertado el proveedor.

```

[TestMethod]
0 | 0 referencias
public void TM_2_Buscar_proveedor()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "cedula"));
        filtros.Add(new Filtros("texto_buscar", "0000000002"));

        List<Proveedor> lstCliente = proveedorCtrl.buscarProveedores(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstCliente.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}

```

El segundo método de prueba realiza la búsqueda de un proveedor por su cédula. En este se espera obtener un único objeto en la lista, mismo que fue ingresado en el método de prueba anterior.

```

[TestMethod]
✓ | 0 referencias
public void TM_3_Get_proveedor_por_cedula()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        Assert.IsNotNull(proveedor);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Por otro lado, el método de prueba 3, realiza la búsqueda del mismo proveedor, pero ahora lo realiza específicamente por la cédula con la que fue creada. Como resultado, se espera que el objeto no sea nulo.

```

[TestMethod]
✓ | 0 referencias
public void TM_4_Get_proveedor_por_id()
{
    bool encontrado = false;
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        if (proveedor != null)
        {
            proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);
            encontrado = proveedor != null;
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El cuarto método de prueba tiene como objetivo obtener datos del mismo proveedor creado, pero por el id correspondiente. Para ello, se usa el método de buscar por cédula y así poder obtener la cédula para enviarla como parámetro. Como resultado, se espera que se encuentre el proveedor.

```

[TestMethod]
✓ | 0 referencias
public void TM_5_Modificar_proveedor()
{
    bool modificado = false;
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        if (proveedor != null)
        {
            proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);
            proveedor.Nombres = "NombresMod";
            proveedor.Apellidos = "ApellidosMod";

            Respuesta r = proveedorCtrl.modificarProveedor(proveedor);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El quinto método de prueba de la entidad proveedor, obtiene principalmente los datos del proveedor creado y realiza cambios en el nombre y apellido. Luego, se espera que se haya modificado correctamente obteniendo un valor de verdad (true).

```

[TestMethod]
✓ | 0 referencias
public void TM_6_Eliminar_proveedor()
{
    bool eliminado = false;
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        if (proveedor != null)
        {
            proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);

            Respuesta r = proveedorCtrl.eliminarProveedor(proveedor.IdPersona);
            eliminado = r.Completado;
        }

        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000002");
        if (cliente != null)
        {
            clienteCtrl.eliminarCliente(cliente.IdPersona);
        }
        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

Finalmente, se realiza la eliminación del proveedor. Al igual que la “entidad empleado” debido a que esto se trata de pruebas unitarias y no de un flujo en su completitud, posterior a la obtención de los datos del proveedor y su eliminación, es necesario eliminar los datos de la tabla que contiene todos los objetos de tipo Cliente o más específicamente Persona.

3.12.6. Pruebas unitarias para entidad productos

La clase de pruebas unitarias para la entidad productos tenemos varios métodos que nos ayudan a realizar las operaciones básicas de CRUD para de esta manera verificar el correcto funcionamiento de los procesos.

En el siguiente método podemos observar que tenemos varios campos en los cuales están insertar producto con un proveedor determinado. Claro que este proveedor se lo crea en el instante para verificar la correcta relación de las tablas.

```

[TestMethod]
0 referencias
public void TM_1_Insertar_producto()
{
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = new Proveedor();

        proveedor.Cedula = "0000000002";
        proveedor.Apellidos = "Apellidos";
        proveedor.Nombres = "Nombres";
        proveedor.Fecha_Nacimiento = new DateTime(2002, 10, 12).ToString();

        proveedorCtrl.insertarProveedor(proveedor);
        proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        ProductoCtrl productoCtrl = new ProductoCtrl();
        Producto producto = new Producto();

        producto.Nombre_producto = "00Producto00";
        producto.Iva = 0;
        producto.Precio_unitario = 0;
        producto.Proveedor = proveedor;

        Respuesta r = productoCtrl.insertarProducto(producto);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

En este método tenemos el proceso de buscar que lo realiza mediante filtros. Elegimos el texto que deseamos buscar seleccionando mediante el cual campo lo queremos hacer la búsqueda. Si los registros retornados son igual a uno es porque el método devolvió un resultado correcto caso contrario no.

```

[TestMethod]
0 referencias
public void TM_2_Buscar_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_búsqueda", "nombre_producto"));
        filtros.Add(new Filtros("texto_buscar", "00Producto00"));

        List<Producto> lstProducto = productoCtrl.buscarProductoPaginacion(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstProducto.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
    }
}

```

En el siguiente método realizamos una operación similar a la anterior, pero con la diferencia que en esta estamos comparando si el producto que se ha buscado no es nulo es decir que existe dentro del sistema.

```

[TestMethod]
O|Referencias
public void TM_3_Get_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "nombre_producto"));
        filtros.Add(new Filtros("texto_buscar", "00Producto00"));

        Producto producto = productoCtrl.buscarProductoPaginacion(1, 30, Filtros.filtros_a_xml("Filtros", filtros))[0];
        producto = productoCtrl.getProducto(producto.Id_producto);

        Assert.IsNotNull(producto);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

En el siguiente método estamos realizando un conjunto de operaciones, búsqueda y modificación de datos. En este método primero se obtiene un producto de la base de datos mediante el método de búsqueda. De acuerdo con esto se selecciona la primera posición del producto devuelto y con ello se modifican los campos nombres, IVA y precio unitario. Finalmente, el producto se actualiza mediante el método “modificarProducto”.

```

[TestMethod]
O|Referencias
public void TM_4_Modificar_producto()
{
    bool modificado = false;
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        List<Producto> lstProducto = productoCtrl.buscarProducto("00Producto00");

        if (lstProducto.Count > 0)
        {
            Producto producto = lstProducto[0];
            producto.Nombre_producto = "00Producto00Mod";
            producto.Iva = 0;
            producto.Precio_unitario = 0;

            Respuesta r = productoCtrl.modificarProducto(producto);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

En el siguiente proceso vemos el método de eliminación de producto en la cual primero obtenemos el producto que queremos eliminar y posteriormente verificamos si existe si la lista tiene un tamaño mayor a cero. Luego lo eliminamos si existe. Cabe resaltar que en esta verificación comprobamos si la eliminación de realizo correctamente es decir si retorna un resultado verdadero.

```
[TestMethod]
0 referencias
public void TM_5_Eliminar_producto()
{
    bool eliminado = false;
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        List<Producto> lstProducto = productoCtrl.buscarProducto("00Producto00Mod");

        if (lstProducto.Count > 0)
        {
            Producto producto = lstProducto[0];

            Respuesta r = productoCtrl.eliminarProducto(producto.Id_producto);
            eliminado = r.Completado;
        }

        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        proveedor = proveedorCtrl.getProveedorPorId(proveedor.IdProveedor);
        proveedorCtrl.eliminarProveedor(proveedor.IdPersona);

        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientePorCedula("0000000002");
        clienteCtrl.eliminarCliente(cliente.IdPersona);

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

3.12.7. Pruebas unitarias para entidad factura

Para las pruebas unitarias de Factura, se utilizó la notación initialize para registrar un cliente, **proveedor**, producto y empleado para poder realizar las acciones de consulta y manipulación de datos de la factura. A continuación, se muestra el proceso.

```

[ClassInitialize]
0 referencias
public static void Initialize(TestContext context)
{
    try
    {
        //Insertar un empleado y usuario para las pruebas
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        Empleado empleado = new Empleado();
        empleado.Cedula = "0000000001";
        empleado.Apellidos = "Apellidos";
        empleado.Nombres = "Nombres";
        empleado.Fecha_Nacimiento = new DateTime(1955, 1, 1).ToString();

        Usuario usuario = new Usuario();
        usuario.Tipo_usuario.Id_tipo_usuario = 1;
        usuario.Nombre_usuario = "_Usuario_";
        usuario.Contrasenia = "123";
        usuario.Activo = true;

        empleadoCtrl.insertarEmpleado(empleado, usuario);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar empleado: {ex.Message}");
    }

    //Insertar un cliente
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = new Cliente();

        cliente.Cedula = "0000000000";
        cliente.Apellidos = "Apellidos";
        cliente.Nombres = "Nombres";

        clienteCtrl.insertarCliente(cliente);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar cliente: {ex.Message}");
    }

    //Insertar un producto
    try
    {
        ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
        Proveedor proveedor = new Proveedor();

        proveedor.Cedula = "0000000002";
        proveedor.Apellidos = "Apellidos";
        proveedor.Nombres = "Nombres";
        proveedor.Fecha_Nacimiento = new DateTime(2002, 10, 12).ToString();

        proveedorCtrl.insertarProveedor(proveedor);
        proveedor = proveedorCtrl.getProveedorPorCedula("0000000002");

        id_proveedor = proveedor.IdProveedor;

        ProductoCtrl productoCtrl = new ProductoCtrl();
        Producto producto = new Producto();

        producto.Nombre_producto = "00Producto00";
        producto.Iva = 0;
        producto.Precio_unitario = 0;
        producto.Proveedor = proveedor;

        productoCtrl.insertarProducto(producto);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}

```

Llegados a este punto se comprobó el funcionamiento de todas las entidades necesarias para llegar al proceso de facturación. Por esta razón, se inicializa la clase con un conjunto de objetos requeridos para su desarrollo.

```
[TestMethod]
0 | 0 referencias
public void TM_1_Insertar_factura()
{
    try
    {
        FacturaCtrl facturaCtrl = new FacturaCtrl();

        Factura factura = new Factura();
        FacturaDetalle fDetalle = new FacturaDetalle();

        EmpleadoCtrl eCtrl = new EmpleadoCtrl();
        Empleado empleadoPrueba = eCtrl.getEmpleadoPorCedula("0000000001");

        ProductoCtrl pCtrl = new ProductoCtrl();
        Producto productoPrueba = pCtrl.buscarProducto("00Producto00")[0];

        ClienteCtrl cCtrl = new ClienteCtrl();
        Cliente clientePrueba = cCtrl.getClientesPorCedula("0000000000");

        factura.Cliente = clientePrueba;
        factura.Empleado = empleadoPrueba;
        fDetalle.Producto = productoPrueba;
        fDetalle.Cantidad = 0;

        factura.Factura_Detalle.Add(fDetalle);
        Respuesta r = facturaCtrl.insertarFactura(factura);

        id_factura = int.Parse(r.Id);
        id_empleado = empleadoPrueba.Id_empleado;
        id_producto = productoPrueba.Id_producto;

        Assert.IsTrue(r.Completado);
    }
    catch (Exception ex)
    {
        Assert.IsFalse(true);
        throw new Exception($"Error al insertar producto: {ex.ToString()}");
    }
}
```

La inserción de la factura es el primer método de prueba definido para la entidad Factura, en ella se realiza la búsqueda de cada uno de los objetos según lo establecido en la inicialización (mostrada en la ilustración anterior). Una vez se obtienen los datos requeridos, se asignan los valores al objeto de factura detalle y se agregan a la Factura. Así, como resultado se espera que se haya registrado la Factura obtienen un valor de verdad (true).

```

[TestMethod]
0 | 0 referencias
public void TM_2_Buscar_facturas()
{
    try
    {
        FacturaCtrl facturaCtrl = new FacturaCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "id_factura"));
        filtros.Add(new Filtros("texto_buscar", id_factura.ToString()));
        filtros.Add(new Filtros("fecha_inicio", DateTime.Now.ToString("MM-dd-yyyy")));
        filtros.Add(new Filtros("fecha_fin", DateTime.Now.ToString("MM-dd-yyyy")));

        List<Factura> facturas = facturaCtrl.buscarFacturas(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(facturas.Count == 1);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

El segundo y último método de prueba de la entidad Factura es realizar la búsqueda de la factura recién ingresada. Para ello, busca el id de la factura registrada, y que la fecha de registro sea en el momento en el que se ejecutan las pruebas. Luego se ejecuta el método solicitando la primera página de la paginación con 30 elementos, y se espera que la longitud de la lista de objetos de tipo factura sea igual a 1.

[ClassCleanup]

0 referencias

public static void Cleanup()

```
{
    //Eliminar el cliente, producto, empleado y factura utilizado en las pruebas
    //Eliminar factura y su detalle
    FacturaCtrl facturaCtrl = new FacturaCtrl();
    facturaCtrl.eliminarFactura(id_factura);

    //Eliminar el empleado
    EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
    empleadoCtrl.eliminarEmpleado(id_empleado);

    //Eliminar el producto
    ProductoCtrl productoCtrl = new ProductoCtrl();
    productoCtrl.eliminarProducto(id_producto);

    //Eliminar el proveedor
    ProveedorCtrl proveedorCtrl = new ProveedorCtrl();
    proveedorCtrl.eliminarProveedor(id_proveedor);

    //Eliminar el cliente
    ClienteCtrl clienteCtrl = new ClienteCtrl();
    Cliente clientePrueba = clienteCtrl.getClientePorCedula("0000000000");
    clienteCtrl.eliminarCliente(clientePrueba.IdPersona);

    clientePrueba = clienteCtrl.getClientePorCedula("0000000001");
    clienteCtrl.eliminarCliente(clientePrueba.IdPersona);

    clientePrueba = clienteCtrl.getClientePorCedula("0000000002");
    clienteCtrl.eliminarCliente(clientePrueba.IdPersona);
}
```

Finalmente, para eliminar los datos se ejecuta la función Cleanup(), misma que se ejecuta automáticamente al terminar de evaluar cada una de las pruebas unitarias de la clase de Factura.

Así, al ejecutar las pruebas unitarias, se obtienen los siguientes resultados.

Explorador de pruebas		UnitTest5Producto.cs	UnitTest4Proveedor.cs
		33	33
		0	0
Prueba	Duración	Rasgos	M...
PruebasUnitarias (33)	3,9 s		
PruebasUnitarias (33)	3,9 s		
UnitTest0Validaciones (4)	85 ms		
UnitTest1Conexion (3)	420 ms		
UnitTest2Cliente (6)	824 ms		
UnitTest3Empleado (7)	743 ms		
UnitTest4Proveedor (6)	638 ms		
UnitTest5Producto (5)	966 ms		
UnitTest6Factura (2)	197 ms		

O de forma específica.

Explorador de pruebas		UnitTest5Producto.cs	UnitTest4Proveedor.cs
		33	33
		0	0
Prueba	Duración	Rasgos	M...
PruebasUnitarias (33)	3,9 s		
PruebasUnitarias (33)	3,9 s		
UnitTest0Validaciones (4)	85 ms		
MT_Cedula_no_valida	72 ms		
MT_Cedula_valida	< 1 ms		
MT_Contrasenia_no_valida	13 ms		
MT_Contrasenia_valida	< 1 ms		
UnitTest1Conexion (3)	420 ms		
TM_Conexion_abierta	418 ms		
TM_Conexion_cerrada	2 ms		
TM_Conexion_cerrada_sin_i...	< 1 ms		
UnitTest2Cliente (6)	824 ms		
TM_1_Insertar_cliente	381 ms		
TM_2_Buscar_cliente	108 ms		
TM_3_Get_cliente_por_cedula	22 ms		
TM_4_Get_cliente_por_id	40 ms		
TM_5_Modificar_cliente	194 ms		
TM_6_Eliminar_cliente	79 ms		
UnitTest3Empleado (7)	743 ms		
TM_1_Insertar_empleado	95 ms		
TM_2_Get_empleado_por_c...	43 ms		
TM_3_Get_empleado_por_id	43 ms		
TM_4_Modificar_empleado	133 ms		
TM_5_Modificar_contraseni...	73 ms		
TM_6_Modificar_empleado_...	62 ms		
TM_7_Eliminar_empleado	294 ms		
UnitTest4Proveedor (6)	638 ms		
TM_1_Insertar_proveedor	181 ms		
TM_2_Buscar_proveedor	23 ms		
TM_3_Get_proveedor_por_c...	26 ms		
TM_4_Get_proveedor_por_id	40 ms		
TM_5_Modificar_proveedor	151 ms		
TM_6_Eliminar_proveedor	217 ms		
UnitTest5Producto (5)	966 ms		
TM_1_Insertar_producto	269 ms		
TM_2_Buscar_producto	44 ms		
TM_3_Get_producto	92 ms		
TM_4_Modificar_producto	150 ms		
TM_5_Eliminar_producto	411 ms		
UnitTest6Factura (2)	197 ms		
TM_1_Insertar_factura	165 ms		
TM_2_Buscar_facturas	32 ms		

3.13. Repositorio

Enlace: <https://github.com/jhonleturne192005/GCSfacturacion/tree/V1.2T>

 **GCSfacturacion** Public

Watch 1 Fork 0 Star 0

V1.2T 6 Branches 0 Tags Add file Code

This branch is 2 commits ahead of, 4 commits behind `Base`.

Contribute

 Hecmi Create README.md	a287304 · 3 minutes ago	2 Commits
 Backup	Inserción de repositorio completo	6 minutes ago
 Código fuente	Inserción de repositorio completo	6 minutes ago
 Diagramas - Power Designer	Inserción de repositorio completo	6 minutes ago
 Documentación de pruebas unitarias	Inserción de repositorio completo	6 minutes ago
 Línea de referencia	Inserción de repositorio completo	6 minutes ago
 Orden de cambio de Ingeniería	Inserción de repositorio completo	6 minutes ago
 PruebasUnitarias	Inserción de repositorio completo	6 minutes ago
 Solicitud de cambios por parte del cliente	Inserción de repositorio completo	6 minutes ago
 LÉEME.txt	Inserción de repositorio completo	6 minutes ago
 README.md	Create README.md	3 minutes ago

 **README**

No description, website, or topics provided.

 Readme

 Activity

 0 stars

 1 watching

 0 forks

[Report repository](#)

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 2

 **Hecmi** Héctor Casanova

 **jhonleturne192005 theblack**